# Appunti ingegneria del software

## Introduzione alla SE moderna

**Ingegneria del software oggi:** Quando si parla di software, si parla della struttura portante dei sistemi moderni

- trasporti
- comunicazione
- finanza

E i problemi che genera quando fallisce sono pericolosamente alti

- interruzione dei servizi essenziali
- problema di sicurezza e privacy
- costi economici e perdita di fiducia degli utenti

Per **L'ingegneria del software** non si parla solo di programmare, ma serve un approccio **ingegneristico** per:

- Progettare
- costruire
- mantenere

Sistemi complessi e duraturi

**Quando il software fallisce:** I problemi, oltre ad essere "bug fastidiosi", creano anche dei problemi reali

- sistemi di prenotazioni in down durante i picchi di accesso
- piattaforme di pagamento non disponibili
- errori nei sistemi di controllo
- perdita o esposizione dei dati sensibili degli utenti

E in questi casi la risoluzione non consiste solo nella correzione di una riga ma può prevedere anche la ri-progettazione dell'intero sistema. L'**Ingegneria del software** nasce per ridurre la probabilità che questi scenari si verifichino.

**Progettare sistemi complessi:** Nei sistemi reali entrano in gioco diversi fattori

- molti operatori coinvolti
- vincoli tecnici e organizzativi
- grandi moli di dati
- interazione tra componenti eterogenei
- requisiti che evolvono
- tecnologia che cambia
- contesti d'uso diversificati

L'Ingegneria del Software nasce per gestire questa complessità e fornire strumenti, metodi e pratiche per affrontare lo sviluppo in modo sistematico.

**Programmazione $\neq$ ingegneria del software:** La scrittura del codice è solo una delle attività dello sviluppo software e rappresenta una porzione limitata del lavoro complessivo. Prima dell'implementazione c'è la fase di analisi e comprensione del problema:

- definizione dei requisiti
- identificazione dei vincoli
- analisi del dominio.

Dopo l'implementazione inizia un percorso ancora più lungo:

- verifica del comportamento
- correzione dei difetti
- gestione dei cambiamenti
- evoluzione del sistema nel tempo.

L'Ingegneria del Software tiene insieme tutto questo, garantendo che il sistema rimanga comprensibile e modificabile nel tempo, indipendentemente da chi scrive il codice.

**Che cos'è l'ingegneria del software:** L'Ingegneria del Software è l'insieme di principi, metodi e strumenti per sviluppare e mantenere sistemi software affidabili e utili in modo economicamente sostenibile. L'attività di sviluppo si suddivide principalmente in 4 processi:

- **Specifica:** capire che cosa il software deve fare e quali vincoli deve rispettare.
- **Sviluppo:** progettare l'architettura e le strutture interne, scrivere il codice.
- **Validazione:** verificare che il sistema soddisfi davvero i requisiti attesi.
- **Evoluzione:** adattare il software a nuove esigenze, correzioni, cambiamenti di contesto.

**Il costo reale del software è nella manutenzione** Nei progetti reali, la fase iniziale di sviluppo rappresenta solo una parte del costo complessivo. Ma è la **manutenzione** che assorbe la maggior parte delle risorse

- correzione dei bug
- aggiornamenti richiesti
- adattamenti a normative e tecnologie
- estensioni e miglioramenti continui

La sostenibilità del software dipende dalla qualità delle scelte fatte nelle fasi iniziali. Un buon progetto facilita l'evoluzione, mentre un progetto fragile la rende costosa e rischiosa.

**Ciclo di vita:** Il **ciclo di vita** fornisce un quadro concettuale per organizzare le attività

- raccolta dei requisiti
- progettazione soluzione
- implementazione
- verifica
- rilascio
- manutenzione

1. Le attività iniziali seguono un flusso lineare: requisiti → progettazione → implementazione.
2. Dopo la prima versione, il processo entra in un ciclo continuo di test, rilascio e manutenzione.
3. Le attività della seconda riga sono iterative: ogni modifica o bug-fix riattiva il ciclo.
4. La manutenzione genera spesso nuovi requisiti, chiudendo il loop verso la fase iniziale.

**Comprensione del problema:** La costruzione del software deve partire da una reale **comprensione** del problema da risolvere e delle esigenze degli utenti.

- Chi userà il sistema?
- Con quali obiettivi?
- In quali contesti?
- Con quali vincoli?

Un'analisi affrettata porta a sviluppare software poco utile o costoso da correggere; dedicare tempo alla comprensione iniziale evita errori di fondo.

**Nessun progetto procede in linea retta:** Durante lo sviluppo emergono inevitabilmente nuove informazioni e nuove esigenze, che mettono in discussione decisioni prese in precedenza. Il processo reale è **iterativo**

- si torna sui requisiti
- si rivede il design
- si modifica il codice
- si aggiornano i test.

L'ingegneria del software non elimina queste iterazioni, ma fornisce strumenti per gestirle in modo controllato e produttivo.

**La qualità non è un optional:** Un sistema software non deve solo funzionare: deve funzionare bene, con:

- affidabilità
- efficienza
- semplicità d'uso
- capacità di evolvere nel tempo

La qualità si manifesta su due livelli:

- **Front-end:** esperienza d'uso, chiarezza, prestazioni
- **Back-end:** architettura, struttura del codice, dipendenze, documentazione

Se la qualità interna è scarsa, il sistema diventa fragile e difficile da modificare, indipendentemente da come appare all' esterno.

**Le figure presenti nello sviluppo di un progetto:** Un sistema software è il risultato del lavoro di molte figure diverse con ruoli differenti e competenze specifiche.

- **utenti e committenti:** definiscono bisogni e obiettivi
- **analisti e progettisti:** traducono le esigenze in modelli e architetture
- **sviluppatori e tester:** costruiscono e verificano il sistema
- **manutentori:** garantiscono la continuità del servizio nel tempo

La collaborazione tra questi ruoli è un fattore determinante per il successo del progetto. Ogni ruolo ha una prospettiva diversa sul sistema e l'ingegneria del software deve farle convivere in modo coerente.

- **l'utente** si concentra sull'esperienza d'uso e sulla semplicità delle operazioni.
- **lo sviluppatore** guarda alla correttezza e alla struttura del codice.
- **l'architetto** pensa alla coerenza strutturale e alle dipendenze tra moduli;
- **il manutentore** si preoccupa della comprensibilità interna e della facilità di modifica.

Integrare queste prospettive è fondamentale per ottenere un sistema davvero efficace.

**Quando la comunicazione fallisce, fallisce il progetto:** Molti difetti e ritardi non derivano da problemi tecnici, ma da incomprensioni tra le persone coinvolte.

- requisiti fraintesi o ambigui
- aspettative non esplicitate
- decisioni progettuali non documentate
- informazioni critiche che non vengono condivise

Una Comunicazione efficace è **parte integrante dell'ingegneria del software**, tanto quanto il codice.

**Il rischio è parte naturale di ogni progetto:** Ogni decisione progettuale e tecnologica introduce incertezza.

- scelte tecnologiche premature o non verificate.
- requisiti instabili o in continuo cambiamento.
- integrazioni difficili o vincoli non previsti.
- stime errate di tempi e complessità.

L'ingegneria del software non elimina i rischi ma li gestisce in modo da correggerlo in tempo.

**La complessità è il vero avversario del software:** La difficoltà principale nello sviluppo del software non risiede nelle singole funzionalità, ma nell'interazione tra molte parti diverse. La complessità non è solo quantitativa, ma anche qualitativa:

- dipendenze poco chiare
- eccezioni alla regola
- comportamenti impliciti
- interazioni inaspettate.

Senza un metodo rigoroso, la complessità rende ogni modifica rischiosa e tende a far degenerare il sistema nel tempo. Anche se le modifiche sembrano piccole e locali, la complessità interna del sistema tende a crescere in modo non lineare nel corso dei rilasci, soprattutto se mancano buone pratiche di progettazione, pattern, principi SOLIDI e test automatici. L'ingegneria del software offre strumenti per mantenere comprensibile un sistema complesso.

- architetture modulari, che dividono il sistema in parti coerenti
- modelli chiari, che permettono di ragionare sul dominio e sulle interazioni
- responsabilità ben definite, che evitano sovrapposizioni e ambiguità
- documentazione mirata, che supporta la comprensione senza appesantire.

**Obiettivo dell' ingegnere del software:** L'obbiettivo dell'ingegneria del software si pone come obbiettivo finale in un progetto la **riduzione dell'incertezza** durante lo sviluppo

- prevedere i tempi e i costi necessari

- valutare diverse alternative prima dell'implementazione
- rilevare problemi in anticipo
- mantenere il controllo anche in presenza di cambiamenti

L'approccio ingegneristico non elimina gli imprevisti, ma rende possibile affrontarli con metodo senza perdere stabilità.

## Processi software

Un' attività collettiva che coinvolge persone, ruoli e decisioni distribuite nel tempo. Quando si lavora con più persone, ognuna tende a seguire il proprio stile, creando

- fraintendimenti
- funzioni duplicate nel codice
- parti diverse del sistema che lavorano incoerentemente

Un **Processo software** fornisce una struttura comune per decidere come:

- procedere
- coordinarsi
- verificare il tutto in maniera sistematica

L'obbiettivo è ridurre il caos tra i vari team, e le persone all'interno dei vari team. Si devono stilare delle regole per agire coerentemente e in maniera ordinata, sfruttando così al massimo la potenzialità del gruppo

**Dal ciclo di vita al software:** Nella pratica ci si chiede:

- in quale ordine vanno affrontate le attività
- quanto approfondire ciascuna fase
- quanto spesso tornare indietro per rivedere le decisioni già prese

I processi software rispondono proprio a queste domande:

- sono strategie organizzative per combinare le stesse fasi del ciclo di vita (waterfall, incrementale, spirale, agile, ... )
- scegliamo il processo in base al contesto e al livello di rischio / incertezza

**Definizione (di processo):** Un insieme strutturato di attività, ruoli e artefatti che guidano la costruzione e l'evoluzione di un software. In un processo si specifica:

- chi è responsabile di questa cosa
- quali passi devono essere compiuti e in quale sequenza
- quali prodotti intermedi devono essere generati (documenti, modelli, codice, test, ... )

L'obiettivo non è aggiungere burocrazia, ma garantire che il progetto proceda in modo coerente, tracciabile e comprensibile per tutti i soggetti coinvolti.

**Perché serve anche ai buoni programmatori?** Nei piccoli progetti individuali il programmatore può controllare tutto “nella propria testa”, ma questa illusione sparisce appena:

- il numero di funzionalità cresce
- entra in gioco un team
- il software deve essere mantenuto nel tempo

Senza un processo, le decisioni restano implicite e si perdono facilmente:

- diventa difficile capire perché certe scelte sono state fatte
- diventa rischioso modificare parti del sistema già rilasciate

Un processo aiuta a rendere esplicito il modo di lavorare, in modo che qualità e conoscenza non dipendano dai singoli individui ma dal team.

**Attività comuni a tutti i processi:** Qualunque sia il processo adottato, ritroveremo sempre alcune attività fondamentali:

- specifica: capire che cosa il sistema deve fare e quali vincoli deve rispettare;

- sviluppo: progettare e implementare la soluzione;
- validazione: verificare che il sistema soddisfi i requisiti e si comporti come atteso;
- evoluzione: modificare e mantenere il sistema in risposta a nuove esigenze.

**Perché esistono diversi modelli?** Non tutti i progetti sono uguali: alcuni hanno requisiti molto stabili e vincoli normativi forti; altri nascono in ambienti altamente dinamici, con idee ancora da esplorare. Applicare sempre lo stesso processo, senza guardare al contesto, porta facilmente a inefficienze:

- troppa rigidità dove servirebbe flessibilità
- troppa improvvisazione dove servirebbe pianificazione

I modelli di processo offrono quindi strategie diverse, che l'ingegnere del software deve conoscere per poter scegliere (e adattare) quella più adatta al progetto.

**Modelli di processo:** Nel seguito vedremo tre modelli classici di processo software. Non sono schemi rigidi e mutuamente esclusivi: nella pratica molti progetti reali combinano elementi di questi modelli, adattandoli al contesto. L'obiettivo non è “scegliere il vincitore”, ma capire quando e perché ciascun modello può essere più adatto.

**Modello waterfall:** Il modello waterfall (a cascata) organizza lo sviluppo come una sequenza di fasi ben definite, che procedono in modo ordinato e lineare. Ogni fase produce artefatti che diventano l'input per la fase successiva:

- progettazione
- implementazione
- validazione
- rilascio e manutenzione

La logica è quella di minimizzare i ritorni indietro: si passa alla fase successiva solo quando la precedente è considerata completa.

*Vantaggi:* La struttura sequenziale è facile da comunicare a tutti gli stakeholder e rende chiaro in quale fase ci si trovi in ogni momento. La forte enfasi sulla documentazione iniziale favorisce:

- la tracciabilità delle decisioni
- la possibilità di svolgere audit e verifiche formali
- l'allineamento tra team tecnici e figure non tecniche.

È particolarmente adatto quando i requisiti sono ben noti e il contesto cambia lentamente, ad esempio in alcuni ambiti industriali regolamentati.

*Limiti:* Nella realtà, i requisiti cambiano spesso durante lo sviluppo e raramente sono completi o corretti fin dall'inizio. Nel waterfall, tornare indietro a modificare fasi già “chiuse” è costoso e complicato:

- correggere un requisito errato dopo aver progettato e implementato il sistema può richiedere molto lavoro.
- Gli utenti vedono il sistema completo solo alla fine, quando è difficile correggere scelte di fondo: il rischio di scoprire tardi malintesi o mancanze è elevato.

*Costo del cambiamento:*

- Nel modello waterfall gli errori scoperti tardi costano molto di più.
- Il costo del cambiamento cresce in modo non lineare man mano che si avanza nelle fasi

**Modello incrementale:** Nel modello incrementale, il sistema non viene progettato e sviluppato “tutto insieme”, ma costruito per passi successivi. Ogni incremento introduce un insieme coerente di funzionalità che, una volta completate, producono una versione utilizzabile del sistema:

1. Una base minima funzionante
2. Estensioni e miglioramenti progressivi

L'utente può iniziare a usare presto una parte del sistema e fornire feedback che guida gli incrementi successivi.

*Vantaggi:* Con l'approccio incrementale il sistema cresce per piccoli rilasci, ciascuno utilizzabile. Dopo ogni rilascio gli utenti possono provare il sistema e dare feedback immediato. Questo permette di:

- correggere la direzione molto prima
- ridurre il rischio complessivo

- concentrare lo sforzo sulle funzionalità che portano più valore

*Limiti:* La flessibilità dell'approccio incrementale non significa assenza di pianificazione: serve comunque una visione architetturale complessiva del sistema. Se ogni incremento viene progettato come un mini–progetto indipendente, si rischia di accumulare debito tecnico, cioè il “costo futuro” dovuto a soluzioni veloci o provvisorie prese oggi. ad esempio:

- codice duplicato
- strutture provvisorie mai riscritte
- moduli con interfacce incoerenti
- mancanza di test
- architettura confusa.

Come un debito finanziario, il debito tecnico genera “interessi”: nel tempo tutto diventa più lento, più fragile e più costoso da modificare. Per funzionare bene, gli incrementi devono essere pensati come parti di un disegno coerente e non come patch aggiunte una dopo l'altra.

**Modello a spirale:** Il modello a spirale rappresenta lo sviluppo come una serie di cicli successivi, ognuno dei quali affronta rischi e incertezze. Ogni giro della spirale include:

- definizione degli obiettivi
- analisi dei rischi
- sviluppo di una soluzione (spesso parziale)
- valutazione prima di passare al ciclo successivo.

L'idea è quella di accettare che non tutto sia chiaro all'inizio e che servano esperimenti per esplorare alternative. Per prototipo intendiamo una versione semplificata del sistema (o di una sua parte), costruita velocemente per capire se un'idea funziona, senza preoccuparsi ancora di tutti i dettagli. I prototipi vengono usati per ridurre rischi specifici:

- provare una nuova tecnologia
- testare un'interfaccia con gli utenti
- verificare ipotesi sulle prestazioni.

Ogni giro della spirale produce nuova conoscenza che permette di prendere decisioni più informate, evitando sorprese nelle fasi avanzate.

**Confronto dei tre modelli visti:** - Il waterfall è adatto a contesti stabili, con requisiti chiari e forte enfasi sulla documentazione.

- L'approccio incrementale funziona bene quando vogliamo rilasciare valore progressivo e ottenere feedback continui.
- La spirale è indicata quando rischio e incertezza sono alti e servono cicli di esplorazione controllati.

Nella pratica, molti progetti combinano elementi di questi modelli, adattandoli al contesto.

| **Waterfall** | **Incrementale** | **Spirale** |
|---|---|---|
| Sequenziale, fasi fisse | Rilasci progressivi | Cicli iterativi guidati dal rischio |
| Requisiti stabili | Feedback frequente | Gestione esplicita dell'incertezza |
| Documentazione forte | Adattamento continuo | Prototipi per ridurre rischi |
| Poco flessibile | Più flessibile | Molto flessibile ma più costoso |

**Perché servono metodi agili?** I modelli di processo più tradizionali, come il waterfall e altri approcci sequenziali, nascono in contesti in cui i requisiti sono relativamente stabili e i rilasci poco frequenti. Oggi molti progetti vivono in ambienti altamente dinamici:

- richieste che cambiano rapidamente
- mercati competitivi
- aspettative di aggiornamenti continui

I metodi agili nascono per affrontare proprio questa realtà, introducendo iterazioni brevi, feedback costanti e collaborazione continua.

**Principi chiave dell'approccio agile:** L'attenzione si sposta da piani rigidi a una capacità di adattamento, in cui il cambiamento non è un problema da evitare ma un dato di fatto da gestire. Il software funzionante diventa la misura principale di avanzamento del progetto, più importante della quantità di documenti prodotti. Vengono valorizzate:

- le persone e le interazioni
- la comunicazione diretta
- il lavoro di team
- la collaborazione continua con gli stakeholder

**Triangolo del progetto: tradizionale vs agile:** Nei progetti tradizionali si tende a fissare lo scopo (tutte le funzionalità), lasciando variabili tempo e costo. Nei metodi agili si fissano tempo (durata dello sprint) e costo (dimensione del team), rendendo flessibile lo scopo in base al valore e alle priorità.

**Scrum:** Scrum è un framework organizzativo: definisce come un team collabora, prende decisioni e rilascia valore in modo iterativo. Osserveremo in particolare:

- i **ruoli essenziali** (Product Owner, Scrum Master, Development Team)
- il **backlog** come lista ordinata di funzionalità
- gli **sprint** come cicli brevi che producono incrementi funzionanti.

**Ruoli fondamentali in Scrum:** Il **Product Owner** rappresenta gli interessi di utenti e stakeholders:

- definisce la visione del prodotto
- stabilisce le priorità nel backlog

Lo **Scrum Master** facilita il lavoro del team:

- aiuta a rimuovere gli ostacoli
- promuove le pratiche agili e la collaborazione

Il **Development Team**

- realizza concretamente gli incrementi del sistema

**Backlog e user stories:** Il **backlog** è l'elenco ordinato delle funzionalità, dei miglioramenti e dei bug da gestire sul sistema. Molti elementi del backlog vengono descritti come **user stories:** brevi frasi dal punto di vista dell'utente, **che chiariscono chi vuole cosa e per quale motivo**. Il **backlog** è un artefatto vivo: viene aggiornato continuamente **sulla base dei feedback e dei risultati degli sprint.**

**Sprint e incrementi:** Lo sviluppo procede per sprint, cioè intervalli di tempo brevi e fissi (ad esempio 2 settimane), con obiettivi limitati ma chiari. Alla fine di ogni sprint il team dovrebbe poter mostrare un incremento funzionante del sistema, anche se piccolo. Questo ritmo cadenzato consente:

- di valutare spesso i progressi
- di adattare il piano sulla base dell'esperienza
- di coinvolgere regolarmente gli stakeholder

Le funzionalità non vengono affrontate tutte insieme, ma organizzate in:

- epic → macro–aree funzionali
- user stories → unità più piccole e verificabili
- sprint → passi concreti che producono un incremento funzionante.

Questo rende visibile l'evoluzione del sistema e permette di:

- adattare le priorità nel tempo
- collegare ogni incremento a valore per gli utenti

A partire dalle epic possiamo ricavare user stories più piccole e concrete, ciascuna con un obiettivo chiaro e verificabile. Le user stories vengono scritte dal punto di vista dell'utente, specificando:

- chi è l'attore
- che cosa vuole ottenere
- perché quella funzionalità è utile.

## Requisiti funzionali e non funzionali

**Requisito:** Un requisito è una descrizione chiara e condivisa di qualcosa che il sistema deve fare oppure di un vincolo che deve rispettare. Possiamo vederlo come una promessa tra chi richiede il sistema e chi lo realizza. Un requisito ben scritto:

- riduce le ambiguità e le interpretazioni personali (devono essere chiari);
- permette di verificare in modo oggettivo se il sistema si comporta come atteso (se ho dei vincoli questi vengono rispettati);
- guida direttamente la progettazione dei test.
- Criterio guida: un requisito deve essere verificabile (e, quando possibile, anche misurabile).

> **Example**
>
> - Il sistema deve permettere allo studente di iscriversi a un appello d'esame.
> - L'accesso alla piattaforma deve essere possibile solo tramite credenziali istituzionali.

Un requisito può essere espresso a due livelli di dettaglio:

1. *Lato utente:* linguaggio semplice, orientato ai bisogni, agli obiettivi e al contesto → comprensibile a chi usa il sistema
2. *Lato sistema:* descrizione più precisa, utile per progettare e implementare concretamente -> utilizzato da sviluppatori e progettisti

Non sono due categorie distinte, ma di due modi di esprimere la stessa necessità. Questi due li velli sono spesso indicati come *requisiti utente* e *requisiti di sistema*.

> **Example**
>
> Versione lato utente
>
> - «Il sistema deve permettere al docente di ottenere un riepilogo degli appelli d'esame.»
>
> Versione lato sistema
>
> - Il sistema genera un elenco contenente: corso, data, iscritti, esiti.
> - Il riepilogo può essere filtrato per periodo, corso o docente.
> - Il risultato è disponibile sia a video sia in formato scaricabile.

I requisiti si dividono in due grandi famiglie:

1. *Requisiti funzionali:* descrivono cosa il sistema deve fare, cioè servizi, operazioni e comportamenti osservabili dagli utenti (quali servizi deve offrire il sistema)
2. *Requisiti non funzionali:* descrivono come il sistema deve funzionare, quali vincoli deve rispettare e quale livello di qualità deve garantire (in che condizioni questi servizi devono funzionare bene)

Queste due categorie si completano in quanto i funzionali definiscono la funzione mentre i non funzionali definiscono qualità, limiti e contesto operativo

**Requisiti funzionali:** Sono i requisiti più intuitivi, descrivono le azioni che il sistema deve permettere e le interazioni possibili tra utenti e piattaforma. Riguardano tutto ciò che è osservabile durante l'uso:

- inserire, modificare, cancellare informazioni;
- consultare elenchi o dati specifici;
- avviare procedure, confermare scelte, scaricare documenti.

I requisiti funzionali definiscono la trama del sistema: se mancasse anche solo una funzione essenziale, il sistema non potrebbe svolgere la sua attività principale.

> **Example**
>
> - Lo studente può iscriversi a un appello d'esame.
> - La segreteria può creare un nuovo corso e associarvi un docente.

- Il docente può registrare i voti degli studenti iscritti.»

Un requisito funzionale vago è una fonte di errore. Un requisito vago porta inevitabilmente a:

- interpretazioni diverse
- risultati incoerenti
- bisogno di riscrivere e correggere in corso d'opera

Un requisito formulato in modo generico non è verificabile: non indica con chiarezza quale risultato ci si aspetta e quindi non guida la definizione dei test. Ambiguità di questo tipo tendono ad accumularsi e generano debito tecnico: prima o poi sarà necessario rivedere codice e documentazione per riallinearli a una definizione più precisa

**Requisiti non funzionali:** Non descrivono azioni specifiche del sistema, ma come il sistema deve comportarsi, quali vincoli deve rispettare e quale livello di qualità deve garantire. Questi riguardano aspetti trasversali che influenzano l'esperienza d'uso e l'affidabilità complessiva, come:

- prestazioni: tempi di risposta, capacità di gestire molti utenti contemporanei;
- sicurezza: protezione dei dati, autenticazione, autorizzazioni;
- affidabilità: continuità del servizio, gestione degli errori, recupero dopo un guasto;
- accessibilità e usabilità: chiarezza dell'interfaccia, facilità d'uso, barriere ridotte;
- compatibilità: integrazione con sistemi già esistenti;
- rispetto di norme e regolamenti: ad esempio le regole sulla gestione dei dati personali.

Questi requisiti incidono profondamente sulla riuscita del progetto: un sistema che “fa tutto” ma è lento, complesso da usare o poco sicuro risulta comunque inadeguato. Possiamo raggruppare i requisiti non funzionali in 3 famiglie (per non trascurare aspetti importanti ma meno visibili rispetto alle funzionalità):

1. *Legati al prodotto* = qualità interne del sistema (prestazioni, tempi di risposta, affidabilità, consumo di risorse).
2. *Legati all'organizzazione* = vincoli del contesto in cui il sistema è sviluppato o usato (strumenti obbligatori, procedure interne, integrazioni con sistemi esistenti).
3. *Legati all'ambiente esterno* =norme, regolamenti e standard tecnici che il sistema deve rispettare.

**Example**

Requisiti non funzionali legati al prodotto

- Le operazioni di iscrizione a un appello devono avere un tempo di risposta inferiore a 2 secondi nella maggior parte dei casi.
- Il sistema deve poter gestire accessi contemporanei da parte di migliaia di studenti durante i periodi di picco.

Requisiti legati all'organizzazione

- L'accesso alla piattaforma avviene esclusivamente tramite le credenziali istituzionali dell'ateneo.
- Il sistema deve integrarsi con i servizi già presenti: anagrafe studenti, archivio docenti, gestione dei corsi.

Requisiti legati all'ambiente esterno

- Il trattamento dei dati degli studenti deve rispettare la normativa vigente sulla protezione dei dati personali.
- La piattaforma deve garantire la conservazione dei verbali secondo le disposizioni ministeriali.

**Requisiti verificabili:** Un requisito è utile solo se possiamo verificarlo in modo oggettivo. Spesso nelle fasi iniziali emergono affermazioni molto generali: «facile da usare», «veloce», «sicuro», «intuitivo». Questi obiettivi sono utili per capire la direzione desiderata, ma non dicono concretamente come il sistema deve comportarsi. In questa forma non sono verificabili: due persone possono interpretarli in modo anche molto diverso. Occorre quindi un passaggio successivo: trasformare gli obiettivi in requisiti misurabili, cioè descrizioni che permettono di verificare in modo oggettivo se il sistema rispetta ciò che promette.

**Example**

Da: «Il sistema deve essere veloce.»

A: «Il sistema deve rispondere entro 2 secondi per almeno il 95% delle richieste di iscrizione agli appelli.»

Perché questa trasformazione è importante?

- Permette di verificare il comportamento del sistema in modo oggettivo.
- Evita interpretazioni diverse tra utenti, progettisti e sviluppatori.
- Guida scelte architetturali e tecniche coerenti.

È del tutto normale che, nelle prime fasi, i requisiti non siano perfetti: alcuni mancano semplicemente perché nessuno ci ha ancora pensato, altri risultano in conflitto (ad esempio: «facile da usare» ma anche «richiedere password molto complesse»), altri ancora riflettono punti di vista diversi tra studenti, docenti, segreteria, tecnici. Questa situazione non è un difetto del progetto, ma una caratteristica naturale dei sistemi reali: le esigenze si scoprono poco alla volta e spesso emergono contraddizioni da risolvere.
L'analisi dei requisiti serve proprio a:

- chiarire cosa si intende davvero
- verificare che le richieste siano realistiche e compatibili
- negoziare priorità e compromessi tra stakeholder con bisogni diversi

I requisiti non emergono spontaneamente dal codice o dalla tecnologia: occorre andare a cercarli parlando con chi userà davvero il sistema. Per raccoglierli si usano strumenti diversi, spesso complementari:

- colloqui con gli utenti per capire bisogni, difficoltà e aspettative;
- osservazione delle attività reali per vedere come i processi funzionano oggi, senza filtri;
- analisi di procedure, moduli e documenti esistenti per cogliere vincoli già presenti nell'organizzazione;
- prototipi e dimostrazioni preliminari per far emergere idee e problemi che non verrebbero fuori solo parlando.

Raccogliere requisiti è un'attività progressiva: solo con un confronto continuo con gli stakeholder si costruisce una visione chiara e condivisa del sistema. Una volta raccolti i bisogni, serve però un modo semplice e comprensibile per rappresentarli: da qui nasce l'uso delle user stories.

**User stories:** Una user story descrive un'esigenza dal punto di vista dell'utente, in modo semplice e comprensibile anche a chi non è tecnico. Risponde a tre domande fondamentali:

- chi usa il sistema
- che cosa vuole ottenere
- perché quell'azione è importante.

La forma tipica è molto essenziale: «Come (tipo di utente), voglio (azione) per (obiettivo/beneficio).». Le user stories non descrivono i dettagli tecnici, ma chiariscono l'intenzione: servono ad allineare utenti e sviluppatori e a popolare un backlog comprensibile e prioritizzabile. Per essere davvero utili, dovrebbero:

- avere un chiaro valore per l'utente
- essere abbastanza piccole da stare in uno sprint
- risultare verificabili tramite criteri di accettazione espliciti.

**Info**

- **User story:** frase breve orientata a chi-cosa-perché, utile per raccogliere e prioritizzare bisogni nel backlog.
- **Use case:** descrizione più strutturata dell'interazione, che useremo più avanti nella parte di modellazione UML.

**Criteri di accettazione:** Ogni user story deve essere accompagnata da uno o più criteri di accettazione, cioè condizioni che stabiliscono con chiarezza quando la funzionalità può essere considerata davvero completata. I criteri di accettazione servono a:

- eliminare ambiguità su cosa ci si aspetta dal sistema;
- definire un confine preciso tra “fatto” e “non ancora fatto”;
- costituire una base solida per i test che verificheranno il corretto comportamento del sistema.

In altre parole: la user story racconta l'intenzione, i criteri di accettazione ne specificano la verifica concreta. Alcuni criteri possono essere espressi anche in forma strutturata, ad esempio con frasi del tipo «Dato che … Quando … Allora … », facilitando la verifica sistematica del comportamento del sistema.

> **Example**
>
> User story
>
> - «Come studente, voglio iscrivermi a un appello d'esame per poter sostenere la prova nella data che scelgo.»
>
> Criteri di accettazione
>
> - Lo studente vede solo gli appelli disponibili per il proprio corso.
> - Se i posti sono esauriti, il sistema mostra un messaggio chiaro e comprensibile.
> - Dopo l'iscrizione, lo studente visualizza immediatamente la conferma nella propria area personale.
> - Il docente può consultare l'elenco aggiornato degli iscritti all'appello.

**Backlog:** Le user stories raccolte diventano elementi del backlog che guida lo sviluppo: una raccolta ordinata di ciò che il sistema dovrà offrire. Non tutti gli elementi hanno lo stesso peso: occorre stabilire delle priorità. Alcune funzionalità vanno affrontate prima perché:

- portano più valore agli utenti;
- comportano rischi tecnici o organizzativi da affrontare presto;
- dipendono da altre funzionalità che devono essere pronte.

Il backlog è un artefatto vivo: cambia, si aggiorna, viene riorganizzata man mano che emergono nuove informazioni o nuove esigenze. I requisiti non funzionali non devono essere trascurati: anche loro entrano a pieno titolo nel backlog. Alcuni diventano attività specifiche, da pianificare come vere e proprie funzionalità:

- migliorare i tempi di risposta nelle operazioni più frequenti;
- introdurre o potenziare un meccanismo di sicurezza;
- rendere l'interfaccia più accessibile o più semplice da usare.

Altri, invece, sono vincoli trasversali: non compaiono come singole attività, ma devono essere rispettati in ogni funzionalità sviluppata (ad esempio sicurezza, normativa sui dati, compatibilità con sistemi esistenti). Ignorarli nelle prime fasi porta quasi sempre a interventi più costosi, più complessi e più rischiosi nelle fasi successive.

## Modellazione concettuale

**Dominio** = Per dominio si intende l'insieme di: persone, oggetti, regole e processi reali che il software deve rappresentare e supportare.

**Modellazione concettuale:** La modellazione concettuale è l'attività che mira a:

- identificare i concetti importanti del dominio;
- chiarire quali responsabilità ha ciascun concetto;
- descrivere le relazioni tra questi concetti.

È una descrizione logica del problema, indipendente dalla soluzione tecnica. Nella pratica: stiamo cercando di costruire un vocabolario condiviso che tutti possano capire (utenti, analisti, sviluppatori).

**Concetti:**Per concetti intendiamo gli elementi significativi del dominio (es. Studente, Corso, Appello), non ancora classi, tabelle o dettagli implementativi. I requisiti che abbiamo scritto non sono solo “liste di funzionalità”: contengono già molti suggerimenti sul mondo che il sistema deve rappresentare. Leggendo frasi come: «Lo studente si iscrive a un appello d'esame.», «Il docente registra il voto.», «Lo studente visualizza la propria carriera.» possiamo chiederci: quali entità compaiono? quali azioni vengono svolte? quali relazioni tra concetti sono implicite? La modellazione concettuale parte spesso proprio da questo lavoro di “lettura attenta” dei requisiti. Dal punto di vista concettuale, nei requisiti ci interessano in particolare:

- i sostantivi (studente, appello, corso, voto, verbale, carriera...) che possono suggerire entità del dominio;
- i verbi (iscriversi, registrare, consultare, creare, chiudere...) che possono suggerire azioni o relazioni;
- le espressioni di vincolo (solo se, fino a, entro...) che possono nascondere regole del dominio.

Non è una trasformazione meccanica, ma una guida: ci aiuta a individuare i concetti davvero importanti.

**Entità concettuali:** Un'entità concettuale è un elemento del mondo che vogliamo rappresentare in modo esplicito nel sistema:

- esiste indipendentemente dalla tecnologia che useremo;
- ha un significato riconoscibile anche da chi non è tecnico.

**Esempi generici**

- persona, ordine, prodotto, pagamento, prenotazione
- studente, corso, esame, voto, appello

L'errore da evitare è confondere subito entità concettuali con strutture di codice.

**Ricorda:**

Nel nostro caso di studio, una prima lista di entità concettuali naturali è:

- Studente = la persona che segue i corsi e sostiene esami
- Corso = rappresenta l'insegnamento (nome, docente, CFU,...)
- Appello = occasione concreta per sostenere l'esame
- Iscrizione = atto con cui lo studente si registra ad uno specifico appello
- Voto = esito ottenuto in quell'appello

In questo modo posso rappresentare situazioni reali come:

- uno studente si può iscrivere più volte
- può sostenere lo stesso corso in appelli diversi
- un appello può avere molti iscritti e molti voti

L'idea non è ancora “definire le classi”, ma capire:

- che cosa rappresenta ciascuna di queste entità;
- perché è utile distinguerle;
- che tipo di informazioni ci aspettiamo che custodiscano.

A ogni entità possiamo associare alcune responsabilità (dominio delle operazioni consentite ad un'entità), cioè:

- quali informazioni custodisce
- in quali interazioni è coinvolta
- quali decisioni del dominio “passano da lei”.

Definire bene le responsabilità riduce il rischio di “entità onnipotenti” o confuse.

**Example**

- lo Studente è responsabile dei propri dati anagrafici e della propria carriera
- l'appello è responsabile di data, ora, aula, lista potenziale degli iscritti
- iscrizione è responsabile del legame tra uno specifico studente e uno specifico appello

Le entità non vivono isolate: ciò che rende interessante un dominio sono spesso le relazioni tra i concetti. In questa fase distinguiamo due famiglie utili:

- relazioni che descrivono la struttura del dominio;
- relazioni che descrivono le azioni che avvengono nel dominio.

Capire bene le relazioni è essenziale per rispondere a domande tipiche:

- «Quali appelli può vedere questo studente?»
- «Quali studenti si sono iscritti a questo appello?»
- «Quali voti ha ottenuto uno studente in un certo arco di tempo?»

Alcune relazioni descrivono la struttura stabile del dominio:

- uno studente ha una carriera composta da esami e voti;
- un corso è tenuto da uno o più docenti;
- un appello è relativo a un certo corso e a una certa data.

**Example**

- Corso – Docente: un corso è tenuto da uno o più docenti.
- Corso – Appello: un corso ha uno o più appelli nel tempo.
- Studente – Iscrizione: uno studente può avere più iscrizioni.
- Iscrizione – Appello: ogni iscrizione si riferisce a uno specifico appello.

Queste relazioni esistono “a prescindere” dal singolo utilizzo del sistema: descrivono il modo in cui è organizzato il mondo universitario. Sono la base della nostra mappa concettuale del dominio. Altre relazioni descrivono azioni/eventi che avvengono nel dominio:

- Studente → Iscrizione: crea una nuova iscrizione a un appello;
- Docente → Voto: registra o aggiorna l'esito di uno studente;
- Segreteria → Corso/Appello: crea o aggiorna i dati organizzativi.

Queste relazioni sono più vicine al comportamento del sistema: hanno tipicamente precondizioni, effetti e controlli.

**Regole del dominio:** Le regole del dominio esprimono vincoli che devono valere sempre, indipendentemente dalla tecnologia.

**in UniManager**

- uno studente può iscriversi solo ad appelli del proprio corso;
- non ci si può iscrivere dopo la chiusura delle iscrizioni;
- il voto può essere registrato solo per studenti regolarmente iscritti all'appello;
- la verbalizzazione chiude la modifica libera dei voti.

Queste regole guidano sia la mappa concettuale sia i controlli che progetteremo nelle fasi successive.

**Caratteristiche iscrizione**

- una data in cui è stata compiuta;
- un eventuale stato (attiva, ritirata, annullata ...);
- può essere usata per verificare la regolarità di uno studente in un appello.

**Entità voto**

- è legato a uno studente e a un appello specifico;
- può avere uno stato (proposto, verbalizzato, rifiutato ...);

- può essere oggetto di visualizzazione, controllo, statistiche.

**Confine del sistema**

Oltre alle entità del dominio, è importante chiarire chi interagisce con il sistema: studenti, docenti, segreteria, eventuali amministratori di sistema. Dobbiamo chiederci: che cosa sta dentro UniManager e che cosa sta fuori?

**Elementi esterni:**

- sistemi di autenticazione dell'ateneo;
- basi dati istituzionali di anagrafe
- altri sistemi amministrativi

**Rappresentazione contesto in modo informale:** Prima ancora di usare notazioni formali, possiamo disegnare schemi semplici:

- un riquadro per il sistema UniManager;
- attorno, gli attori principali (studenti, docenti, segreteria);
- frecce che indicano i tipi di interazione (iscrizione, consultazione, gestione corsi ...).

Questi schemi servono a:

- chiarire chi parla con chi;
- evitare di attribuire al sistema responsabilità che non ha;
- distinguere ciò che dobbiamo progettare da ciò che esiste già.

Prima della mappa concettuale è utile esplicitare alcune azioni tipiche del sistema, ad esempio:

- lo studente trova gli appelli disponibili;
- lo studente si iscrive;
- il docente registra i voti;
- lo studente consulta la propria carriera.

Possiamo rappresentarle in modo semplice, ad esempio con:

- una sequenza di passi,
- frecce tra attori e sistema,
- brevi descrizioni delle interazioni principali.

**Rappresentare la struttura concettuale:** Dopo la vista delle azioni, passiamo alla vista strutturale del dominio con una mappa concettuale:

- nodi per le entità (Studente, Corso, Appello, Iscrizione, Voto ...);
- collegamenti che indicano le relazioni principali (ad esempio “Studente è iscritto a Appello tramite Iscrizione”).

Usiamo una rappresentazione informale:

- niente simboli complicati,
- etichette in italiano,
- solo gli elementi davvero utili per capire il dominio.

Questa mappa sarà la base del futuro modello del dominio in notazione più formale. Come prima approssimazione, possiamo disegnare:

- Studente, Corso, Appello come entità principali;
- una relazione che lega Corso e Appello;
- collegamenti tra Studente e Appello attraverso l'idea di iscrizione

Questa versione iniziale è volutamente semplice:

- ci permette di parlare, discutere, correggere;
- mette in evidenza eventuali concetti mancanti

Una volta condivisa la prima versione, possiamo arricchirla con:

- l'entità Iscrizione come collegamento esplicito;
- l'entità Voto come esito dell'iscrizione;
- eventuali altri concetti emersi (sessione, verbale, carriera ...).

La mappa concettuale è uno strumento vivo:

- può cambiare quando scopriamo nuovi bisogni;
- può essere semplificata se ci accorgiamo di avere concetti inutilmente complicati.

Passiamo ora da una mappa minima a una mappa più ricca, aggiungendo tre elementi:

- Iscrizione come collegamento esplicito tra Studente e Appello;
- Voto come esito associato all'iscrizione;
- alcuni concetti opzionali (sessione, verbale, carriera) per mostrare come il modello può evolvere.

Obiettivo della versione estesa:

- aumentare chiarezza e capacità esplicativa del modello
- senza introdurre ancora dettagli implementativi.

Una rappresentazione più completa del dominio, dove compaiono:

- l'entità Iscrizione come collegamento esplicito tra Studente e Appello;
- l'entità Voto come esito dell'iscrizione;
- alcuni concetti aggiuntivi possibili (sessione, verbale, carriera) per mostrare come la mappa possa evolvere

**Checklist rapido:** Prima di proseguire, facciamo un controllo rapido:

- *Nomi chiari:* ogni entità ha un nome comprensibile anche a chi non è tecnico?
- *Relazioni esplicite:* i collegamenti principali sono scritti e coerenti con i requisiti?
- *Niente dettagli tecnici:* stiamo evitando classi, tabelle DB, API e scelte implementative?

Se una risposta è “no”, conviene correggere subito: nelle fasi successive gli errori costano di più. Una buona modellazione concettuale:

- riduce le ambiguità,
- aiuta a prendere decisioni progettuali più consapevoli,
- facilita il dialogo tra persone con competenze diverse.

Al contrario, se si salta questa fase:

- si rischiano fraintendimenti tra utenti e sviluppatori;
- si introducono vincoli tecnici non necessari;
- diventa più difficile correggere gli errori in seguito.

Investire tempo qui fa risparmiare tempo nelle fasi successive.

**Errori tipici:** Alcuni errori ricorrenti:

- confondere subito entità concettuali e classi di implementazione;
- introdurre dettagli tecnici (database, interfacce grafiche...) troppo presto;
- creare entità “che fanno tutto” e relazioni poco chiare;
- usare nomi vaghi o poco significativi (dati1, oggetto, elemento...).

È importante ricordare che in questa fase lo scopo non è “scrivere codice”, ma capire bene il problema.

## Use case diagram

**UML:** Iniziamo ad usare una notazione standard per rappresentare i modelli: UML (unified modelling language) che è un linguaggio visivo condiviso. UML è un linguaggio di modellazione standard, non un linguaggio di programmazione. Fornisce un insieme di diagrammi per descrivere:

- la struttura statica del sistema
- il comportamento dinamico
- le interazioni tra parti del sistema e attori esterni.

Non obbliga a un particolare processo (waterfall, agile,...), ma può essere usato in tutti. UML nasce negli anni '90 per unificare diverse notazioni di analisi e progettazione. I principali contributori iniziali sono Grady Booch, Ivar Jacobson e James Rumbaugh. La prima standardizzazione avviene in ambito OMG (Object Management Group). Nel tempo UML ha avuto più versioni: la famiglia 1.x e poi 2.x, con estensioni e chiarimenti della notazione. Oggi UML resta un riferimento diffuso per comunicare modelli in modo condiviso tra team diversi. UML mette a disposizione diversi tipi di diagrammi. In questo corso vedremo soprattutto:

- Use Case Diagram — vista del sistema dal punto di vista degli attori esterni;
- Modello di dominio — modello concettuale del dominio, spesso rappresentato con un class diagram semplificato;
- Class Diagram — struttura statica del dominio (classi, attributi, associazioni);
- Sequence Diagram — interazioni nel tempo tra oggetti/attori;
- Activity Diagram — flusso di attività, decisioni e responsabilità di processo.

UML include anche altri diagrammi (ad esempio State Machine, Component, Deployment). Inizieremo dall'Use Case Diagram, perché:

- è vicino ai requisiti e alle user stories,
- riguarda chi usa il sistema e per quali obiettivi.

> **I casi d'uso descrivono gli obiettivi significativi per gli utenti.**

Abbiamo già ragionato su:

- chi interagisce con un sistema software;
- che cosa sta dentro e che cosa sta fuori dal sistema.

Nel Use Case Diagram UML:

- gli attori sono ruoli esterni;
- il sistema è rappresentato da un rettangolo che racchiude i casi d'uso;
- le interazioni diventano associazioni.

**Attore:** Un attore è un ruolo esterno che interagisce con il sistema:

- una persona (studente, docente),
- un altro sistema (sistemi di autenticazione, basi dati esterne),
- un dispositivo.

È importante ricordare che:

- non rappresenta una singola persona, ma un tipo di utilizzatore;
- è sempre fuori dal sistema.

In UniManager i nostri attori principali sono: Studente, Docente, Segreteria.

**Caso d'uso:** Un caso d'uso rappresenta un obiettivo significativo per un attore non è un singolo “click” ne una funzione tecnica (“query sul DB”), ma un risultato che ha senso dal punto di vista dell'utente.

> **Esempi tipici in UniManager:**
>
> - Iscrizione all'appello,
> - Registrazione voti,
> - Consultazione libretto,
> - Creazione appello

Nella notazione UML, i casi d'uso sono rappresentati come ellissi con un nome. Nei diagrammi completi i casi d'uso stanno dentro il confine del sistema; negli esempi minimali di queste slide il rettangolo può essere omesso per semplicità visiva. Nella lezione precedente abbiamo letto frasi come:

- «Lo studente si iscrive a un appello d'esame.»
- «Il docente registra il voto.»
- «Lo studente visualizza la propria carriera.»

Dal punto di vista dei casi d'uso:

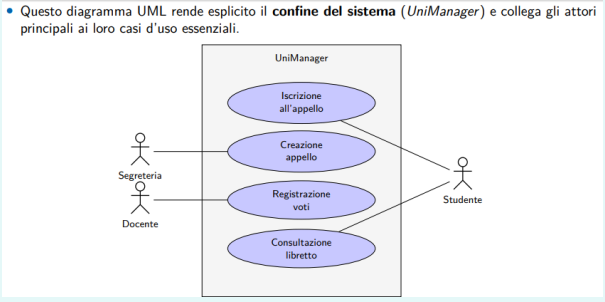
- attore: chi compie l'azione (Studente, Docente,...);
- obiettivo: che cosa vuole ottenere (essere iscritto, vedere la carriera, registrare il voto);
- sistema: che parte del lavoro è di UniManager.

Il Use Case Diagram sintetizza queste relazioni in forma visuale. Dai requisiti e dalle user stories emergono quattro casi d'uso fondamentali:

- Iscrizione all'appello;
- Creazione appello;
- Registrazione voti;
- Consultazione libretto.

Questi casi d'uso coinvolgono i tre attori principali del sistema: Studente, Docente, Segreteria. Sono il punto di partenza per costruire il primo Use Case Diagram di UniManager.

- Questo diagramma UML rende esplicito il **confine del sistema** (*UniManager*) e collega gli attori principali ai loro casi d'uso essenziali.




**Relazione *include* (obbligatorio):** Include rappresenta un comportamento comune e obbligatorio tra più casi d'uso. Un caso d'uso A include B se:

- ogni volta che si esegue A, a un certo punto è necessario eseguire B;
- B è un “pezzo riusabile” usato da più casi d'uso.

A livello grafico:

- la relazione è una freccia tratteggiata con l'etichetta «include»;
- la freccia punta verso il caso d'uso incluso (da A verso B).

Serve quando una parte del comportamento è identica e va eseguita sempre prima o durante un caso d'uso principale.

### Esempio uso include

- Casi d'uso coinvolti:
  - **Iscrizione all'appello** (Studente);
  - **Consultazione libretto** (Studente);
  - **Autenticazione** (caso d'uso comune).
- Entrambi richiedono che lo studente sia autenticato:
  - l'autenticazione non è un fine in sé,
  - è un passo obbligatorio per altri obiettivi.



**Relazione *extend* (opzionale):** Extend rappresenta una variante opzionale di un caso d'uso. Un caso d'uso B estende A se:

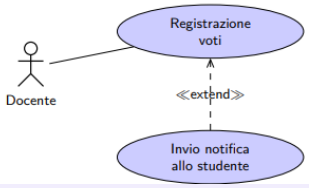
- A descrive il flusso “normale”;
- B aggiunge passi opzionali o condizioni particolari.

A livello grafico:

- la relazione è una freccia tratteggiata con etichetta «extend»;
- la freccia punta verso il caso d'uso esteso (da B verso A).
- Serve quando un comportamento aggiuntivo può avvenire, ma non è obbligatorio.

**Esempio uso extend**

- Caso d’uso principale:
  - **Registrazione voti** (Docente).
- Caso d’uso opzionale:
  - **Invio notifica allo studente**.
- In UML:
  - “Invio notifica allo studente” **estende** “Registrazione voti”,
  - l’estensione si attiva solo in certe condizioni.




**Generalizzazione tra attori:** È possibile organizzare gli attori in una gerarchia, serve quando più attori condividono gli stessi casi d'uso. Significato:

- l'attore “padre” rappresenta un ruolo più generale;
- l'attore “figlio” eredita le sue interazioni.

A livello grafico (UML):

- si usa una linea continua con una freccia triangolare non piena;
- la freccia punta verso l'attore più generale (da Specializzato → Generale).

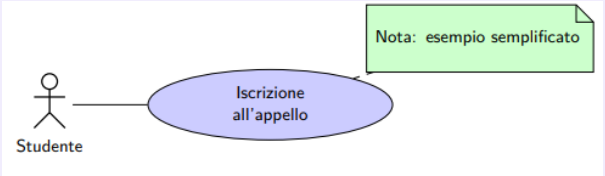
Va usata solo se semplifica il modello o riduce ripetizioni. In alcuni casi può essere utile introdurre un attore più generale: Utente come ruolo comune. Attori più specifici:

- Studente,
- Docente.

La generalizzazione permette di evitare ripetizioni:

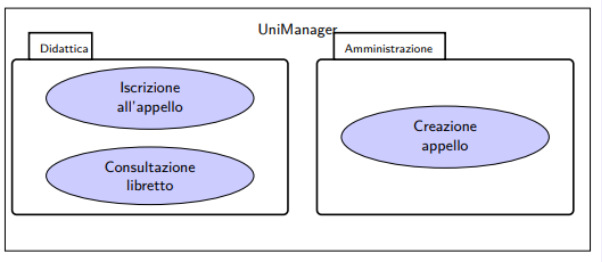
- i casi d'uso comuni a tutti gli utenti possono collegarsi a Utente;
- i casi specifici rimangono ai ruoli specializzati.

**Note e commenti:** In UML possiamo aggiungere note/commenti per chiarire assunzioni e scelte. Le note aiutano a esplicitare dettagli utili senza appesantire i casi d'uso. Sono particolarmente utili quando una regola non è rappresentabile in modo diretto con soli attori e relazioni. Una buona pratica è quella di scrivere note brevi e mirate, collegate solo ai punti davvero ambigui.




**Package e raggruppamenti:** In UML possiamo raggruppare casi d'uso in package/aree funzionali. Ciò è utile quando il diagramma cresce e vogliamo separare aree funzionali. La motivazione principale è la leggibilità:

meno incroci visivi, più chiarezza su responsabilità e perimetri. I package non cambiano il comportamento dei casi d'uso: organizzano la vista del modello.




**Molteplicità sulle relazioni:** In alcuni casi possiamo esplicitare la molteplicità sulle relazioni (senza esagerare per rimanere il più asciutti possibile, altrimenti si rischia solo rumore). Va usata solo quando aggiunge chiarezza al significato del modello. Nei Use Case Diagram è una notazione possibile ma non centrale (più comune nei Class Diagram).

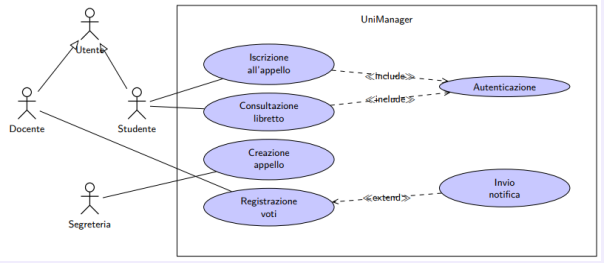
**Example**

- Esempio 1 (1–*): uno Studente può effettuare molte iscrizioni nel tempo.
- Esempio 2 (1–0..*): un Docente può registrare zero o più voti.

**Vincoli:** In UML possiamo esprimere vincoli con testo tra parentesi graffe. I vincoli servono per esplicitare regole di business non evidenti dal solo diagramma (riguardo l'implementazione). Sono utili quando una condizione è decisiva per il dominio (scadenze, prerequisiti, autorizzazioni). Un vincolo deve essere verificabile: formulazioni vaghe riducono il valore del modello.

**Example**

- Mettendo insieme i pezzi visti finora, il Use Case Diagram di *UniManager* include:
  - **Attori**: Utente (generale), Studente, Docente, Segreteria;
  - **Casi d'uso principali**: Iscrizione all'appello, Creazione appello, Registrazione voti, Consultazione libretto;
  - **Casi d'uso di supporto** (comuni o opzionali): Autenticazione, Invio notifica;
  - **Relazioni**: associazioni, include, extend, eventuali generalizzazioni.
- Questo diagramma fornisce una **vista ad alto livello** del sistema:
  - non mostra i dettagli tecnici,
  - ma chiarisce quali obiettivi gli attori possono raggiungere tramite *UniManager*.




**Good practices:**

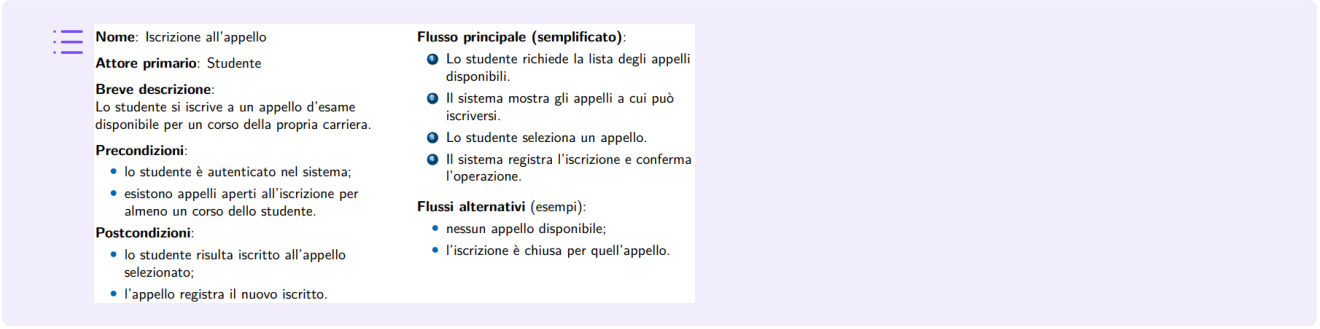
- Usare i casi d'uso per descrivere obiettivi, non schermate o click.
- Mantenere un livello di dettaglio gestibile: pochi casi d'uso macro, non decine di micro-casi.
- Evitare di usare include/extend in modo eccessivo o artificiale.
- Annotare, quando serve, qualche breve descrizione testuale per chiarire le intenzioni.
- Ricordare che il Use Case Diagram è uno strumento di comunicazione con gli stakeholder: deve essere leggibile anche da chi non è tecnico.

**Descrizione testuale:** Il diagramma dei casi d'uso dà una vista sintetica, ma spesso non basta. Per i casi d'uso più importanti conviene scrivere una scheda testuale con:

- Nome del caso d'uso;
- Attori coinvolti;
- Breve descrizione (obiettivo principale);
- Precondizioni (cosa deve essere vero prima);
- Postcondizioni (cosa deve essere vero dopo);
- Flusso principale (passi “normali”);
- eventuali flussi alternativi / eccezioni.

L'idea è avere un livello di dettaglio sufficiente per:

- chiarire il comportamento atteso,
- preparare i diagrammi successivi (sequence, activity... ).




**Da use case a domain model:** Nella lezione precedente abbiamo visto:

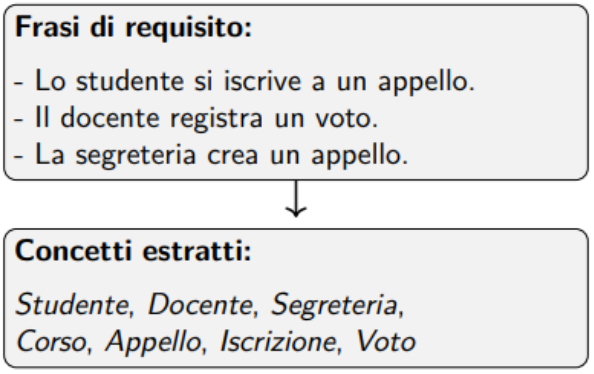
- attori esterni al sistema;
- casi d'uso come obiettivi significativi;
- relazioni include/extend e generalizzazioni.

Il diagramma dei casi d'uso risponde alla domanda: «Chi usa il sistema e per quali obiettivi?» Oggi cambiamo prospettiva: non più “chi usa il sistema”, ma “che cosa esiste nel mondo del sistema”.

Requisiti, user stories e casi d'uso contengono storie:

- «Lo studente si iscrive a un appello.»
- «Il docente registra un voto.»
- «La segreteria crea un appello.»

Da queste storie possiamo estrarre le entità concettuali del dominio: Studente, Docente, Corso, Appello, Iscrizione, Voto…




**Domain model:** È una rappresentazione concettuale del dominio del problema. Mostra:

- i concetti rilevanti (entità, oggetti, ruoli);
- le relazioni tra questi concetti;
- alcuni attributi essenziali che li caratterizzano.

È una fotografia del mondo a cui il sistema si riferisce:

- non contiene dettagli tecnici;
- non parla di database, API, interfacce grafiche;
- non contiene metodi o algoritmi.

Non è:

1. un diagramma delle classi implementativo:
   - non rappresenta direttamente il codice Java/C++/Python;
   - non specifica metodi, eccezioni, pattern di design.
2. il modello del database:
   - non specifica chiavi primarie, indici, tabelle tecniche;
   - non è ottimizzato per performance o storage.
3. un diagramma dei casi d'uso:

- non descrive scenari o obiettivi,
- ma gli oggetti coinvolti in quegli scenari.

Aiuta a capire il problema prima di pensare alla soluzione. È un linguaggio condiviso tra:

- sviluppatori,
- analisti,
- domain expert,
- stakeholder non tecnici.

È un termometro della comprensione del dominio:

- se il modello è confuso, probabilmente non abbiamo capito bene il problema;
- se è chiaro e coerente, il passaggio alla progettazione sarà più semplice.

Punto di partenza: descrizioni testuali, requisiti, casi d'uso:

- evidenziare i sostantivi significativi (candidati concetti);
- evidenziare i verbi principali (candidati per associazioni o responsabilità);
- discutere con gli stakeholder cosa è davvero importante.

Domanda guida: «Se togliamo questo concetto, capiamo ancora il dominio?»

Un concetto è un buon candidato se:

- è nominato spesso nei requisiti;
- ha regole di business proprie;
- gli stakeholder lo riconoscono come entità reale del loro mondo;
- può cambiare nel tempo e queste modifiche sono importanti.

**Attributi essenziali:** Gli attributi descrivono informazioni importanti su un concetto. Dovrebbero essere:

- pochi (minimalismo);
- stabili nel tempo;
- non derivabili facilmente da altri;
- rilevanti per le regole di business.

> **Esempio:**
>
> - Studente: matricola, nome, cognome;
> - meno adatti come attributi nel Domain Model: colore preferito, ultima data di login.

**Errori frequenti degli attributi:**

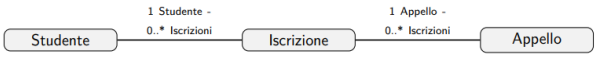
- Confondere il Domain Model con il database: inserire id tecnici, timestamp di audit, campi di log…
- Aggiungere attributi troppo tecnici: flag di caching, token di sessione, dettagli di autenticazione.
- Inserire attributi derivabili: numero di esami superati (derivabile dalle iscrizioni e dai voti) o media voti (derivabile dai singoli voti).

**Associazioni tra concetti:** Le associazioni rappresentano legami concettuali tra elementi del modello. Rispondono alla domanda: «Questi due concetti hanno una relazione significativa nel dominio?»

> **Esempi:**
>
> - uno Studente è iscritto a un Appello;
> - un Appello appartiene a un Corso;
> - un Docente tiene un Corso.

**Molteplicità:** La molteplicità esprime quante istanze di un concetto possono essere collegate a un altro. Lettura rapida:

- 1: esattamente una;
- 0..1: opzionale;
- 0..:* zero o molte;
- 1..:* almeno una




Nelle associazioni, spesso ci interessa la molteplicità:

- uno studente può avere molte iscrizioni;
- un appello riguarda un solo corso;
- un corso può avere più appelli.

Le molteplicità non sono numerini casuali:

- esprimono vincoli reali del dominio;
- aiutano a scoprire regole di business nascoste.

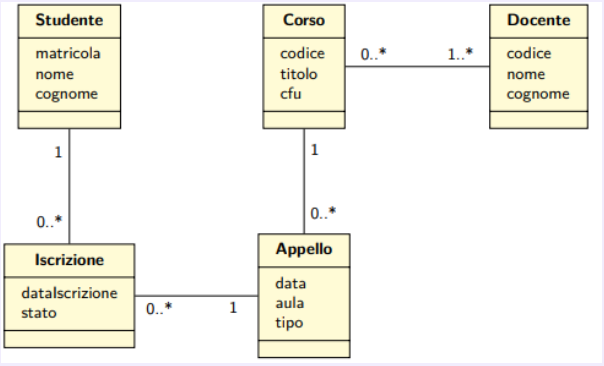
**A partire dalle lezioni precedenti, possiamo individuare alcuni concetti chiave:**

- Studente
- Docente
- Corso
- Appello
- Iscrizione
- (eventualmente) Voto, Carriera, Piano di studi…

Ogni concetto cattura un nucleo di significato del dominio universitario.

**Rappresentazione concettuale del dominio di UniManager:**

- concetti principali, attributi essenziali, relazioni e molteplicità;
- nessun dettaglio di implementazione (tipi, metodi, framework…)




**Domain model come strumento di verifica:** Possiamo usare il Domain Model per fare domande al domain expert:

- «Esistono appelli senza corso?»;
- «Uno studente può iscriversi più volte allo stesso appello?»;
- «Può esistere un corso senza docente associato?».

Ogni risposta può portare a correggere o raffinare il modello:

- aggiungere o rimuovere un concetto;
- modificare una molteplicità; introdurre una nuova associazione.

**Da domain model a class diagram:** Il Domain Model è concettuale e indipendente dalla tecnologia. Il Class Diagram inizia a guardare verso il software:

- introduce il concetto di classe;

- prepara il terreno per attributi e metodi;
- sarà la base per codice e architettura.

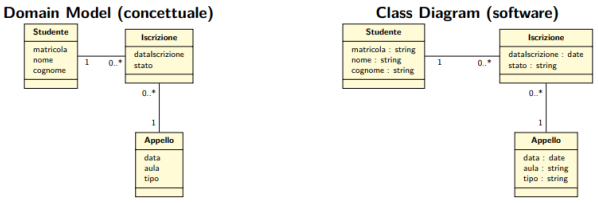
Domain Model:

- concetti del mondo reale;
- attributi essenziali per capire il problema;
- relazioni concettuali tra entità.

Class Diagram:

- classi del sistema software;
- attributi che saranno effettivamente memorizzati;
- relazioni che guideranno la struttura del codice.

Spesso i nomi coincidono, ma il punto di vista è diverso.




**Da domain model alle classi: checklist:** Cosa portiamo nel primo Class Diagram:

- il nome (chiaro e significativo);
- gli attributi essenziali del dominio;
- le relazioni con altre classi (ispirate al Domain Model).

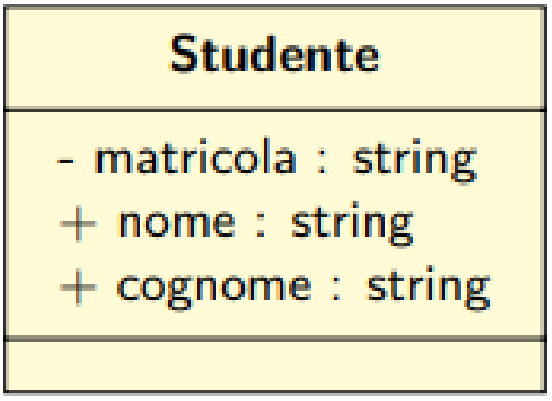
Cosa lasciamo fuori in questa fase:

- metodi e logica di dettaglio;
- classi tecniche (Repository, DAO = Data Access Object, Controller, helper…);
- Repository/DAO: accesso e persistenza dei dati; Controller: coordinamento tra input e logica applicativa;
- dettagli legati a framework specifici.

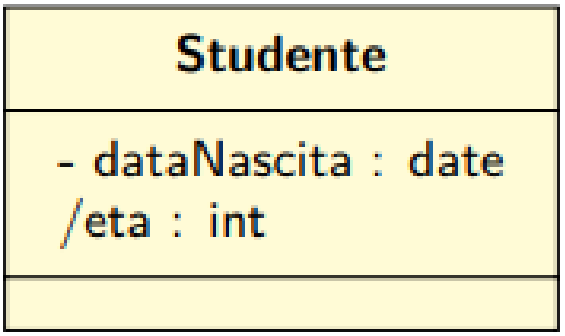
Obiettivo: restare vicini al dominio, ma con una struttura già utile al design software.

**Notazione UML: visibilità degli attributi:** In un Class Diagram possiamo indicare la visibilità:

- 
  - public — visibile a tutti
- private — visibile solo all'oggetto stesso
- # protected — visibile a classe e sottoclassi
- ~ package — visibile all'interno del package



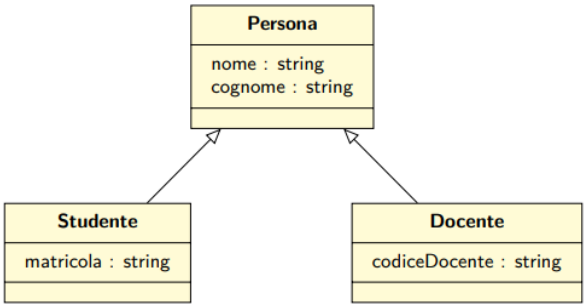

**Attributo derivato:** Un attributo è derivato se può essere calcolato da altri dati. Notazione: si usa il simbolo / davanti al nome.




**Attributo statico:** La notazione UML prevede attributi di classe (statici). Notazione: nome sottolineato.




**Ereditarietà:** La generalizzazione si indica con una freccia a triangolo vuoto verso la superclasse. Le sottoclassi ereditano attributi e operazioni.

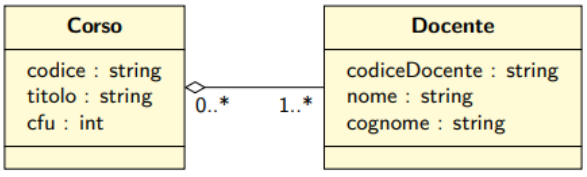



**Aggregazione e composizione:** Sono due tipi particolari di associazione UML. In entrambi i casi c'è un rapporto tra un tutto e una parte.

- *Aggregazione* (rombo vuoto): la parte può esistere anche senza quel tutto.
- *Composizione* (rombo pieno): la parte dipende dal tutto per esistere.

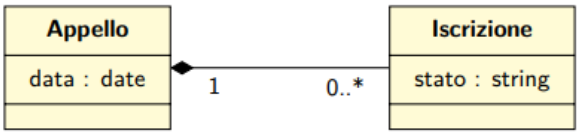
*Aggregazione:* la parte resta indipendente dal tutto.

- nell'esempio: Corso (lato con rombo) è il tutto, Docente è la parte aggregata;
- In breve: se elimino un Corso, il Docente resta.



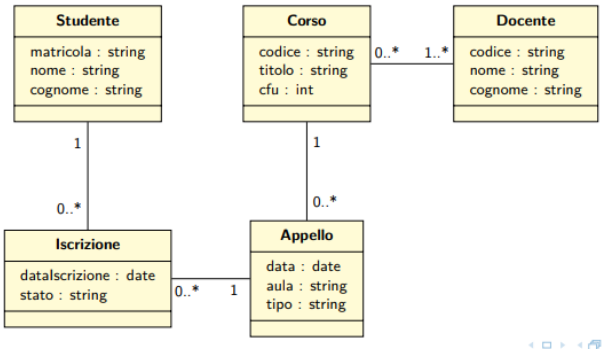

*Composizione:* la parte dipende dal tutto.

- nell'esempio: Appello (lato con rombo pieno) è il tutto, Iscrizione è la parte contenuta;
- In breve: se elimino un Appello, l'Iscrizione non esiste più.




**Class diagram di *Unimanager:*** Traduzione del Domain Model in una rappresentazione orientata al software:

- classi con attributi tipizzati;
- relazioni che suggeriscono i futuri collegamenti tra oggetti;
- in questo diagramma mostriamo solo associazioni.




**Errori tipici nel passaggio a class diagram:** Aggiungere subito:

- metodi per ogni scenario immaginabile;
- classi per ogni schermata dell'interfaccia grafica;
- dettagli di persistenza (DB, ORM, ecc.).
- ORM: meccanismo per mappare oggetti ↔ database;

Confondere classi del dominio con:

- classi di infrastruttura,
- classi di supporto (logger, helper, utility…).

Dimenticare che il Class Diagram deve essere ancora vicino al dominio, non al framework.

**Struttura statica vs comportamento dinamico:** Il Domain Model e il Class Diagram descrivono la struttura statica:

- quali entità esistono;
- come sono collegate;
- quali informazioni portano con sé.

Non dicono ancora:

- come gli oggetti collaborano;
- quali messaggi si scambiano;
- in che ordine avvengono le operazioni.

## Sequence diagram e activity diagram

Nella lezione precedente abbiamo definito la struttura del sistema:

- **Domain Model:** concetti del dominio e relazioni principali;
- **Class Diagram:** classi software e legami tra classi.

Oggi passiamo al livello dinamico:

- come gli oggetti collaborano durante uno scenario;
- in quale ordine avvengono azioni e decisioni.

Useremo due viste complementari:

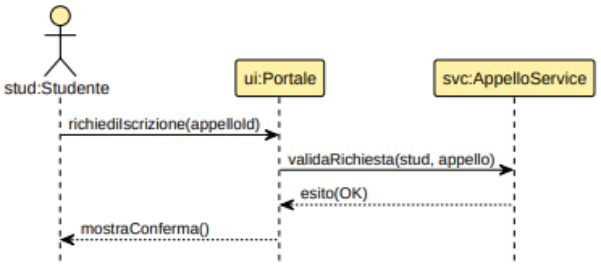
- **Sequence Diagram** per le interazioni tra partecipanti;
- **Activity Diagram** per il flusso del processo.

## Sequence diagram

Si utilizza per chiarire come uno scenario viene eseguito nel tempo. Risponde a tre domande operative:

- chi interagisce con chi;
- in quale ordine avvengono i messaggi;
- chi prende in carico ogni responsabilità.

È particolarmente utile per scenari di caso d'uso con esito positivo e varianti di errore. Non è il diagramma migliore per descrivere l'intero processo: quello è il ruolo degli Activity Diagram



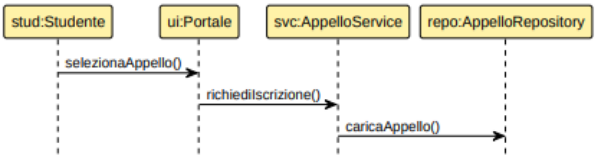

**Lettura:** La lettura corretta segue due assi:

- verticale: il tempo scorre dall'alto verso il basso;
- orizzontale: partecipanti diversi coinvolti nello scenario.

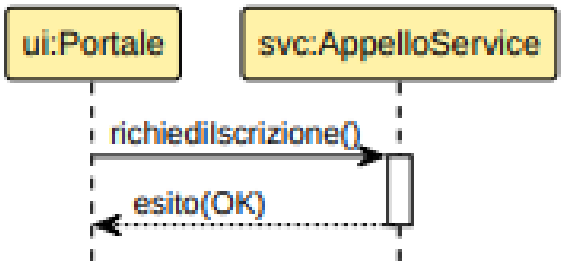
Ogni freccia rappresenta un messaggio tra lifeline e quindi un passo osservabile del comportamento. La lettura consigliata interpreta il diagramma come una traccia esecutiva:

- primo messaggio, eventuale risposta, passo successivo;
- in presenza di rami, capire quale condizione seleziona il percorso.

**Lifeline:** Le lifeline identificano i partecipanti coinvolti nello scenario. Qui distinguiamo attore, interfaccia e servizio: ruoli diversi, responsabilità diverse. L'obiettivo è rendere esplicito chi partecipa prima di parlare dei dettagli dei messaggi. Quando usarlo: all'inizio, per fissare i confini dello scenario. Errore tipico: introdurre partecipanti non necessari al caso in esame.



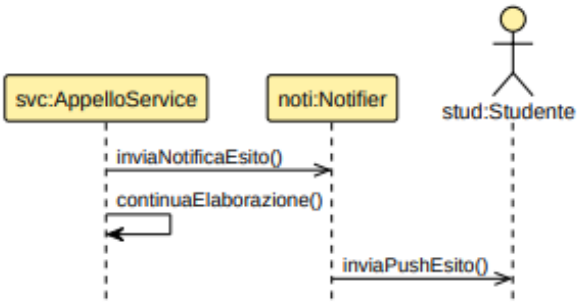

**Attivazione:** La barra di attivazione indica il periodo in cui il partecipante sta eseguendo lavoro. Nel diagramma si vede chiaramente quando *svc:AppelloService* è attivo e quando restituisce il controllo a *ui:Portale*. La freccia tratteggiata *esito(OK)* è un return message: rappresenta la risposta alla chiamata, non un messaggio asincrono. Serve a leggere meglio durata logica e nidificazione delle chiamate. Quando usarlo: quando la durata logica del lavoro è rilevante. Errore tipico: disegnare attivazioni ovunque senza valore informativo.



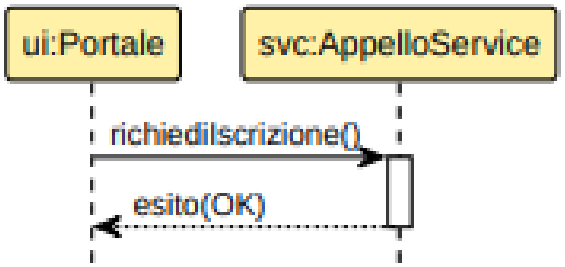

**Messaggio sincrono:** Il chiamante invia il messaggio e attende il completamento. Nel nostro esempio *ui:Portale* invia la richiesta a *svc:AppelloService* e resta in attesa dell'esito. Il messaggio sincrono rappresenta una dipendenza temporale immediata tra chiamata e risultato. Quando usarlo: per chiamate con dipendenza immediata dal risultato. Errore tipico: usare sync anche per notifiche/eventi che non bloccano.

**Messaggio asincrono:** Il chiamante invia il messaggio ma prosegue senza blocco. Nel nostro esempio *svc:AppelloService* invia una notifica a *noti:Notifier* senza attendere risposta immediata. Qui il focus è la

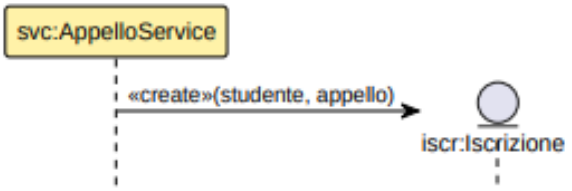
disaccoppiata temporale tra chi invia e chi riceve. Quando usarlo: in integrazioni event-driven o code di messaggi. Errore tipico: usare async senza chiarire chi gestisce l'esito.




**Return message:** La freccia tratteggiata rappresenta il risultato della richiesta. Qui *svc:AppelloService* restituisce a *ui:Portale* l'esito della richiesta (OK/errore). Nelle slide useremo il return solo quando porta informazione utile. Quando usarlo: se il valore di ritorno guida una decisione successiva. Errore tipico: mostrare tutti i return e appesantire il diagramma.



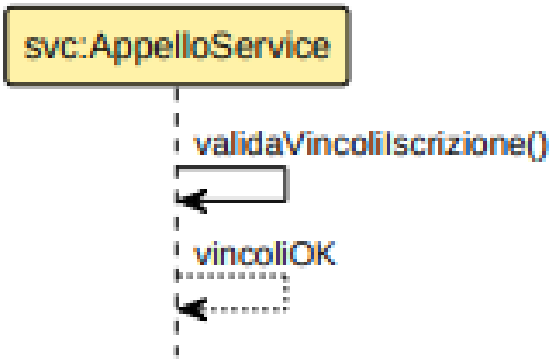

**Create:** Mostra che un oggetto nasce durante l'interazione, non esiste dall'inizio. In UniManager, *svc:AppelloService* crea una nuova *iscr:Iscrizione* quando i vincoli sono soddisfatti. Questo chiarisce ownership e momento di istanziazione. Quando usarlo: se il momento di creazione è parte del comportamento. Errore tipico: creare oggetti tecnici ovunque invece di riusarli.



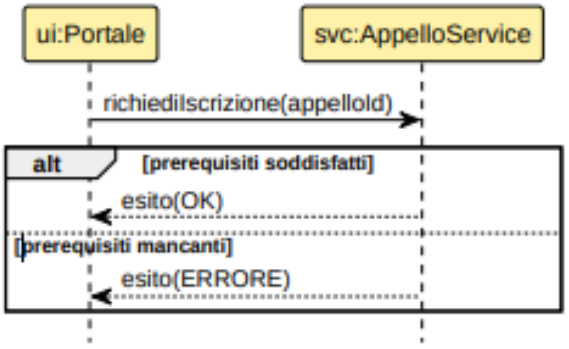

**Destroy:** Rappresenta la fine del ciclo di vita di un oggetto temporaneo. In UniManager, *svc:AppelloService* può eliminare una *bozza:Iscrizione* scaduta/non confermata. Utile per risorse effimere e cleanup esplicito. Quando usarlo: per risorse temporanee con fine vita esplicita. Errore tipico: distruggere entità che dovrebbero restare persistenti.



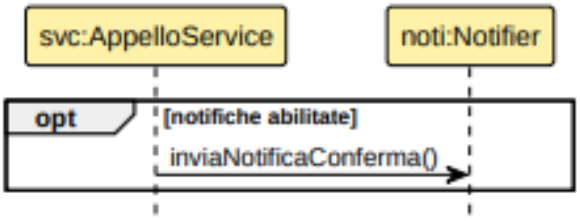

**Self-call:** Un partecipante può invocare una propria operazione interna. In UniManager, *svc:AppelloService* richiama internamente *validaVincoliIscrizione()* prima di proseguire. Evitiamo eccesso di self-call: solo quelle che chiariscono il comportamento. Quando usarlo: per evidenziare una sotto-fase interna significativa. Errore tipico: dettagliare micro-metodi irrilevanti per lo scenario.



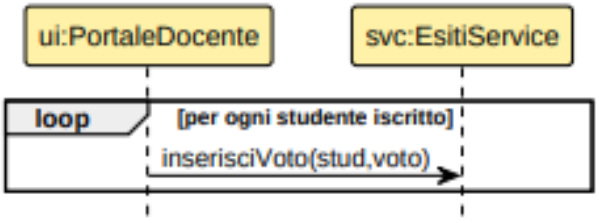

**Frammento alt:** `alt` modella rami alternativi mutuamente esclusivi. Nel nostro esempio, *svc: AppelloService* sceglie il ramo in base ai prerequisiti dello studente. È la forma naturale per success/failure nello stesso scenario. Quando usarlo: per rami alternativi che non possono coesistere. Errore tipico: guardie sovrapposte o non mutuamente esclusive.



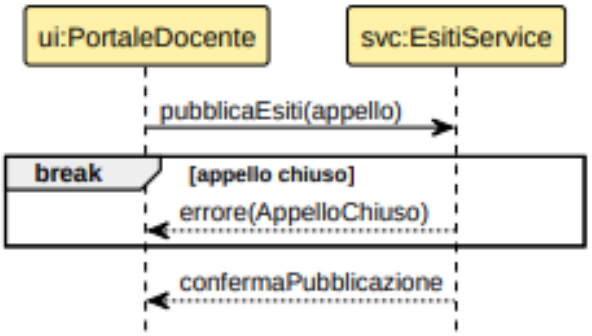

**Frammento opt:** `opt` esprime un comportamento opzionale (if senza else). Il flusso principale resta valido anche senza il blocco opzionale. Tipico per notifiche, log o passi accessori. Quando usarlo: per comportamenti accessori non sempre presenti. Errore tipico: usare opt per passi in realtà obbligatori.



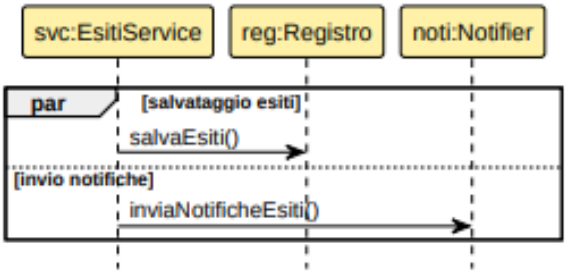

**Frammento loop:** `loop` rappresenta una ripetizione controllata da condizione. Qui la stessa operazione si applica a più elementi. Il blocco rappresenta lo stesso micro-flusso ripetuto secondo un criterio esplicito. Quando usarlo: su operazioni ripetute con criterio esplicito. Errore tipico: loop senza condizione chiara o troppo generica.




**Frammento break:** `break` interrompe il normale scenario in caso eccezionale. Quando il break scatta, il resto del flusso principale non prosegue. Utile per errori critici e stop immediato dell'operazione. Quando usarlo: per uscita anticipata in condizioni eccezionali. Errore tipico: usarlo al posto di un normale ramo alt.



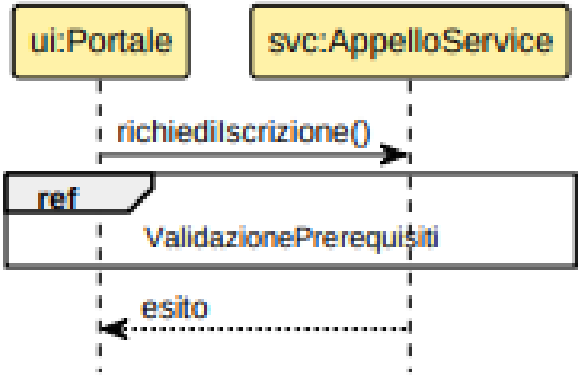

**Frammento par:** `par` descrive rami concorrenti eseguibili in parallelo. Nel mini-esempio salvataggio e notifica sono indipendenti. Aiuta a discutere latenza e sincronizzazione senza entrare nel codice. Quando usarlo: per attività indipendenti eseguibili in parallelo. Errore tipico: segnare par su rami che in realtà si bloccano.



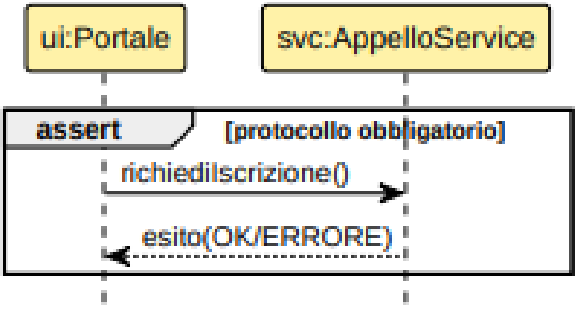

**Frammento critical:** `critical` delimita una sezione che non deve essere interrotta da altre interazioni concorrenti. Serve per proteggere aggiornamenti sensibili da interferenze. Lo usiamo per mettere in evidenza vincoli di mutua esclusione. Quando usarlo: su sezioni che richiedono mutua esclusione. Errore tipico: marcare blocchi lunghi senza reale criticità.



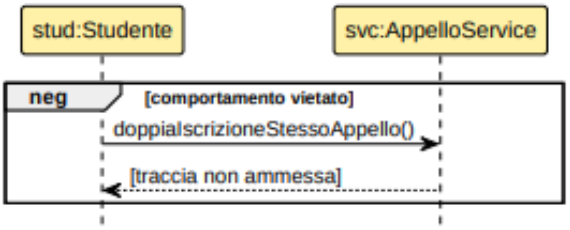

**Frammento ref:** `ref` rimanda a un'interazione definita altrove, in modo analogo alla chiamata di una funzione. Riduce duplicazione quando lo stesso sotto-scenario ricorre spesso. Mantiene il diagramma principale leggibile e modulare. Quando usarlo: per riuso di sotto-scenari comuni. Errore tipico: nascondere dentro ref passaggi fondamentali del caso.




**Frammento assert:** `assert` indica una traccia che deve valere obbligatoriamente. Nel diagramma usiamo un blocco etichettato assert per evidenziare il vincolo. È utile per evidenziare protocolli o vincoli forti. Se la traccia reale diverge, siamo fuori specifica. Quando usarlo: per specificare sequenze obbligatorie di protocollo. Errore tipico: usarlo senza una regola verificabile a supporto.




**Frammento neg:** `neg` rappresenta una traccia vietata. Nel diagramma usiamo un blocco etichettato neg per evidenziare il contro-esempio. Qui evidenziamo un comportamento che il sistema non deve ammettere. È didatticamente utile per discutere contro-esempi. Quando usarlo: per rendere espliciti comportamenti vietati. Errore tipico: confondere neg con un ramo alternativo valido.



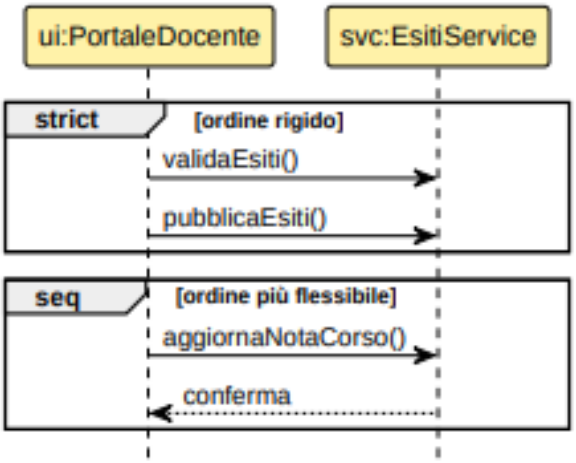

**Frammenti strict vs seq:**

- `strict`: ordine globale totale tra eventi (no interleaving).
- `seq`: ordine parziale (weak); preserva solo i vincoli locali. Con `seq` è possibile interleaving tra lifeline diverse. `seq` è il comportamento di default nei diagrammi UML.

Uso:

- `strict`: protocolli o scenari con ordine rigidamente definito.
- `seq`: sistemi concorrenti, per evitare over-constraining.

Errore tipico: usare strict quando basta seq, irrigidendo il modello.




## Activity diagram

Con i Sequence Diagram abbiamo visto chi parla con chi e in quale ordine. Ora cambiamo punto di vista:

- non più il dettaglio dei messaggi tra oggetti;
- ma il flusso complessivo del processo.

L'Activity Diagram è utile per analizzare:

- passi operativi end-to-end;
- decisioni e guardie;
- parallelismi e sincronizzazioni

Uso l'activity diagram:

- Per descrivere flussi di attività e regole decisionali
- Focus sul processo, non sui singoli messaggi

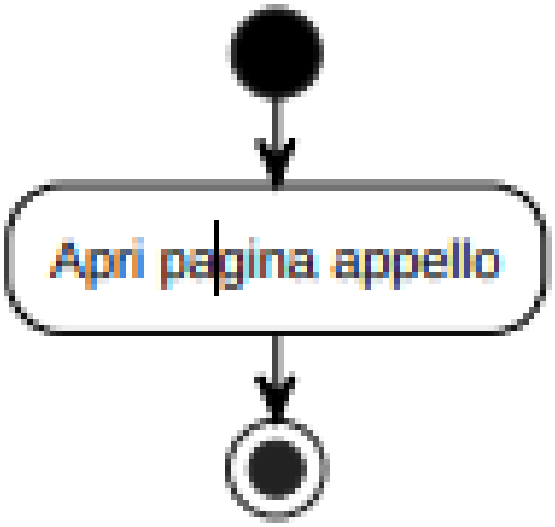
**Lettura di base:**

- Nodo iniziale/finale, azioni e archi
- Decisioni con guardie, parallelismo con fork/join

**Initial e final:** Ogni Activity parte da un nodo iniziale e termina in uno finale. Notazione grafica:

- nodo iniziale: cerchio pieno (nero);
- nodo finale: cerchio pieno con bordo esterno (bersaglio).

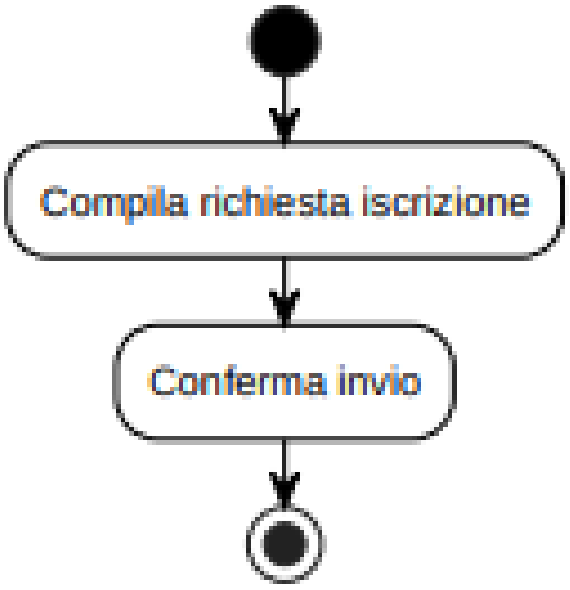
Questa cornice minima definisce inizio e fine del processo. Anche negli esempi semplici è bene mantenerla esplicita. Quando usarlo: sempre, per delimitare chiaramente il processo. Errore tipico: final implicito che lascia il flusso ambiguo.




**Action e control flow:** Le azioni sono i passi di lavoro; gli archi definiscono l'ordine. Notazione grafica:

- azione: rettangolo con angoli arrotondati;
- control flow: freccia piena tra un'azione e la successiva.

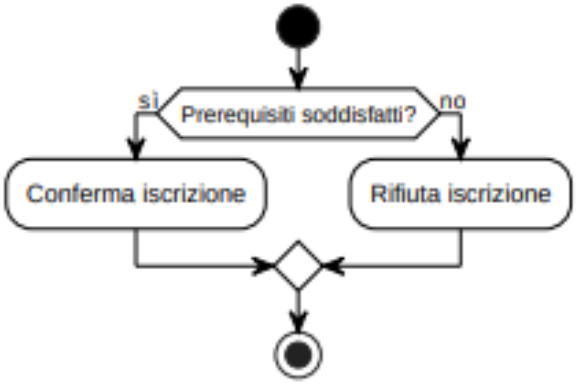
Qui il flusso è lineare: un passo prepara il successivo. È la base su cui poi aggiungiamo decisioni e parallelismo. Quando usarlo: per modellare la sequenza operativa principale. Errore tipico: azioni troppo grandi che nascondono decisioni



**Decision e merge:** Decision separa i rami; merge li ricompone. Notazione grafica:

- decision: rombo con un ingresso e più uscite;
- merge: rombo con più ingressi e una sola uscita.

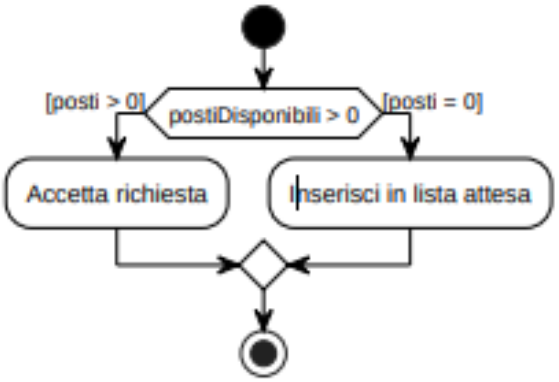
Le guardie devono essere chiare e possibilmente mutualmente esclusive. Nel diagramma si vede il classico esito positivo/negativo. Quando usarlo: quando un punto del processo biforca i casi. Errore tipico: dimenticare il merge e lasciare rami scollegati




**Guardie:** Le guardie esplicitano la condizione logica su ogni ramo. Notazione grafica:

- ogni uscita dal nodo decision ha una condizione tra parentesi quadre, ad esempio (posti > 0);
- le guardie devono coprire i casi rilevanti e non sovrapporsi.

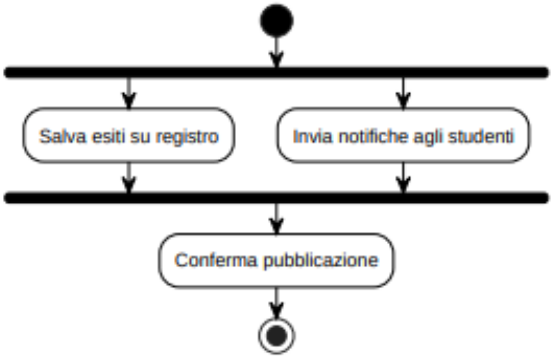
Etichette come (posti > 0) evitano ambiguità interpretative. Sono fondamentali per verificare completezza dei casi. Quando usarlo: per esplicitare condizioni precise sui rami. Errore tipico: guardie vaghe (es. 'ok') non verificabili.




**Fork e join:** Fork apre esecuzioni concorrenti; join le sincronizza. Notazione grafica:

- fork: barra nera spessa con un ingresso e più uscite;
- join: barra nera spessa con più ingressi e una sola uscita.

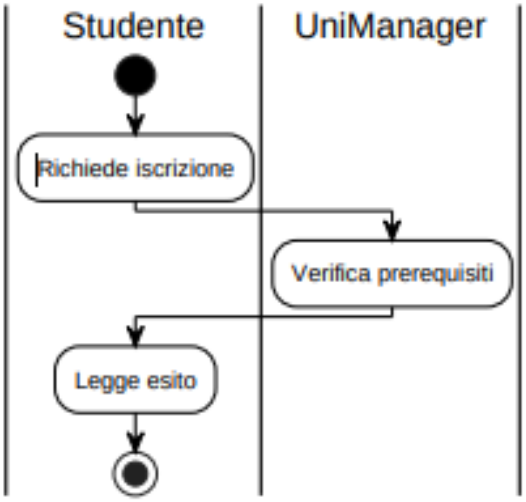
Utile quando due attività sono indipendenti ma devono convergere. Qui salvataggio e notifica procedono in parallelo. Quando usarlo: per parallelismo reale con sincronizzazione finale. Errore tipico: fork senza join quando la convergenza è richiesta.



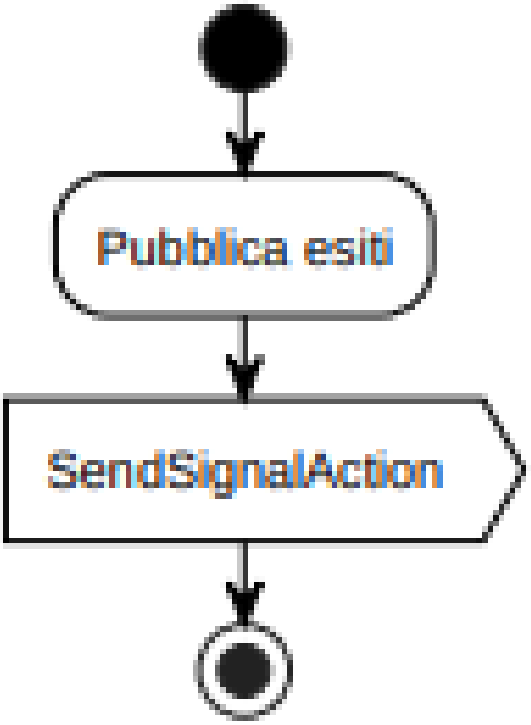
**Swimlane:** Le swimlane assegnano responsabilità organizzative ai passi. Notazione grafica:

- il diagramma è diviso in partizioni (corsie) etichettate;
- ogni azione è collocata nella corsia del ruolo/sistema responsabile.

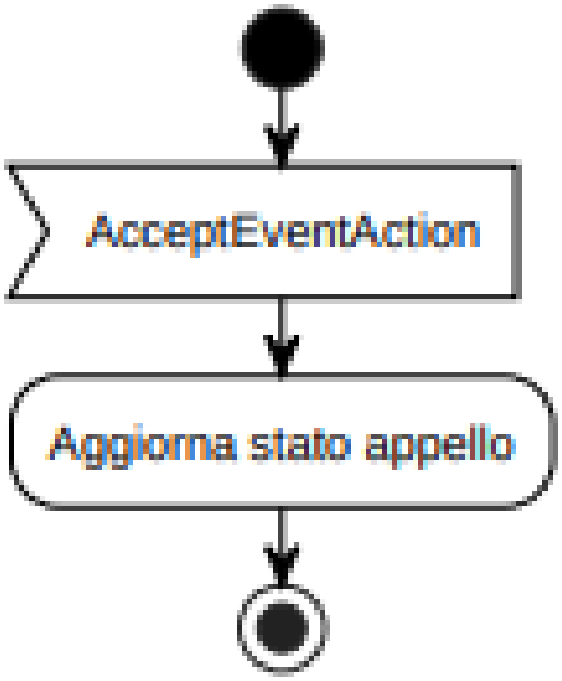
Si capisce subito chi fa cosa (studente vs sistema). Ottimo strumento di comunicazione con stakeholder non tecnici. Quando usarlo: per chiarire ownership tra ruoli/sistemi. Errore tipico: cambiare lane senza criterio di responsabilità.



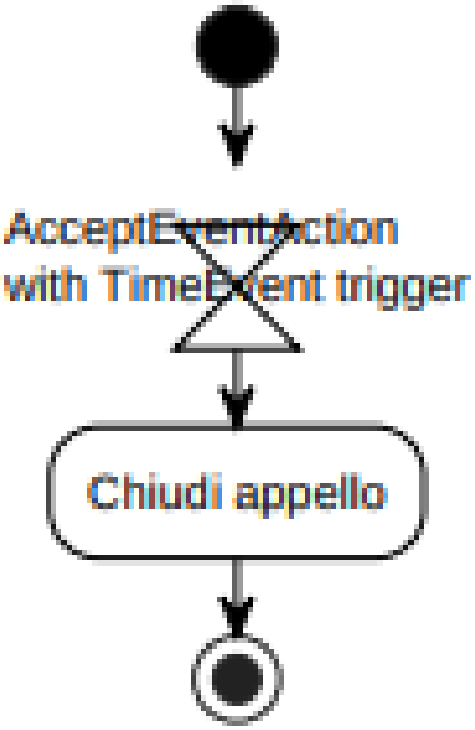

**Send signal:** L'attività emette un segnale verso l'esterno. Notazione grafica: azione di invio evento, con flusso che prosegue senza attesa di risposta immediata. È un punto di integrazione event-driven nel processo. Qui l'evento comunica che gli esiti sono disponibili. Quando usarlo: quando il processo pubblica un evento esterno. Errore tipico: usare send signal per normali chiamate sincrone.



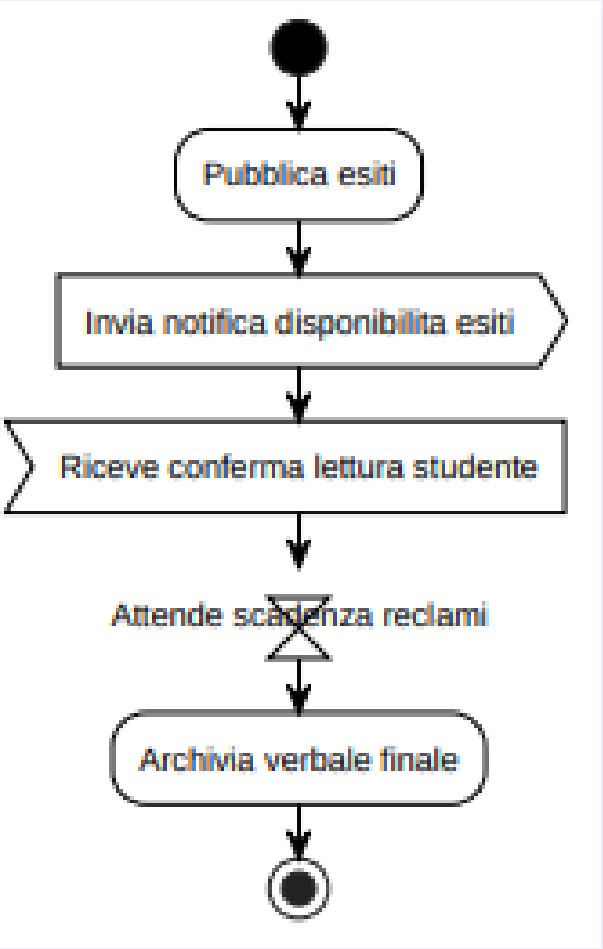

**Receive signal:** Il flusso resta in attesa di un segnale in ingresso. Notazione grafica: azione di ricezione evento, che abilita la prosecuzione del flusso. La prosecuzione dipende da un evento esterno. Utile per modellare processi reattivi e asincroni. Quando usarlo: per stati in attesa di input/evento esterno. Errore tipico: non specificare chi emette il segnale atteso.




**Time event:** Il processo reagisce a una scadenza temporale. Notazione grafica: azione di attesa temporale, che abilita una transizione al verificarsi della scadenza. Rende esplicite le transizioni guidate dall'orologio. Nel nostro esempio, il flusso prosegue quando viene raggiunta la scadenza delle iscrizioni. Quando usarlo: per transizioni innescate dal tempo. Errore tipico: modellare tempi con testo libero invece di eventi

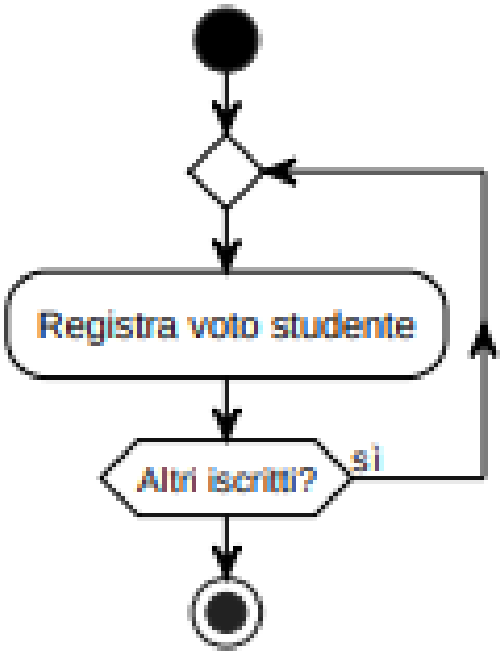



Esempio con send, receive e time

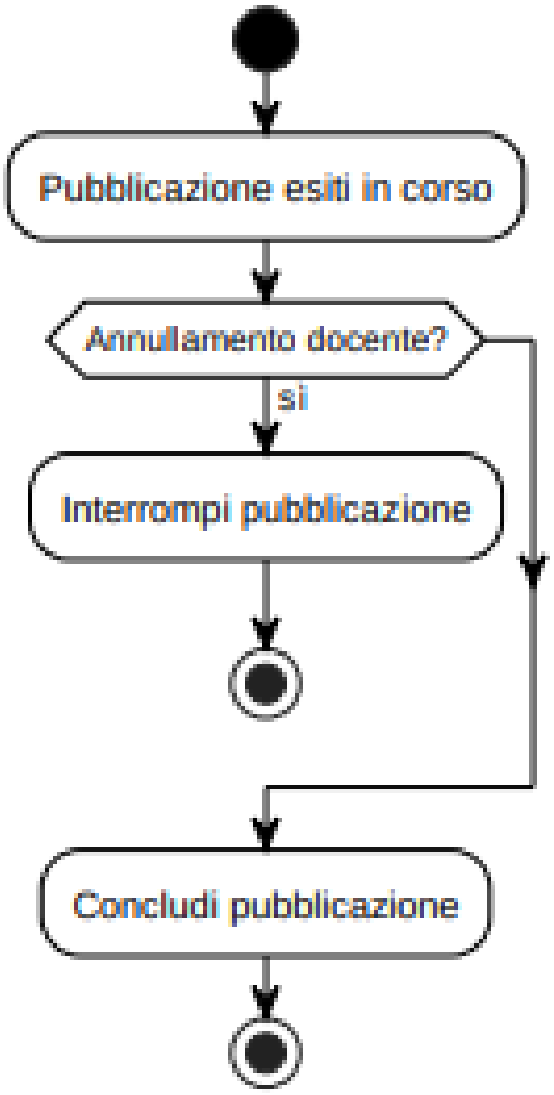



- Lo scenario combina in un unico flusso invio evento, attesa evento e transizione temporale.
- Sequenza logica: pubblicazione esiti, notifica esterna, conferma ricevuta, attesa scadenza, archiviazione finale.
- Il diagramma chiarisce come eventi esterni e tempo guidano avanzamento e chiusura del processo.

**Loop structure:** Un blocco viene ripetuto finché la condizione resta vera. Qui il ciclo copre tutti gli elementi da elaborare. Esplicitare il loop evita di nascondere lavoro ripetitivo. Quando usarlo: per cicli di processo su insiemi di elementi. Errore tipico: non esplicitare condizione di uscita del ciclo.



**Interruptable region:** Una regione può essere interrotta da un evento di annullamento. Il diagramma evidenzia il ramo di uscita anticipata. Fondamentale per processi lunghi con comando stop. Quando usarlo: su processi lunghi annullabili dall'utente. Errore tipico: annullamento modellato ma senza cleanup logico.




**Confronto finale:**

- Sequence Diagram:
  - risponde a: chi interagisce con chi, in quale ordine, con quali messaggi;
  - è ideale per analizzare responsabilità tra componenti e interfacce.
- Activity Diagram:
  - risponde a: quali passi compongono il processo, con quali decisioni/parallelismi;
  - è ideale per descrivere il flusso end-to-end anche a livello di business.
- Criterio di scelta:
  - usa Sequence quando il rischio è distribuire male la logica tra oggetti;
  - usa Activity quando il rischio è perdere la visione complessiva del processo.

## Principi di architettura software

Nelle tre lezioni UML precedenti abbiamo usato cinque diagrammi per capire che cosa deve fare il sistema:

1. Use Case Diagram per funzionalità e attori;
2. Domain Model per i concetti del dominio;
3. Class Diagram per la struttura software;
4. Sequence Diagram per i messaggi nel tempo;
5. Activity Diagram per il flusso di processo.

Questa analisi ci dà una buona fotografia del comportamento e del dominio di UniManager. Il passo successivo è chiederci: come organizziamo il software in modo che questi elementi siano implementati in maniera ordinata, comprensibile e mantenibile?

L'architettura software è il ponte tra: l'analisi (requisiti, casi d'uso, modelli UML), e l'implementazione concreta (classi, moduli, codice). Se l'analisi ci dice cosa deve succedere, l'architettura ci dice come organizzare il sistema perché quelle funzionalità possano essere realizzate in modo coerente. Un'architettura chiara rende più semplice:

- assegnare il lavoro a più sviluppatori,
- prevedere l'impatto dei cambiamenti,
- discutere le scelte progettuali usando una mappa condivisa.




**Parliamo di architettura in questo momento del corso perché:**

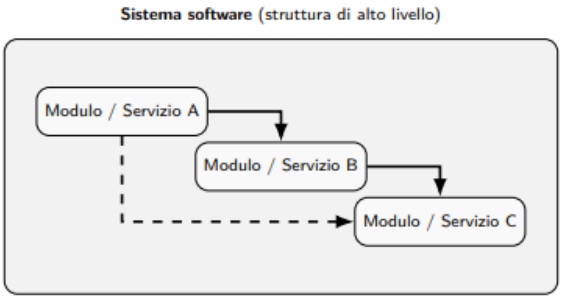
- abbiamo già compreso il dominio di UniManager (gestione di corsi, appelli, iscrizioni e voti),
- abbiamo identificato gli attori e i casi d'uso principali di questo dominio (Studente, Docente, Segreteria),
- abbiamo un modello concettuale stabile degli elementi centrali del dominio (Studente, Corso, Appello, Iscrizione).

È il momento di decidere come strutturare il codice in modo che rispecchi questa comprensione, evitando di arrivare all'implementazione con un insieme di file disorganizzati. Le scelte architetturali prese all'inizio sono difficili da cambiare e condizionano l'evoluzione futura del sistema: per questo è importante renderle esplicite.

**Che cos'è l'architettura software?** Possiamo definire l'architettura software come:

- la struttura di alto livello del sistema,
- la suddivisione in componenti principali (moduli, servizi, sottosistemi),
- le relazioni e le dipendenze tra questi componenti.

L'architettura non entra nei dettagli di ogni singolo metodo, ma definisce il contenitore in cui questi dettagli andranno a collocarsi. Una buona architettura aiuta chiunque, anche non autore del codice, a farsi un'idea del sistema “a colpo d'occhio”.




Componenti principali collegati da **relazioni** e **dipendenze** ben definite

Possiamo considerare l'architettura software a due livelli:

1. *in piccolo:* riguarda la struttura interna di un singolo sistema applicativo, come UniManager. Ci chiediamo come suddividere il software in componenti, servizi, moduli.
2. *in grande:* riguarda sistemi più complessi che coinvolgono molte applicazioni, servizi distribuiti, componenti remoti o cloud. Qui le decisioni architetturali includono protocolli, distribuzione, scalabilità.

Nel corso ci concentreremo soprattutto sull'architettura in piccolo, perché è la base su cui costruire soluzioni più complesse. Il passaggio all'architettura “in grande” avviene spesso in modo naturale, man mano che un sistema cresce e integra componenti esterni.

**Esempio concreto di architettura “in grande”:**

- una piattaforma e-commerce come Amazon, con sistemi distinti per catalogo, ordini, pagamenti, spedizioni e raccomandazioni,

- applicazioni dedicate per utenti finali e operatori (web, app mobile, app per venditori e logistica),
- servizi distribuiti e comunicazione tra sistemi tramite API/eventi.

L'architettura è anche l'insieme delle decisioni strutturali che:

- sono difficili da cambiare una volta implementate,
- influenzano molte parti del sistema,
- impattano su prestazioni, manutenibilità, riusabilità.

**Alcuni esempi di decisioni architetturali:**

- separare o meno la logica di business dall'accesso ai dati,
- usare o meno livelli distinti per presentazione, logica, persistenza,
- organizzare il codice per funzionalità (Studenti, Corsi, ... ) o per tipo di responsabilità (domain = entità e regole di business, service = logica applicativa/casi d'uso, repository = accesso ai dati e persistenza).

Queste decisioni definiscono confini e regole di dipendenza:

- chi può chiamare chi,
- dove può risiedere una regola di business,
- quali componenti possono conoscere dettagli tecnologici.

**In UniManager, per esempio, decidere che la CLI parli solo con i Service e non direttamente con i Repository è una decisione architetturale, non di dettaglio.**

**Progettare l'architettura:** significa prendere una serie di decisioni che influenzano profondamente il sistema, per esempio:

- come suddividere il software in componenti,
- quale forma far assumere ai confini tra i componenti,
- come far comunicare i diversi moduli tra loro,
- quali dipendenze ammettere e quali vietare,
- quanto isolare il dominio dai dettagli tecnologici.

Ogni decisione architetturale ha un costo e un beneficio: l'obiettivo è trovare l'equilibrio che rende il sistema comprensibile, efficace e manutenibile. L'architettura emerge quindi non come una singola scelta, ma come un insieme coerente di decisioni che lavorano insieme.

**Architettura vs design di dettaglio:** È utile distinguere l'architettura dal design di dettaglio:

- l'architettura definisce i grandi blocchi e le loro relazioni,
- il design di dettaglio decide come questi blocchi sono implementati internamente.

Le due dimensioni non sono in conflitto:

- l'architettura crea i confini (chi fa cosa),
- il design di dettaglio popola questi confini con classi, metodi e algoritmi specifici.

**Esempi tipici di design di dettaglio:**

- firma dei metodi e scelta dei parametri (iscriviStudente(matricola, idAppello));
- struttura delle classi interne a un modulo e responsabilità dei singoli oggetti;
- algoritmo di ordinamento/ricerca adottato in un servizio;
- gestione puntuale di eccezioni, validazioni e messaggi di errore.

**Strumento per il team:** In un progetto reale, l'architettura non è solo un concetto tecnico, ma un strumento di comunicazione:

- permette a sviluppatori, progettisti e responsabili di parlare del sistema con un linguaggio comune,
- aiuta a spiegare rapidamente “come è fatto” il software a chi entra nel team.

Sapere in quale componente collocare una nuova funzionalità riduce discussioni inutili e favorisce un'evoluzione ordinata del codice. Un'architettura condivisa rende più semplice anche la revisione del codice, perché si può valutare se una modifica rispetta o meno i confini stabiliti.

**Se non ci fosse l'architettura:** Se l'architettura non viene pensata o viene lasciata “emergere” senza guida, il rischio è che il sistema si trasformi in una *“big ball of mud”:*

- file molto grandi e confusi,
- logica di business mescolata all'accesso ai dati,
- interfaccia utente che conosce dettagli interni del dominio.

In questo scenario ogni modifica è potenzialmente pericolosa, nessuno sa esattamente che cosa rompe cambiando una riga e si generano “zone del codice” che nessuno vuole toccare.

**Obiettivi concreti dell'architettura:** Una buona architettura si pone obiettivi molto pratici:

- ridurre la complessità percepita del sistema,
- facilitare i cambiamenti nel tempo, isolandoli dove possibile,
- favorire la testabilità, separando ciò che è facilmente verificabile da ciò che dipende dall'ambiente,
- supportare la crescita del progetto senza doverlo riscrivere da zero ogni anno.

Nel caso di UniManager, vogliamo che aggiungere una funzionalità (es. un nuovo tipo di report) non significhi toccare codice in dieci posti diversi.

**Architettura e requisiti non funzionali:** L'architettura non serve solo a organizzare il codice: è lo strumento principale per soddisfare molti requisiti non funzionali. Aspetti come:

- prestazioni (tempi di risposta, carico),
- affidabilità e tolleranza ai guasti,
- scalabilità,
- sicurezza,
- manutenibilità ed estendibilità,

dipendono molto più dalle scelte architetturali che dal codice delle singole classi

Un'architettura ben pensata permette di prevedere come il sistema si comporterà sotto stress e come potrà evolvere in futuro. Per questo motivo, le decisioni architetturali vengono prese molto presto nel progetto e orientano molte attività successive.

**Decisioni architetturali e trade-off:** In architettura non esistono scelte “gratis”: quasi ogni decisione porta vantaggi su alcuni aspetti e costi su altri.

> **Esempi tipici di trade-off:**
>
> - più livelli e confini → maggiore manutenibilità, ma maggiore complessità iniziale;
> - moduli molto indipendenti → migliore evoluzione, ma più interfacce da progettare;
> - accesso ai dati centralizzato → coerenza, ma possibile collo di bottiglia.

Per questo una decisione architetturale va sempre motivata rispetto a:

- requisiti prioritari del progetto,
- vincoli tecnici e organizzativi,
- costo di implementazione e di evoluzione.

**Documentare le decisioni architetturali:** Anche in progetti didattici conviene rendere esplicite le decisioni architetturali, evitando che restino solo “nella testa” di chi le ha proposte. Un formato minimo efficace può contenere:

- *contesto:* quale problema dobbiamo risolvere;

- *decisione:* quale scelta adottiamo;
- *alternative:* quali opzioni abbiamo scartato e perché;
- *conseguenze:* effetti attesi, rischi, impatti.

Buona pratica operativa:

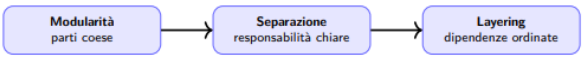
- una decisione per documento, titolo chiaro e data;
- aggiornamento del documento quando la decisione cambia.

> **Esempio in UniManager:**
>
> “i comandi CLI invocano solo i Service, senza accesso diretto ai Repository”. Questo riduce ambiguità nel team e facilita revisioni future.

**Tre principi fondamentali:** Per lavorare sull'architettura useremo come filo conduttore tre principi semplici ma molto potenti:

- *modularità:* dividere il sistema in parti coese,
- *separazione delle responsabilità:* evitare componenti “tuttofare”,
- *layering:* organizzare il sistema in livelli con ruoli distinti.

Questi principi non sono legati a un linguaggio specifico, e valgono anche oltre l'OS, come criterio generale di organizzazione.




**Vocabolario operativo: CLI/UI, Service, Domain, Repository:** Fissiamo i quattro concetti che useremo nel resto del corso:

- *CLI/UI (Command-Line Interface / User Interface):* riceve input utente e mostra output, senza contenere regole di business;
- *Service:* coordina i casi d'uso applicativi (sequenza dei passi, validazioni applicative, gestione esiti/errore) e orchestra le interazioni tra Domain e Repository;
- *Domain:* rappresenta entità e regole del dominio (vincoli, invarianti, significato dei dati), indipendenti da UI e persistenza;
- *Repository:* si occupa di accesso ai dati e persistenza (lettura/scrittura).

Regola guida:

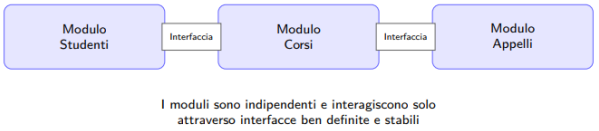
- la *regola di business* vive tra Service e Domain;
- CLI/UI e Repository sono livelli di interazione, non sede della logica di dominio.




> **Esempio di una decisione in UniManager**
>
> - Problema: dove collocare la regola “uno studente non può iscriversi due volte allo stesso appello”?
> - Possibili scelte:
>   - nella CLI/UI (scelta errata: vincolo di dominio in livello di presentazione),
>   - nel Repository (scelta parziale: verifica tecnica sui dati ma non governo del caso d'uso),
>   - in Service + Domain (scelta coerente con le responsabilità architetturali).
> - Decisione architetturale coerente:
>   - la CLI/UI invia la richiesta di iscrizione al Service;
>   - il Service applica la regola di business usando concetti e vincoli di Domain;
>   - il Repository viene usato solo per leggere/scrivere lo stato persistente;
>   - l'esito (OK/errore) torna alla CLI/UI senza esporre dettagli di persistenza.

**Primo principio- modularità:** La modularità consiste nel dividere il sistema in parti che:

- hanno un ruolo chiaro,
- comunicano tramite interfacce ben definite,
- sono ragionabili e testabili quasi indipendentemente.



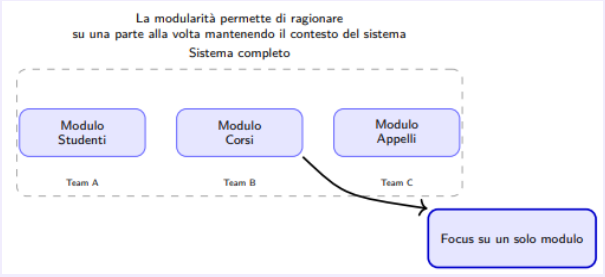

I moduli sono indipendenti e interagiscono solo attraverso interfacce ben definite e stabili

**Perché la modularità è essenziale?** La modularità riduce la porzione di sistema da analizzare per prendere una decisione progettuale:

- possiamo concentrarci su un modulo alla volta,
- il resto del sistema viene visto come un “servizio” con alcune funzioni disponibili.

Un sistema modulare:

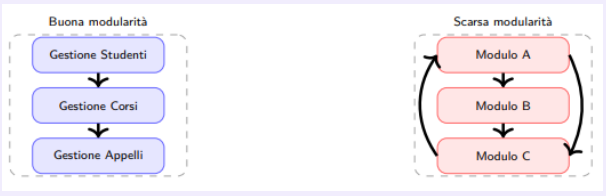
- si presta meglio al lavoro in team (ogni gruppo segue un modulo),
- facilita il riuso (un modulo può essere sfruttato in altri contesti),
- rende più prevedibili gli impatti di ogni modifica.




**Segnali di buona e cattiva modularità:**

- *Buona modularità:* nomi dei moduli intuitivi e stabili nel tempo, interfacce semplici (pochi metodi, significativi), dipendenze chiare, idealmente in una sola direzione.
- *Scarsa modularità:* moduli che cambiano continuamente scopo, componenti che conoscono dettagli interni l'uno dell'altro, dipendenze incrociate difficili da districare.

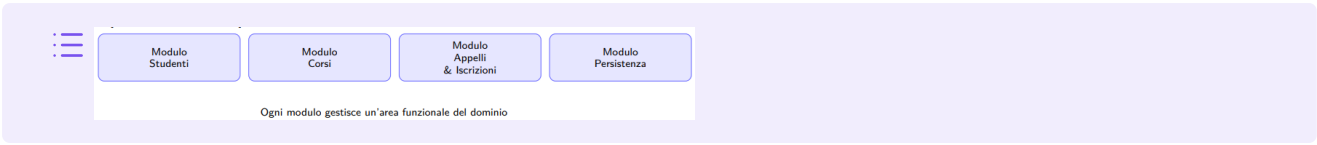
**Example**




**Modularità funzionale in UniManager:** Guardando ai requisiti di UniManager, emergono in modo naturale alcune aree funzionali:

- gestione degli Studenti,
- gestione dei Corsi,
- gestione degli Appelli e delle Iscrizioni,
- gestione della persistenza delle informazioni.

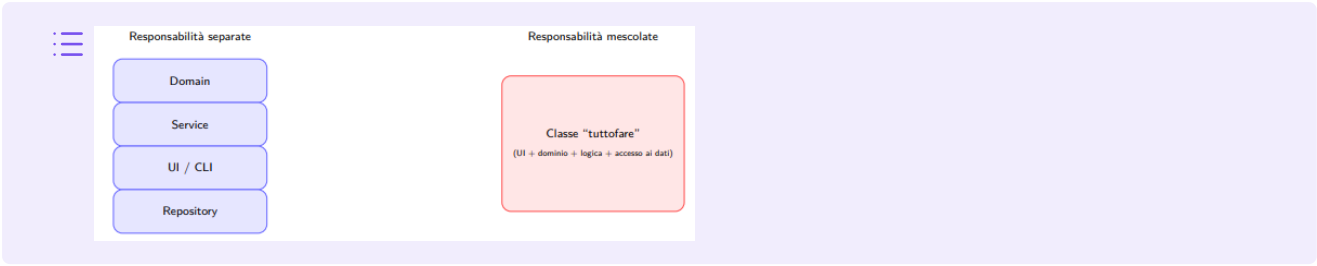
Possiamo quindi immaginare di avere, almeno concettualmente, un modulo per ciascuna di queste aree, che si occupa di quella parte del dominio. Questa è una prima forma di modularità, che poi andremo a raffinare con il principio della separazione delle responsabilità.




**Secondo principio- separazione delle responsabilità:** La separazione delle responsabilità invita a evitare componenti che fanno “un po' di tutto”. Ogni parte del sistema dovrebbe avere una responsabilità principale ben riconoscibile:

- rappresentare dati del dominio,
- applicare regole di business,
- comunicare con l'utente,
- accedere al database.

Quando questi ruoli si mescolano, il codice diventa più difficile da capire, da testare e da modificare.




**Perché separare le responsabilità:** Una separazione chiara delle responsabilità offre diversi vantaggi:

- rende più semplice capire “a chi compete” un certo cambiamento,
- riduce il rischio di introdurre effetti collaterali non desiderati,
- permette di sostituire una parte (es. la persistenza) mantenendo le altre stabili.

Al contrario, una classe o un modulo che gestisce:

- logica del dominio,
- interfaccia utente,
- accesso ai file

è destinato a diventare un punto critico del sistema.

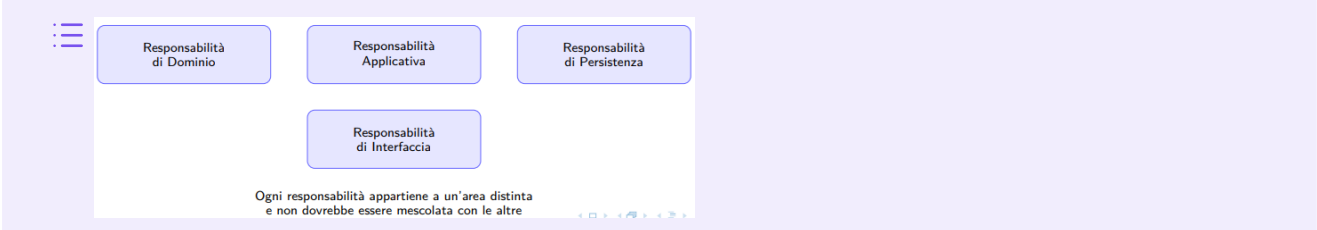
**Tipi di responsabilità in UniManager:** In questa slide adottiamo una vista per responsabilità tecniche (non per aree funzionali):

- è una prospettiva diversa da “modularità funzionale” (Studenti, Corsi, Appelli),
- serve a chiarire chi fa cosa tra i componenti tecnici.

In UniManager possiamo individuare almeno quattro tipi di responsabilità:

- dominio: rappresentare entità e regole del mondo universitario,
- logica applicativa: orchestrare i casi d'uso,
- persistenza: leggere e scrivere dati,
- interfaccia: interagire con l'utente.

Separare queste responsabilità ci aiuta a progettare componenti più puliti e facili da testare.




**Separazione in pratica- entità, servizi, repository, CLI:** Possiamo tradurre queste responsabilità in quattro famiglie di componenti:

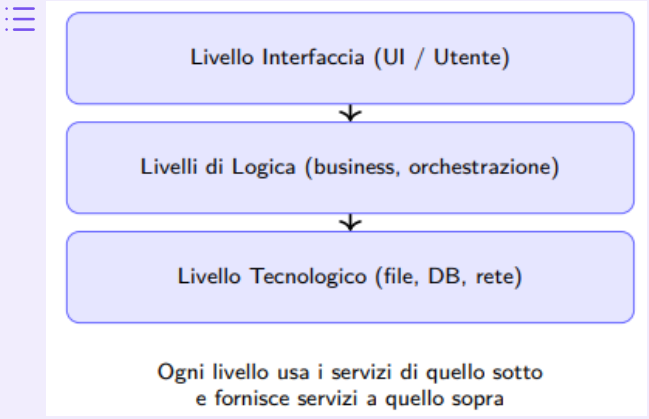
- *entità del dominio* (es. Studente, Corso, Appello), che contengono dati e regole del dominio;
- *servizi applicativi* (es. AppelloService), che implementano le operazioni richieste dai casi d'uso;
- *repository* (es. StudenteRepository), che si occupano di salvare e recuperare i dati;
- *componenti CLI/UI*, che gestiscono input e output verso l'utente.

In questo modo, ogni famiglia ha una responsabilità precisa e riconoscibile.

**Terzo principio- layering:** Il terzo principio riguarda il layering, cioè l'idea di organizzare il sistema in livelli di astrazione:

- un livello più vicino all'utente (interfaccia),
- livelli intermedi di logica,
- un livello più vicino alle tecnologie di base (file, database, rete).

Ogni livello offre servizi al livello superiore e sfrutta i servizi del livello inferiore, secondo regole di dipendenza ben precise.

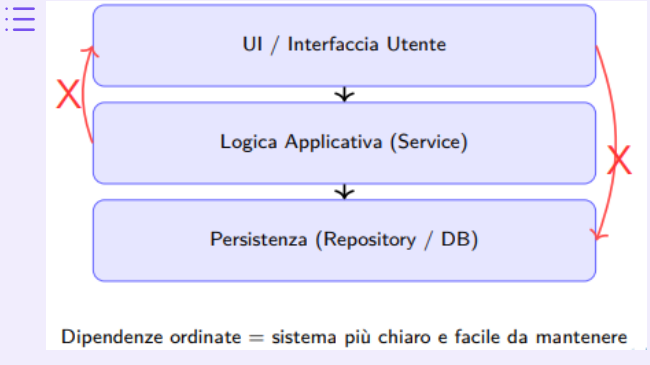



**Perché usare i layer:** I layer servono a tenere sotto controllo le dipendenze:

- il codice dell'interfaccia utente non dovrebbe conoscere i dettagli del database,
- la logica di business non dovrebbe dipendere da come i dati sono memorizzati.

I benefici principali sono:

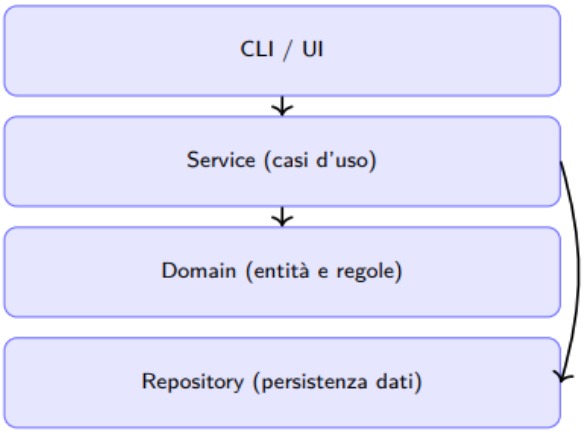
- maggiore chiarezza nella struttura,
- confini più netti per i test,
- facilità nel sostituire un livello (es. cambiare DB) mantenendo stabili gli altri.




**Un layering semplice per UniManager:** Per UniManager possiamo immaginare quattro livelli principali:

- *CLI/UI:* comandi testuali o schermate che l'utente utilizza;
- *Service:* implementazione dei casi d'uso;
- *Domain:* entità e regole del dominio universitario;
- *Repository:* accesso ai dati su file o database.

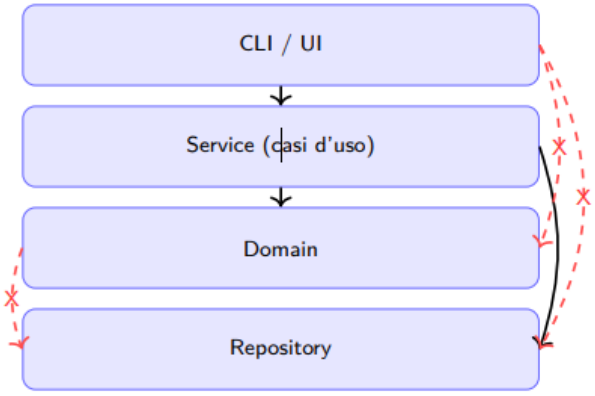
Questi livelli formano una “pila” dove ciascuno ha un ruolo distinto.




**Regole di comunicazione tra i layer:** Per mantenere l'architettura pulita, è utile stabilire alcune regole:

- la CLI/UI parla solo con il livello Service,
- i Service parlano con il Domain e i Repository,
- i Repository non devono dipendere dall'interfaccia utente,
- il Domain non dovrebbe conoscere dettagli di persistenza o di interfaccia.

In questo modo, le dipendenze hanno una direzione chiara e il dominio rimane al centro dell'architettura.




**Checklist rapida di qualità architetturale:** Domande operative da porre al modello architetturale:

- ogni componente ha una responsabilità principale chiara?
- le dipendenze rispettano i confini tra layer?
- una modifica tipica richiede tocchi localizzati o diffusi?
- il dominio resta isolato da UI e persistenza?
- i test possono concentrarsi su un livello alla volta?

Se molte risposte sono negative, l'architettura va rifinita prima di ampliare il codice.

**UniManager- una prima suddivisione in pacchetti:** Possiamo tradurre questi livelli in una prima proposta di struttura del progetto:

- cli, per l'interfaccia testuale o grafica,
- service, per i servizi applicativi,
- domain, per le entità e le regole del dominio,
- repository, per l'accesso ai dati.

Questa suddivisione ha il vantaggio di:

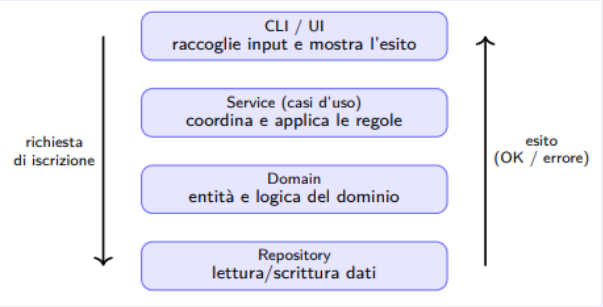
- rendere subito visibile la separazione delle responsabilità,
- dare una mappa chiara del sistema fin dai primi commit.




**Iscrizione a un appello attraverso i layer. Immaginiamo l'operazione "Studente si iscrive a un appello":**

- la CLI raccoglie l'input (matricola, appello) e invoca il servizio opportuno;
- il Service verifica le regole (esistenza di studente e appello, vincoli sulle iscrizioni) usando il Domain;
- il Service chiede al Repository di aggiornare la persistenza;
- il risultato torna verso la CLI, che mostra all'utente conferma o eventuali errori.

La responsabilità di ciascun passo è chiara e localizzata nel layer corretto.




## Stili architetturali

Nella lezione precedente abbiamo visto:

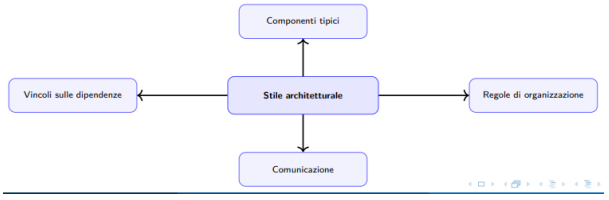
- che cos'è l'architettura software,
- perché è importante pensarla esplicitamente,
- tre principi chiave: modularità, separazione delle responsabilità, layering.

Oggi facciamo un passo avanti: vediamo come questi principi possono essere combinati in forme ricorrenti, cioè in stili architetturali. Obiettivo: dare un vocabolario per riconoscere le strutture tipiche dei sistemi e per discutere le scelte senza basarsi solo su tecnologie o mode del momento.

**Che cos'è uno stile architetturale?** Uno stile architetturale descrive:

- un insieme di componenti tipici,
- le regole di organizzazione (es. livelli, ruoli),
- le modalità di comunicazione tra i componenti,
- i vincoli sulle dipendenze.

Non è una tecnologia o un framework, è un modello concettuale che può essere implementato in molti modi diversi. Serve per ragionare in termini di forme globali del sistema, non di singole classi.




**Perché esistono più stili?** I sistemi software devono soddisfare esigenze molto diverse: interattività, prestazioni, scalabilità, affidabilità, integrazione con altri sistemi, ... Ogni stile è nato per rispondere meglio a alcuni di questi bisogni. Non esiste uno stile “migliore in assoluto”: ogni scelta comporta vantaggi e svantaggi, capire questi compromessi è parte del mestiere dell'ingegnere del software.

In questa lezione ci concentriamo su quattro stili molto diffusi:

1. *Layered:* organizzazione a livelli (es. presentazione, logica, dati).
2. *Client–server:* separazione tra chi chiede un servizio e chi lo fornisce.
3. *Repository:* più componenti che condividono una base dati comune.
4. *Service-based:* sistema composto da servizi indipendenti che espongono interfacce.

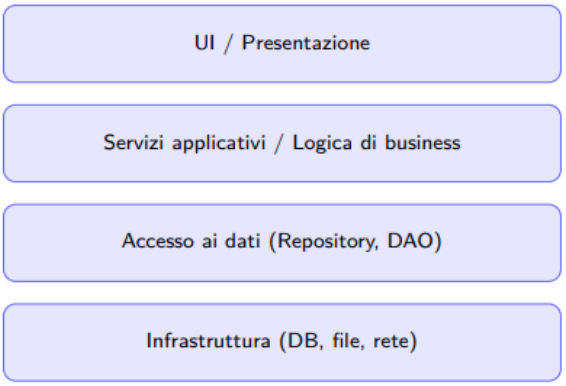
**Architettura layered:** Lo stile layered organizza il sistema in livelli di astrazione:

- ogni livello ha un ruolo specifico,
- i livelli comunicano tra loro secondo regole definite.

Tipicamente si distinguono livelli come:

- presentazione verso l'utente,
- logica di business / servizi applicativi,
- accesso ai dati e infrastruttura di base.

L'idea chiave è tenere separati i diversi tipi di responsabilità e controllare le dipendenze tra le parti del sistema. In pratica, layered risponde bene alla domanda: dove deve vivere una certa logica, per evitare che UI, regole di business e persistenza si mescolino. Non impone una tecnologia specifica: è uno schema organizzativo che può essere applicato in modo simile in stack diversi.




Ogni layer offre servizi a quello sopra e usa i servizi di quello sotto. Le dipendenze seguono una direzione prevalente: dall'alto verso il basso.

**Layered- vantaggi:**

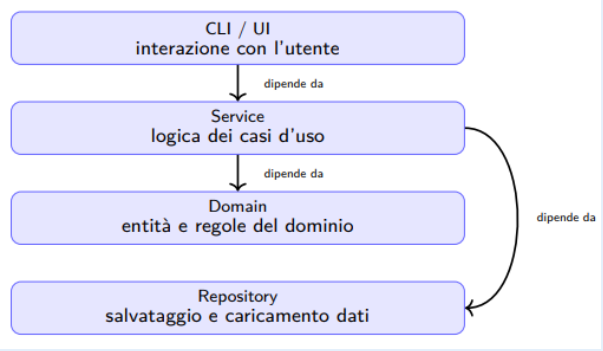
1. *Chiarezza strutturale:*
   - è facile spiegare “come è fatto” il sistema,
   - i confini tra responsabilità sono visibili.
2. *Facilità di manutenzione:*
   - cambiare il database può richiedere modifiche solo nel layer di accesso ai dati,
   - cambiare UI dovrebbe toccare principalmente il layer di presentazione.
3. *Testabilità:*
   - si possono testare i servizi senza coinvolgere né UI né DB reali (mock).
4. *Evoluzione controllata:*
   - una modifica in un layer tende a propagarsi meno agli altri, se le interfacce restano stabili.
5. *Onboarding del team:*
   - la struttura a livelli aiuta a capire rapidamente dove collocare nuova logica.

**Layered: limiti e compromessi:**

1. *Possibile overhead:*
   - le chiamate attraversano più layer, con qualche costo in termini di prestazioni.
2. *Rischio di rigidità:*
   - se i confini tra layer sono troppo rigidi, diventa difficile gestire casi “trasversali”.
3. *Necessità di disciplina:*
   - se i layer iniziano a “bucarsi” (UI che accede direttamente al DB), la struttura si degrada in fretta.
4. *Rischio di stratificazione artificiale:*
   - in sistemi piccoli, troppi layer possono introdurre complessità non necessaria.
5. *Possibile duplicazione di modelli:*
   - spesso gli stessi dati attraversano più layer con trasformazioni ripetitive.

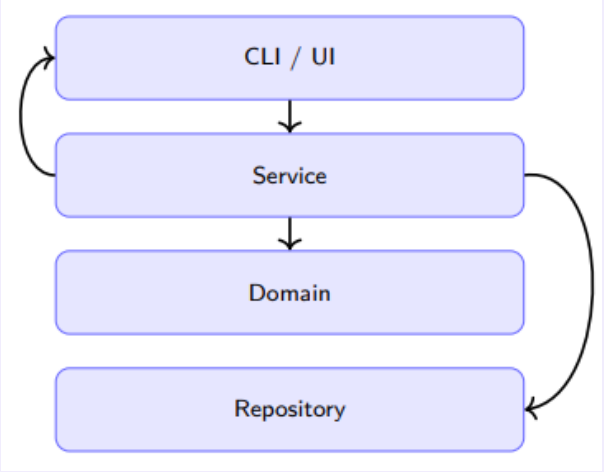
**Per UniManager possiamo riprendere il layering già introdotto:**

- CLI / UI: interazione con l'utente (comandi, menu).
- Service: implementazione dei casi d'uso (iscrizione, creazione appelli, ... ).
- Domain: entità e regole del dominio universitario.
- Repository: accesso ai dati su file o database.




### Iscrizione a un appello nei layer

- Scenario «Studente si iscrive a un appello»:
  1. CLI/UI chiede matricola e appello e invoca AppelloService.
  2. Service controlla i vincoli usando oggetti del Domain.
  3. Service chiede al Repository di aggiornare la persistenza.
  4. CLI/UI mostra il risultato (successo o errore).
- Il flusso attraversa i layer in modo ordinato, con responsabilità chiare



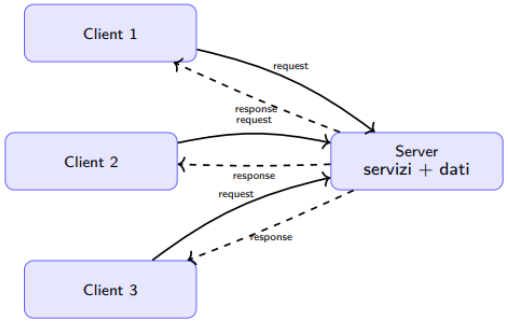

**Architettura client–server:** Lo stile client–server separa:

- il client, che richiede servizi,
- il server, che fornisce servizi.

Tipicamente:

- il server è centrale, mantiene dati e logica,
- i client sono distribuiti, interagiscono con il server via rete.

È il modello concettuale alla base di molte applicazioni web e sistemi distribuiti. La separazione client/server rende esplicito un confine importante: l'esperienza utente sta lato client, mentre regole e dati condivisi stanno lato server. Questo stile è spesso il primo passo quando un sistema locale deve diventare multiutente.




I client inviano richieste (request), il server risponde (response).

**Client–serverà- vantaggi:**

- *Centralizzazione:*
  - dati e logica critici restano concentrati sul server,
  - più facile aggiornare e controllare il sistema.
- *Multi-client:*
  - lo stesso server può servire molti client diversi (desktop, web, mobile).
- *Sicurezza:*
  - le regole di accesso ai dati sono gestite lato server.
- *Governance semplificata:*
  - aggiornamenti di logica e policy avvengono centralmente, senza distribuire patch su ogni client.
- *Riuso dei servizi:*
  - più canali (web, mobile, desktop) possono sfruttare lo stesso backend.

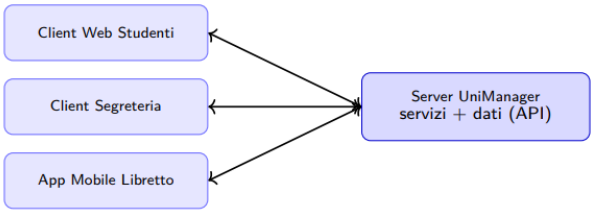
**Client–server- limiti e compromessi:**

- *Single point of failure:*
  - se il server è giù, tutti i client sono bloccati.
- *Scalabilità:*
  - se il numero di client cresce molto, il server può diventare un collo di bottiglia.
- *Complessità di rete:*
  - bisogna gestire latenza, errori di comunicazione, autenticazione, ...
- *Dipendenza dalla connettività:*
  - in assenza di rete, il client può perdere funzionalità critiche.
- *Scelte di stato/sessione:*
  - la gestione dello stato lato server o lato client impatta scalabilità e semplicità.

Al momento, UniManager è pensato come applicazione locale (es. CLI). Potremmo però immaginare un'evoluzione:

- un server UniManager che espone servizi (API),
- diversi client:
  - applicazione web per gli studenti,
  - applicazione interna per la segreteria,
  - app mobile per consultare il libretto.

In questo scenario, lo stile client–server diventa il modello concettuale di riferimento.

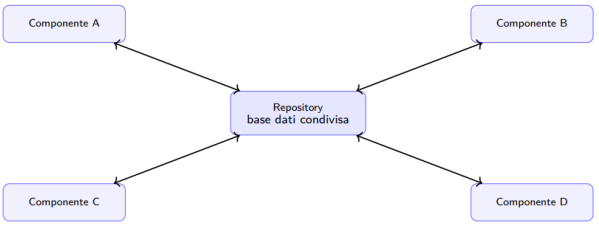



**Architettura repository:** Nello stile repository, più componenti condividono un'unica base dati comune. Il repository è il “cuore” dove risiedono le informazioni condivise. Intorno ad esso ci sono componenti che:

- elaborano dati,
- offrono funzionalità,
- reagiscono ai cambiamenti nel repository.

La priorità di questo stile è la coerenza informativa: più moduli diversi lavorano sullo stesso patrimonio dati. È adatto quando il valore principale del sistema sta nei dati condivisi più che nel comportamento di un singolo componente.

**Repository:** Nello stile repository, tutti i componenti ruotano attorno a un'unica base dati condivisa. Ogni componente legge e aggiorna il repository, ma rimane indipendente dagli altri.



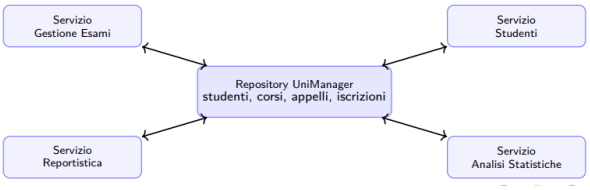
**Repository- vantaggi:**

- *Consistenza dei dati:*
  - una sola fonte di verità per le informazioni condivise.
- *Condivisione:*
  - sottosistemi diversi possono cooperare accedendo agli stessi dati.
- *Evoluzione dei componenti:*
  - è possibile sostituire o aggiornare i componenti attorno al repository mantenendo stabile la base dati (entro certi limiti).
- *Tracciabilità:*
  - avere un punto dati centrale facilita audit, report e analisi trasversali.
- *Integrazione:*
  - nuovi componenti possono essere aggiunti collegandosi al repository condiviso.

**Repository- limiti e compromessi:**

- *Punto critico di fallimento:*
  - se il repository si blocca o viene corrotto, l'intero sistema è in difficoltà.
- *Scalabilità e prestazioni:*
  - molte richieste concorrenti possono creare colli di bottiglia.
- *Dipendenza forte dal modello dati:*
  - cambiare lo schema del repository può impattare molti componenti.
- *Accoppiamento semantico:*
  - componenti nominalmente separati possono diventare dipendenti dagli stessi dettagli di schema.
- *Migrazioni delicate:*
  - evolvere il repository richiede coordinamento tra molti moduli e finestre di rilascio ben pianificate.

In UniManager abbiamo concettualmente una base dati universitaria: studenti, corsi, appelli, iscrizioni, voti, ... Per ora la implementiamo in modo semplice (file / strutture in memoria), ma concettualmente è un repository informativo condiviso. In una versione più evoluta: servizi diversi (es. gestione esami, reportistica, analisi statistiche) potrebbero accedere allo stesso repository.




**Architettura service-based:** Nello stile service-based, il sistema è costituito da servizi:

- ciascun servizio è un componente indipendente,
- espone una interfaccia ben definita,
- può essere sviluppato, distribuito e aggiornato separatamente.

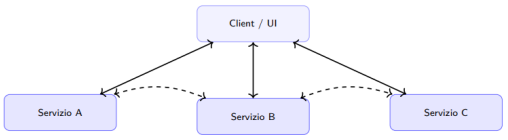
I servizi comunicano tra loro tramite chiamate di rete (es. API). Ogni servizio presidia una responsabilità funzionale chiara: questo riduce l'impatto dei cambiamenti e favorisce evoluzione incrementale. L'attenzione si sposta dai confini tra classi ai contratti tra servizi (endpoint, payload, versioni).

**Da client–server a service-based:** Lo stile service-based può essere visto come una evoluzione del client–server:

- invece di un unico server “monolitico”, abbiamo più servizi specializzati,
- ogni servizio può avere i propri dati e responsabilità.

**In UniManager:**

- servizio StudenteService per la gestione studenti,
- servizio AppelloService per appelli e voti,
- servizio ReportingService per report e statistiche.



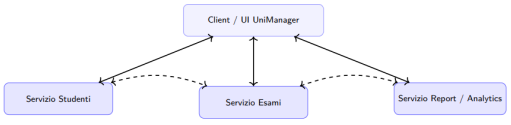

**Service-based:**
- Il client/UI parla con più servizi indipendenti, ciascuno con una responsabilità ben definita.
- I servizi possono anche collaborare tra loro tramite chiamate di rete (linee tratteggiate).

**Service-based- vantaggi:**

- *Scalabilità:*
  - ogni servizio può essere scalato indipendentemente (più istanze del solo servizio critico).
- *Autonomia dei team:*
  - team diversi possono sviluppare e rilasciare servizi separati.
- *Evoluzione tecnologica:*
  - servizi diversi possono usare tecnologie diverse, purché rispettino i contratti di interfaccia.
- *Isolamento dei guasti:*
  - un problema in un servizio può avere impatto limitato, se i fallback sono progettati bene.
- *Rilasci mirati:*
  - è possibile aggiornare un singolo servizio senza rilasciare l'intero sistema.

**Service-based- limiti e compromessi:**

- *Maggiore complessità complessiva:*
  - bisogna gestire comunicazione tra servizi, fallimenti parziali, versioni delle API.
- *Osservabilità e debugging più difficili:*
  - un flusso di richiesta può attraversare molti servizi diversi.
- *Coerenza dei dati:*
  - se ogni servizio ha il proprio storage, garantire consistenza diventa più complesso.
- *Transazioni distribuite:*
  - operazioni che coinvolgono più servizi richiedono sequenze coordinate e azioni di annullamento in caso di errore.
- *Costo operativo:*
  - servono strumenti maturi per monitoraggio, tracing, deployment e gestione API.




Per il corso manteniamo UniManager semplice e monolitico (layered). Ma possiamo usare il modello service-based per:
- discutere come dividere il sistema in aree funzionali indipendenti,
- ragionare su possibili evoluzioni future (es. servizi separati per studenti, esami, analytics),
- introdurre l'idea di contratti tra componenti (interfacce ben definite).

**Stili a confronto** Ogni stile risponde meglio a una domanda diversa:

- *Layered:* come separare responsabilità interne e tenere ordinate le dipendenze?
- *Client–server:* come distribuire il sistema tra client e server condiviso?
- *Repository:* come far cooperare componenti diversi sugli stessi dati?
- *Service-based:* come far evolvere parti del sistema in modo indipendente?

Sintesi operativa:

- priorità a manutenibilità interna e chiarezza: *Layered*;
- priorità a multiutente via rete: *Client–server*;
- priorità a consistenza e patrimonio dati comune: *Repository*;
- priorità a scalabilità organizzativa e rilasci indipendenti: *Service-based*.

Nei sistemi reali la scelta è spesso ibrida, ma i confini devono restare espliciti.

Ogni stile ha impatti diversi su:

- *prestazioni*,
- *scalabilità*,
- *affidabilità*,
- *manutenibilità*,
- *evolutività*.

**Example**

- layered aiuta molto la manutenibilità e la testabilità;
- client–server e service-based introducono nuove sfide di prestazioni e affidabilità di rete;
- repository facilita la consistenza, ma crea possibili colli di bottiglia.

**Come scegliere uno stile (o più di uno):** Nella pratica, i sistemi reali sono spesso una combinazione di stili:

- un'applicazione web può essere client–server e internamente layered,
- più servizi possono condividere un repository o avere ognuno il proprio.

Alcune domande utili:

- Quanto è importante la manutenibilità interna? (layered)
- Il sistema è distribuito o locale? (client–server / service-based)
- I dati condivisi sono il vero cuore del sistema? (repository)
- Ci sono team diversi che devono lavorare in modo indipendente? (service-based)

Sintesi per UniManager in questa fase:

- base layered con repository centrale dei dati,
- client–server e service-based come possibili evoluzioni successive.

## Principi SOLID

Finora ci siamo concentrati su:

- principi architetturali generali (modularità, separazione delle responsabilità, layering),
- stili architetturali (layered, client–server, repository, service-based).

Oggi scendiamo di livello: come progettare le classi e le loro dipendenze, in modo coerente con l'architettura scelta. L'obiettivo è quello di collegare le decisioni locali (sulle classi) alla struttura globale del sistema

**Perché servono principi di design?** Il codice non deve solo funzionare oggi: deve poter evolvere senza degradarsi. Quando il progetto cresce, emergono problemi ricorrenti:

- classi con troppe responsabilità e confini poco chiari;
- dipendenze rigide che propagano gli effetti di ogni modifica;
- regressioni frequenti e test costosi da mantenere.

In queste condizioni, il costo principale non è scrivere nuova logica, ma capire e cambiare quella esistente. I principi di design (come *SOLID*) servono come criteri operativi per ridurre accoppiamento, aumentare coesione e migliorare testabilità.

**Che cos'è SOLID?** SOLID è un acronimo che indica cinque principi:

1. **Single Responsibility Principle (SRP):** principio di responsabilità singola
2. **Open/Closed Principle (OCP):** principio aperto/chiuso
3. **Liskov Substitution Principle (LSP):** principio di sostituzione di Liskov
4. **Interface Segregation Principle (ISP):** principio di segregazione delle interfacce
5. **Dependency Inversion Principle (DIP):** principio di inversione delle dipendenze

Non sono leggi assolute, ma linee guida per prendere decisioni migliori.

**SRP — Single Responsibility Principle:** Una classe dovrebbe avere un solo motivo per cambiare. Per “motivo per cambiare” si intende una famiglia omogenea di modifiche (stesso tipo di responsabilità). Quando una classe mescola responsabilità diverse:

- aumenta l'accoppiamento interno;
- crescono effetti collaterali e regressioni;
- test e manutenzione diventano più costosi.

**Esempio in UniManager:**

- una classe che contiene insieme regole d'esame, persistenza e messaggi UI viola SRP;
- queste responsabilità vanno separate in componenti distinti

In UniManager conviene distinguere almeno queste responsabilità:

- business: regole del dominio (vincoli su appelli, prerequisiti, esiti);
- persistenza: lettura/scrittura su file o database;
- presentazione: messaggi verso CLI/UI e formati di output;
- cross-cutting: log, audit, monitoraggio.

Criterio pratico per rilevare una violazione SRP: “quante ragioni indipendenti ho per modificare questa classe?”. Se la risposta è più di una, serve separare. L'obiettivo è quello di avere classi piccole ma coese, con confini chiari e test più mirati.

```java
class GestoreEsami {

    public void creaAppello(Corso corso, LocalDate data) {
        // (1) Logica di business:
        // controlla sovrapposizioni, scadenze, ecc.
        // ...

        // (2) Persistenza:
        // salva l'appello su file
        salvaSuFile(corso, data);

        // (3) Presentazione:
        // mostra un messaggio a video
        stampaReportCreazione(corso, data);
    }

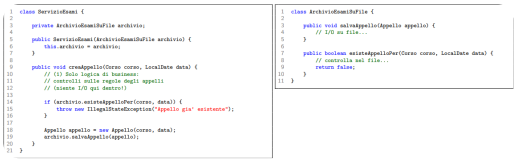
    private void salvaSuFile(Corso corso, LocalDate data) {
        // codice di I/O su file...
    }

    private void stampaReportCreazione(Corso corso, LocalDate data) {
        // System.out.println(...);
    }
}
```

GestoreEsami mescola:

- business (regole sugli appelli),
- persistenza (salvataggio su file),
- presentazione (stampa su console).

Cambia il formato di salvataggio? ⇒ modifico GestoreEsami. Cambia il modo di comunicare con l'utente? ⇒ modifico ancora GestoreEsami. Risultato: classe fragile, difficile da testare e da capire.



```java
class ServizioEsami {

    private ArchivioEsamiSuFile archivio;

    public ServizioEsami(ArchivioEsamiSuFile archivio) {
        this.archivio = archivio;
    }

    public void creaAppello(Corso corso, LocalDate data) {
        // (1) Solo logica di business:
        // controlli sulle regole degli appelli
        // (niente I/O qui dentro!)

        if (archivio.esisteAppelloPer(corso, data)) {
            throw new IllegalStateException("Appello già esistente");
        }

        Appello appello = new Appello(corso, data);
        archivio.salvaAppello(appello);
    }
}
```

```java
class ArchivioEsamiSuFile {

    public void salvaAppello(Appello appello) {
        // I/O su file...
    }

    public boolean esisteAppelloPer(Corso corso, LocalDate data) {
        // controlla sul file...
        return false;
    }
}
```

**Prima:**

- UI → GestoreEsami (regole + salvataggio su file + stampa).
- Un'unica classe con responsabilità miste (business, persistenza, presentazione).

**Dopo:**

- UI → ServizioEsami (solo regole di business),
- ServizioEsami delega a ArchivioEsamiSuFile la persistenza,
- la UI (CLI) si occupa dei messaggi verso l'utente (stampe, errori, conferme).

**Effetti:**

- ogni classe ha un motivo principale per cambiare,
- il comportamento si capisce per “blocchi” separati,
- il design è pronto per introdurre OCP e DIP.

**OCP — Open/Closed Principle:** Il software dovrebbe essere aperto all'estensione, ma chiuso alla modifica. Interpretazione operativa:

- nuovo comportamento = aggiungere nuove classi/composizioni;
- codice stabile = evitare modifiche continue agli stessi metodi centrali.

OCP non significa “mai modificare nulla”, ma ridurre le modifiche invasive nelle parti già consolidate.

Esempio in UniManager: aggiungere un nuovo criterio di ordinamento senza riscrivere ServizioEsami.

OCP nasce a causa di un problema ricorrente: il requisito resta lo stesso, ma cambiano le varianti (nuovi casi, nuove regole, nuove modalità). Se ogni variante richiede di riaprire lo stesso componente centrale:

- aumenta il rischio di regressioni;
- i test da rieseguire crescono a ogni modifica;
- quel componente diventa un collo di bottiglia del team.

OCP nasce per gestire la variabilità in modo più stabile:

- la variabilità va isolata in estensioni dedicate;
- il nucleo resta stabile e riusabile.

```java
class ServizioEsami {

    public List<Appello> ordina(List<Appello> appelli, String criterio) {

        if (criterio.equals("data")) {
            Collections.sort(appelli, new Comparator<Appello>() {
                    public int compare(Appello a, Appello b) {
                        return a.getData().compareTo(b.getData());
                    }
                });

        } else if (criterio.equals("corso")) {
            Collections.sort(appelli, new Comparator<Appello>() {
                    public int compare(Appello a, Appello b) {
                        return a.getCorso().getNome().compareTo(b.getCorso().getNome()
        );
                    }
                });

        } else if (criterio.equals("iscritti")) {
            Collections.sort(appelli,
                new Comparator<Appello>() {
                    public int compare(Appello a, Appello b) {
                        return a.getNumIscritti() - b.getNumIscritti();
                    }
                });
        }

        return appelli;
    }
}
```

Metodo "monolitico" con tre comportamenti diversi. Ogni nuovo criterio richiede di:

- modificare ServizioEsami,
- aggiungere un nuovo blocco if/else.

Questo viola OCP:

- il codice non è chiuso alla modifica,
- cresce nel tempo e diventa fragile.

La logica di ordinamento è tutta concentrata qui, difficile da testare in isolamento.

Perché questa versione viola OCP? Ogni nuovo criterio di ordinamento richiede:

- di modificare ServizioEsami,
- di aggiungere un altro blocco if/else,
- di rischiare di rompere casi già funzionanti.

Criterio diagnostico: se una nuova variante obbliga a riaprire sempre lo stesso metodo, il design non è OCP. Il metodo cresce continuamente: code smell di variabilità concentrata.

```java
// Interfaccia comune
interface OrdinatoreAppelli {
    void ordina(List<Appello> appelli);
}

class OrdinaPerData implements OrdinatoreAppelli {
    public void ordina(List<Appello> appelli) {
        Collections.sort(appelli, new Comparator<Appello>() {
            public int compare(Appello a, Appello b) {
                return a.getData().compareTo(b.getData());
            }
        });
    }
}

class OrdinaPerCorso implements OrdinatoreAppelli {
    public void ordina(List<Appello> appelli) {
        Collections.sort(appelli, new Comparator<Appello>() {
            public int compare(Appello a, Appello b) {
                return a.getCorso().getNome().compareTo(b.getCorso().getNome
        ());
            }
        });
    }
}
```

```java
class ServizioEsami {
    private OrdinatoreAppelli ordinatore;

    // Iniezione del criterio
    public ServizioEsami(OrdinatoreAppelli ord) {
        this.ordinatore = ord;
    }

    public List<Appello> ordina(List<Appello> appelli) {
        ordinatore.ordina(appelli);
        return appelli;
    }
}

// Scelta del criterio (da fuori)
OrdinatoreAppelli o1 = new OrdinaPerData();
ServizioEsami s1 = new ServizioEsami(o1);

// Oppure
OrdinatoreAppelli o2 = new OrdinaPerCorso();
ServizioEsami s2 = new ServizioEsami(o2);
```

Separiamo il criterio di ordinamento dal servizio. OrdinatoreAppelli descrive l'operazione in astratto. Ogni criterio (per data, per corso, per iscritti, . . . ) diventa:

- una classe separata,
- aggiungibile senza modificare codice esistente.

ServizioEsami non cambia più:

- riceve un ordinatore dall'esterno,
- lo usa senza sapere come funziona.

Questo rispetta pienamente OCP:

- aperto a nuovi criteri: basta aggiungere una classe;
- chiuso alle modifiche: nessun if/else da aggiornare.

Condizioni operative: La soluzione resta OCP solo se il punto di estensione è stabile:

- interfaccia chiara (OrdinatoreAppelli),
- servizio dipendente dall'astrazione, non dai criteri concreti.

Per introdurre una nuova variante:

- si aggiunge una classe (OrdinaPer...),
- si seleziona il criterio in composizione/iniezione,
- ServizioEsami resta invariato.

Checklist rapida OCP:

- una nuova variante richiede solo codice nuovo?
- i test esistenti restano validi senza riscritture massive?
- il comportamento variabile è confinato in componenti dedicati?

**LSP — Liskov Substitution Principle:** Le sottoclassi dovrebbero poter sostituire le superclassi senza cambiare il comportamento atteso del programma. In pratica:

- il codice client deve poter usare la sottoclasse al posto della superclasse senza accorgersene;
- stessa chiamata, stesso significato, stesso tipo di esito.

Segnali tipici di violazione:

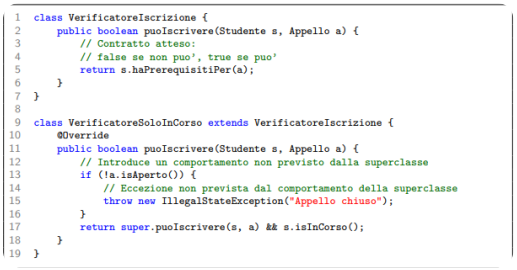
- la sottoclasse rifiuta casi che la superclasse accetta;
- la sottoclasse introduce errori/eccezioni inattese;
- il client deve aggiungere controlli sul tipo concreto per evitare bug.

In molti sistemi OO usiamo ereditarietà e polimorfismo per riuso. Senza LSP, il polimorfismo diventa fragile:

- il chiamante si aspetta un comportamento della superclasse,
- alcune sottoclassi rompono quelle attese.

Effetti pratici:

- if speciali per distinguere sottoclassi,
- bug difficili da vedere in revisione,
- test incompleti perché dipendono dal tipo concreto.



```
class VerificatoreIscrizione {
    public boolean puoIscrivere(Studente s, Appello a) {
        // Contratto atteso:
        // false se non puo', true se puo'
        return s.haPrerequisitiPer(a);
    }
}

class VerificatoreSoloInCorso extends VerificatoreIscrizione {
    @Override
    public boolean puoIscrivere(Studente s, Appello a) {
        // Introduce un comportamento non previsto dalla superclasse
        if (!a.isAperto()) {
            // Eccezione non prevista dal comportamento della superclasse
            throw new IllegalStateException("Appello chiuso");
        }
        return super.puoIscrivere(s, a) && s.isInCorso();
    }
}
```

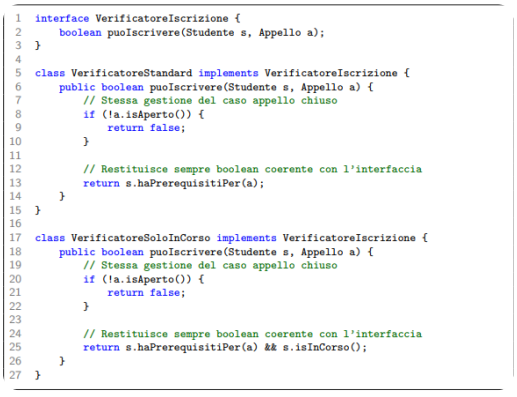
La superclasse promette un contratto semplice:
- input valido,
- output booleano.

La sottoclasse introduce un comportamento incompatibile: lancia eccezioni inattese. Un chiamante polimorfico può rompersi pur rispettando il contratto della base.

Perché questa versione viola LSP? La sottoclasse VerificatoreSoloInCorso restringe il contratto:

- aggiunge una precondizione implicita (appello aperto),
- cambia la modalità di errore (eccezione invece di false).

Chi usa il tipo base non è più protetto: serve conoscere il tipo concreto per evitare crash. Se serve un comportamento diverso, è meglio modellare una strategia, non una sottoclasse incompatibile.



```
interface VerificatoreIscrizione {
    boolean puoIscrivere(Studente s, Appello a);
}

class VerificatoreStandard implements VerificatoreIscrizione {
    public boolean puoIscrivere(Studente s, Appello a) {
        // Stessa gestione del caso appello chiuso
        if (!a.isAperto()) {
            return false;
        }

        // Restituisce sempre boolean coerente con l'interfaccia
        return s.haPrerequisitiPer(a);
    }
}

class VerificatoreSoloInCorso implements VerificatoreIscrizione {
    public boolean puoIscrivere(Studente s, Appello a) {
        // Stessa gestione del caso appello chiuso
        if (!a.isAperto()) {
            return false;
        }

        // Restituisce sempre boolean coerente con l'interfaccia
        return s.haPrerequisitiPer(a) && s.isInCorso();
    }
}
```

Tutte le implementazioni rispettano lo stesso contratto:
- nessuna eccezione inattesa,
- risultato sempre booleano.

Le varianti cambiano la regola, non il tipo di comportamento osservabile. Il chiamante resta stabile e sostituibile.

La soluzione resta LSP solo se il contratto è uniforme:

- stessi input ammessi (niente precondizioni più forti nelle sottoclassi),
- stesso tipo di output/errore osservabile dal client.

Le varianti devono cambiare la regola, non il significato del comportamento atteso. Checklist rapida LSP:

- il client funziona uguale con qualunque implementazione?
- servono if sul tipo concreto per evitare crash?
- una sottoclasse introduce eccezioni o vincoli non previsti?

**ISP — Interface Segregation Principle:** È meglio avere più interfacce piccole e specifiche che un'unica interfaccia “gigante” che fa di tutto. In pratica:

- ogni client dovrebbe vedere solo le operazioni del proprio ruolo;
- le funzionalità non pertinenti vanno in interfacce separate.

Violazioni tipiche:

- interfacce con molti metodi opzionali,
- classi client che devono implementare metodi “vuoti” o fittizi.

Problema ricorrente:

- un'unica interfaccia viene usata da ruoli diversi (studente, docente, segreteria),
- col tempo accumula metodi eterogenei.

Effetti pratici:

- i client dipendono da operazioni che non usano;
- compaiono metodi vuoti o UnsupportedOperationException;
- una modifica locale impatta molti componenti non coinvolti.

ISP nasce per separare le interfacce per ruolo/scenario e ridurre l'accoppiamento inutile.

```java
interface PortaleDidattico {
    void iscriviAppello(String matricola, String idAppello);
    void consultaLibretto(String matricola);
    void registraVoto(String idAppello, String matricola, int voto);
    void creaAppello(String idCorso, LocalDate data);
    void generaReportCorso(String idCorso);
}

class ClientStudente implements PortaleDidattico {
    public void iscriviAppello(String m, String a) {
        /* ok */
    }

    public void consultaLibretto(String m) {
        /* ok */
    }

    public void registraVoto(String a, String m, int v) {
        throw new UnsupportedOperationException();
    }

    public void creaAppello(String c, LocalDate d) {
        throw new UnsupportedOperationException();
    }

    public void generaReportCorso(String c) {
        throw new UnsupportedOperationException();
    }
}
```

ClientStudente dipende da metodi solo docenti/segreteria. Implementa metodi non pertinenti con eccezioni fittizie. Ogni cambiamento dell'interfaccia impatta tutti i client.

La violazione nasce da un problema di modellazione: PortaleDidattico mescola responsabilità di ruoli diversi in un unico contratto. Effetto sui client:

- ogni client dipende anche da metodi che non dovrebbe conoscere;
- compaiono implementazioni fittizie (UnsupportedOperationException).

Conseguenza pratica: modifiche pensate per un ruolo impattano anche client di altri ruoli. Correzione: separare l'interfaccia per responsabilità funzionali (Studente, Docente, Segreteria).

```java
interface FunzioniStudente {
    void iscriviAppello(String matricola, String idAppello);
    void consultaLibretto(String matricola);
}

interface FunzioniDocente {
    void registraVoto(String idAppello, String matricola, int voto);
    void creaAppello(String idCorso, LocalDate data);
}

interface FunzioniSegreteria {
    void generaReportCorso(String idCorso);
}

class ClientStudente implements FunzioniStudente {
    public void iscriviAppello(String m, String a) {/* ... */}

    public void consultaLibretto(String m) {/* ... */}
}

class ClientDocente implements FunzioniDocente {
    public void registraVoto(String idAppello, String m, int voto) {/* ... */
        }

    public void creaAppello(String idCorso, LocalDate data) {/* ... */}
}

class ClientSegreteria implements FunzioniSegreteria {
    public void generaReportCorso(String idCorso) {/* ... */}
}
```

Ogni client dipende solo da ciò che usa davvero. I contratti sono più piccoli e chiari. Le evoluzioni su un ruolo non forzano modifiche sugli altri.

La soluzione resta ISP se le interfacce sono coerenti con i ruoli:

- interfacce distinte per Studente, Docente, Segreteria;
- ogni client implementa solo metodi pertinenti.

Quando aggiungi una funzionalità:

- modifichi solo l'interfaccia del ruolo coinvolto;
- eviti impatti trasversali sui client non interessati.

Checklist rapida ISP:

- compaiono metodi vuoti o UnsupportedOperationException?
- un client dipende da operazioni che non usa?
- una modifica locale forza cambi in client di altri ruoli

**DIP — Dependency Inversion Principle:** I moduli di alto livello e quelli di basso livello dovrebbero dipendere da astrazioni; i dettagli concreti dovrebbero dipendere da tali astrazioni. In pratica:

- alto livello = logica applicativa/business, più astratta (es. ServizioEsami);
- basso livello = dettagli tecnici, più concreti (es. ArchivioEsamiSuFile);
- entrambi si collegano tramite un'astrazione comune (es. ArchivioEsami).

Effetto: il business resta stabile anche se cambiano tecnologie e componenti infrastrutturali.

Problema ricorrente:

- i servizi applicativi creano direttamente classi infrastrutturali (new Archivio..., new ClientHttp...);
- quindi il livello alto dipende direttamente dal livello basso.

Effetti pratici:

- cambiare tecnologia richiede modifiche nel business;
- i test diventano lenti e fragili perché dipendono da risorse reali;
- cresce l'accoppiamento tra livelli che dovrebbero restare separati.

DIP nasce per invertire questa dipendenza:

- il business dipende da interfacce stabili,
- i dettagli concreti si agganciano a quelle interfacce.

Mappa dei ruoli (in UniManager):

- alto livello: ServizioEsami (regole applicative);
- basso livello: ArchivioEsamiSuFile (I/O su file);
- astrazione: ArchivioEsami.

Regola DIP: ServizioEsami deve dipendere da ArchivioEsami, non da ArchivioEsamiSuFile. Se dipende dal dettaglio concreto: cambiare persistenza (file, DB, memoria) richiede modifiche nel servizio. DIP mantiene pulito il confine tra business e infrastruttura.

```java
class ArchivioEsamiSuFile {
    private Path path;

    public ArchivioEsamiSuFile(Path path) {
        this.path = path;
    }

    public void salvaAppello(Appello appello) {
        // I/O su file...
    }
}

// Servizio che dipende da una classe concreta
class ServizioEsami {

    private ArchivioEsamiSuFile archivio;

    public ServizioEsami() {
        // Dipendenza creata direttamente qui
        this.archivio = new ArchivioEsamiSuFile(Paths.get("esami.txt"));
    }

    public void creaAppello(Corso corso, LocalDate data) {
        // Logica di business (semplificata)
        Appello appello = new Appello(corso, data);

        // Persistenza: usa sempre il file esami.txt
        archivio.salvaAppello(appello);
    }
}
```

ServizioEsami dipende da ArchivioEsamiSuFile, una classe concreta. La scelta del file ("esami.txt") è codificata dentro il servizio. Problemi:

- per usare un database devo modificare ServizioEsami;
- per fare test devo scrivere davvero sul file;
- la logica di business è legata a un dettaglio di infrastruttura.

Qui DIP è violato: un modulo di alto livello dipende da un modulo di basso livello.

Nel nostro esempio:

- alto livello: ServizioEsami;
- basso livello: ArchivioEsamiSuFile.

Violazione: l'alto livello dipende direttamente dal basso livello concreto. Il servizio crea direttamente la dipendenza: new ArchivioEsamiSuFile(...) dentro il costruttore. Effetto: ogni variazione infrastrutturale entra nel codice business. Criterio diagnostico: se per cambiare tecnologia devo modificare un servizio di dominio/applicazione, DIP non è rispettato.

```java
// Astrazione nel dominio
interface ArchivioEsami {
    void salvaAppello(Appello appello);
    boolean esisteAppelloPer(Corso corso, LocalDate data);
}

// Implementazione concreta su file
class ArchivioEsamiSuFile implements ArchivioEsami {
    public ArchivioEsamiSuFile(Path path) {/* salva path */}

    @Override
    public void salvaAppello(Appello appello) {
        // I/O su file...
    }

    @Override
    public boolean esisteAppelloPer(Corso corso, LocalDate data) {
        // controlla nel file...
        return false;
    }
}

// Ora il servizio dipende dall'astrazione
class ServizioEsami {
    private ArchivioEsami archivio;
    public ServizioEsami(ArchivioEsami archivio) {
        this.archivio = archivio;
    }
    public void creaAppello(Corso corso, LocalDate data) {
        Appello appello = new Appello(corso, data);
        archivio.salvaAppello(appello);
    }
}
```

```java
// Composizione: scelta concreta fuori dal servizio
Path path = Paths.get("esami.txt");
ArchivioEsami arch = new ArchivioEsamiSuFile(path);
ServizioEsami srv = new ServizioEsami(arch);

```

Spostiamo l'interfaccia ArchivioEsami nel dominio (modulo più stabile). ServizioEsami dipende solo da ArchivioEsami, non da classi concrete. Le implementazioni concrete (SuFile, SuDb, InMemoria, . . . ) dipendono dall'astrazione, non il contrario. Abbiamo invertito la dipendenza:

- il modulo di alto livello (service) non conosce più i dettagli di persistenza;
- i dettagli dipendono dall'astrazione comune.

La soluzione resta DIP se:

- il servizio dipende da ArchivioEsami (astrazione), non da ArchivioEsamiSuFile;
- la scelta dell'implementazione avviene all'esterno (composizione/iniezione).

Impatto diretto sui test:

- si può usare un'implementazione finta:
  - ArchivioEsamiInMemoria,
  - oppure un mock creato con un framework di test.
- i test non richiedono file reali, database o configurazioni esterne.

Checklist rapida DIP:

- il servizio conosce classi concrete infrastrutturali?
- la dipendenza concreta viene istanziata dentro il servizio?
- è possibile testare il servizio senza risorse esterne?

**I principi SOLID: come lavorano insieme:**

- **SRP:** mantiene coese le classi (una responsabilità principale per classe).
- **OCP:** consente nuove varianti senza modificare continuamente il nucleo stabile.
- **LSP:** rende affidabile la sostituzione tra superclasse e sottoclasse.
- **ISP:** evita interfacce troppo ampie e dipendenze inutili lato client.
- **DIP:** separa business e infrastruttura tramite astrazioni.

Insieme, questi principi supportano:

- manutenibilità,
- estendibilità,
- testabilità.

Applicare SOLID non significa:

- creare mille micro-classi inutili,
- astrarre tutto “per principio”.

Significa:

- riconoscere code smell (classi sovraccariche, dipendenze rigide),
- avere strumenti concettuali per migliorare il design,
- trovare un equilibrio tra semplicità e flessibilità.

**SOLID e refactoring:** Spesso non si progetta “perfettamente” al primo colpo: SOLID aiuta a migliorare in modo incrementale. Workflow tipico di refactoring guidato da SOLID:

- individuare code smell e punti di accoppiamento eccessivo;
- chiarire responsabilità (SRP) e interfacce (ISP);

- isolare variabilità (OCP) e dipendenze infrastrutturali (DIP);
- verificare sostituibilità corretta tra tipi correlati (LSP).

Obiettivo: migliorare il design senza riscrivere tutto da zero.

**Mini-checklist per UniManager:**

1. SRP:
   - ci sono classi che fanno sia business che I/O?
   - posso separare persistenza, log, presentazione?
2. OCP:
   - ci sono grandi if/switch su tipo o modalità?
   - posso spostare logiche variabili in classi dedicate, evitando di toccare sempre lo stesso metodo?
3. LSP:
   - una sottoclasse può sostituire la superclasse senza rompere il client?
   - compaiono controlli sul tipo concreto o eccezioni inattese?
4. ISP:
   - i client dipendono da metodi che non usano?
   - ci sono metodi vuoti o UnsupportedOperationException?
5. DIP:
   - i servizi dipendono da classi concrete di repository?
   - posso introdurre interfacce nel dominio e implementazioni nei layer esterni?

## Design patterns essenziali: strategy e factory method

Nella lezione precedente abbiamo visto tutti e cinque i principi SOLID. Questi principi ci dicono come dovrebbe essere un buon design:

- classi con ruoli chiari;
- codice estendibile senza modificare quello già scritto;
- dipendenze dirette da astrazioni e non da dettagli concreti.

Domande naturali:

- quali soluzioni concrete possiamo usare per mettere in pratica questi principi?
- come riconoscere schemi ricorrenti di buona progettazione?

In questa lezione vedremo due pattern che realizzano nella pratica questi principi:

- **Strategy** soprattutto a supporto di OCP,
- **Factory Method** molto legato a **DIP** (e a OCP sulla creazione).

**Design pattern:** I design pattern nascono da problemi che si ripetono:

- varianti di comportamento gestite con `if/else` crescenti;
- creazione di oggetti sparsa in molti punti del codice.

Un pattern fornisce:

- una struttura riusabile;
- un linguaggio condiviso nel team;
- una soluzione coerente con principi come OCP e DIP.

Criterio pratico:

- usare un pattern quando semplifica evoluzione e manutenzione;
- evitarlo quando aggiunge astrazione senza variabilità reale.

Oggi ci concentriamo su due pattern fondamentali e trasversali:

- *Strategy:* rendere intercambiabili i comportamenti (es. diversi criteri di ordinamento degli appelli in UniManager), in linea con OCP (nuovi comportamenti senza modificare il contesto).
- *Factory Method:* centralizzare e strutturare la creazione degli oggetti (es. creazione di diverse tipologie di ExamSession in UniManager), aiutando a rispettare DIP (dipendere da ExamSession, non dalle sue sottoclassi).

L'idea non è memorizzare definizioni, ma:

- riconoscere quando un problema è adatto a questi pattern;
- saperli usare per mantenere il codice pulito, estensibile e coerente con i principi SOLID visti nella lezione precedente.

**Strategy:** contesto tipico:

- una classe deve eseguire un certo comportamento;
- esistono varie alternative per implementarlo;
- vorremmo poter cambiare questo comportamento senza modificare la classe.

Strategy propone di:

- estrarre il comportamento in un'interfaccia separata;
- avere più strategie concrete intercambiabili;
- far sì che il contesto usi sempre l'interfaccia, non le implementazioni concrete.

Effetto:

- Strategy permette di iniettare comportamenti diversi senza toccare il codice del contesto.
- È uno strumento concreto per applicare OCP e ridurre dipendenze rigide verso implementazioni specifiche.

### grandi `if/else` per i comportamenti

Nell'esempio di UniManager vogliamo ordinare gli appelli:

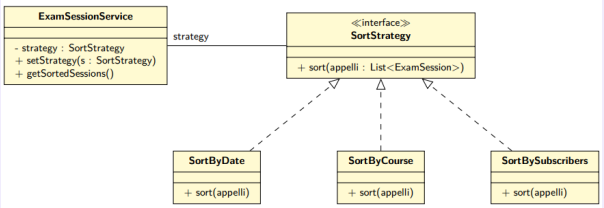
- per data;
- per corso;
- per numero di iscritti.

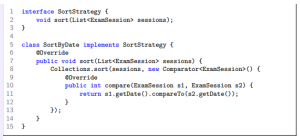
```
if (criterio == DATA) {
    sortByDate(appelli);
} else if (criterio == CORSO) {
    sortByCourse(appelli);
} else if (criterio == ISCRITTI) {
    sortBySubscribers(appelli);
} else {
    // ...
}
```

Problemi: ogni volta che vogliamo aggiungere un nuovo criterio dobbiamo modificare il metodo: il codice non è pensato per essere esteso facilmente. Il metodo, inoltre, cresce sempre di più ed è difficile da mantenere e testare.

Idea:

- definire un'interfaccia SortStrategy;
- avere implementazioni diverse per ogni criterio;
- lasciare che un ExamSessionService usi sempre una SortStrategy.





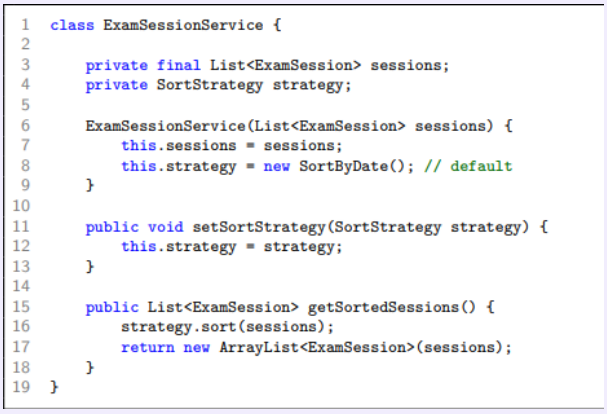
```
interface SortStrategy {
    void sort(List<ExamSession> sessions);
}

class SortByDate implements SortStrategy {
    @Override
    public void sort(List<ExamSession> sessions) {
        Collections.sort(sessions, new Comparator<ExamSession>() {
            @Override
            public int compare(ExamSession s1, ExamSession s2) {
                return s1.getDate().compareTo(s2.getDate());
            }
        });
    }
}
```



```
class SortByCourse implements SortStrategy {
    @Override
    public void sort(List<ExamSession> sessions) {
        Collections.sort(sessions, new Comparator<ExamSession>() {
            @Override
            public int compare(ExamSession s1, ExamSession s2) {
                return s1.getCourse().getName()
                    .compareTo(s2.getCourse().getName());
            }
        });
    }
}

class SortBySubscribers implements SortStrategy {
    @Override
    public void sort(List<ExamSession> sessions) {
        Collections.sort(sessions, new Comparator<ExamSession>() {
            @Override
            public int compare(ExamSession s1, ExamSession s2) {
                return Integer.compare(
                    s1.getNumSubscribers(),
                    s2.getNumSubscribers()
                );
            }
        });
    }
}
```



```
class ExamSessionService {

    private final List<ExamSession> sessions;
    private SortStrategy strategy;

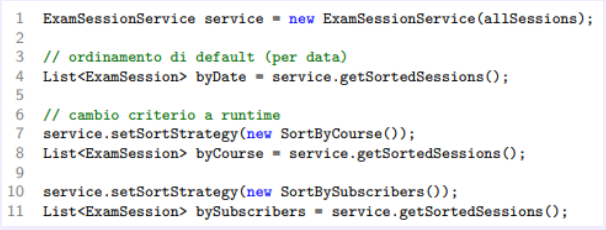
    ExamSessionService(List<ExamSession> sessions) {
        this.sessions = sessions;
        this.strategy = new SortByDate(); // default
    }

    public void setSortStrategy(SortStrategy strategy) {
        this.strategy = strategy;
    }

    public List<ExamSession> getSortedSessions() {
        strategy.sort(sessions);
        return new ArrayList<ExamSession>(sessions);
    }
}
```

Il contesto non conosce le implementazioni concrete, usa sempre e solo l'interfaccia SortStrategy.

Strategie intercambiabili a runtime:



```
ExamSessionService service = new ExamSessionService(allSessions);

// ordinamento di default (per data)
List<ExamSession> byDate = service.getSortedSessions();

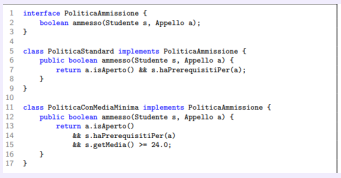
// cambio criterio a runtime
service.setSortStrategy(new SortByCourse());
List<ExamSession> byCourse = service.getSortedSessions();

service.setSortStrategy(new SortBySubscribers());
List<ExamSession> bySubscribers = service.getSortedSessions();
```

Il codice che usa ExamSessionService può:

- cambiare la strategia in base a una scelta dell'utente;
- selezionare la strategia da configurazione;
- sostituire le strategie senza modificare il servizio.

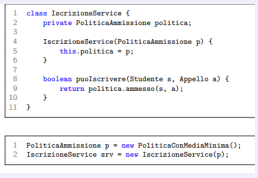
Secondo esempio Strategy: politiche di ammissione:



```
interface PoliticaAmmissione {
    boolean ammesso(Studente s, Appello a);
}

class PoliticaStandard implements PoliticaAmmissione {
    public boolean ammesso(Studente s, Appello a) {
        return a.isAperto() && s.haPrerequisitiPer(a);
    }
}

class PoliticaConMediaMinima implements PoliticaAmmissione {
    public boolean ammesso(Studente s, Appello a) {
        return a.isAperto()
            && s.haPrerequisitiPer(a)
            && s.getMedia() >= 24.0;
    }
}
```



```
class IscrizioneService {
    private PoliticaAmmissione politica;

    IscrizioneService(PoliticaAmmissione p) {
        this.politica = p;
    }

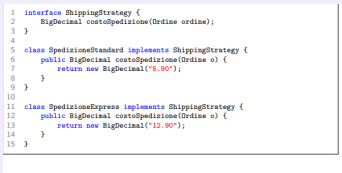
    boolean puoIscrivere(Studente s, Appello a) {
        return politica.ammesso(s, a);
    }
}
```



```
PoliticaAmmissione p = new PoliticaConMediaMinima();
IscrizioneService srv = new IscrizioneService(p);
```

- Il contesto cambia politica senza modificare il servizio.
- Nuova regola: nuova classe, non nuovo `if/else`.

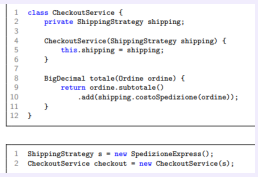
Terzo esempio Strategy: costo spedizione (e-commerce):



```
interface ShippingStrategy {
    BigDecimal costoSpedizione(Ordine ordine);
}

class SpedizioneStandard implements ShippingStrategy {
    public BigDecimal costoSpedizione(Ordine o) {
        return new BigDecimal("5.90");
    }
}

class SpedizioneExpress implements ShippingStrategy {
    public BigDecimal costoSpedizione(Ordine o) {
        return new BigDecimal("12.90");
    }
}
```



```
class CheckoutService {
    private ShippingStrategy shipping;

    CheckoutService(ShippingStrategy shipping) {
        this.shipping = shipping;
    }

    BigDecimal totale(Ordine ordine) {
        return ordine.subtotale()
            .add(shipping.costoSpedizione(ordine));
    }
}
```



```
ShippingStrategy s = new SpedizioneExpress();
CheckoutService checkout = new CheckoutService(s);
```

- Cambia la politica di spedizione, non il CheckoutService.
- Stessa struttura del pattern, dominio completamente diverso.

**Strategy: vantaggi principali (collegamento a SOLID):**

- *OCP (Open/Closed):*
  - per aggiungere un nuovo criterio basta creare una nuova SortStrategy;
  - il contesto non si modifica: si estende senza toccare codice esistente.
- *DIP (Dependency Inversion):*
  - il contesto dipende dall'interfaccia SortStrategy, non dalle classi concrete;
  - riduce dipendenze rigide e rende la sostituzione delle strategie più semplice.
- *SRP (Single Responsibility):*
  - il contesto non deve “conoscere” tutti i criteri di ordinamento;
  - ogni criterio è confinato in una classe dedicata con un compito chiaro.
- *Testabilità:*
  - strategie testabili in isolamento; nel contesto si possono usare strategie fittizie.

**Strategy: errori tipici (rischi su SOLID):**

- *Introdurre Strategy senza varianti reali:*
  - aggiunge classi senza beneficio: aumenta complessità senza migliorare OCP/DIP.
- *Strategie troppo grandi o “onnivore”:*
  - una Strategy dovrebbe incapsulare un comportamento specifico;
  - se dentro finisce “mezza applicazione”, si perde SRP.
- *Contesto che continua a fare if/else sulle strategie:*
  - es. controlli sul tipo concreto (instanceof) o switch sul criterio;
  - si indebolisce OCP e spesso anche DIP.
- *Dipendenze concrete nelle strategie:*
  - se ogni Strategy istanzia direttamente servizi o repository concreti, si rischia una violazione di DIP (dipendenze dai dettagli).

Con Strategy abbiamo visto come rendere flessibili i comportamenti di un oggetto:

- criteri di ordinamento intercambiabili;
- possibilità di cambiare logica senza toccare il codice del servizio.

Ma in un sistema reale esiste un'altra fonte comune di rigidità: la creazione degli oggetti. Spesso troviamo:

- molti new sparsi per il codice;
- istanze create in modi diversi e difficili da uniformare;
- dipendenze concrete “nascoste” dentro i costruttori.

Domanda successiva naturale: possiamo rendere flessibile anche il modo in cui gli oggetti vengono creati?

**Factory Method:** Definiamo:

- un prodotto astratto (es. ExamSession);
- una o più sottoclassi concrete (es. WrittenSession, OralSession);
- una classe Creator che espone un metodo di fabbrica (il Factory Method) per creare il prodotto.

Il codice che usa il Creator:

- lavora solo con il tipo astratto;
- non conosce i dettagli delle classi concrete.

Effetto:

- la decisione su quale sottoclasse istanziare è concentrata nel Factory Method;
- il resto del codice può dipendere solo dal tipo astratto: questo va nella direzione di DIP e OCP.

> **UniManager: creazione di ExamSession**
>
> Immaginiamo di avere varie tipologie di appello:
>
> - WrittenSession (scritto);
> - OralSession (orale);
> - MixedSession (scritto + orale, non mostrata nei diagrammi per brevità).
> - Senza Factory Method:
> - il codice farebbe new WrittenSession(...) o new OralSession(...) in molti punti diversi;
> - cambiare il modo in cui creiamo le sessioni richiederebbe modifiche in tante classi.

Con Factory Method:

- concentriamo la creazione in un creator (o pochi creator);
- il resto del sistema dipende solo da ExamSession.




Factory Method in Java: prodotto astratto:

```java
abstract class ExamSession {

    private final Course course;
    private final LocalDate date;

    protected ExamSession(Course course, LocalDate date) {
        this.course = course;
        this.date = date;
    }

    public Course getCourse() {
        return course;
    }

    public LocalDate getDate() {
        return date;
    }

    public abstract String getType();
}
```

```java
class WrittenSession extends ExamSession {

    public WrittenSession(Course course, LocalDate date) {
        super(course, date);
    }

    @Override
    public String getType() {
        return "written";
    }
}

class OralSession extends ExamSession {

    public OralSession(Course course, LocalDate date) {
        super(course, date);
    }

    @Override
    public String getType() {
        return "oral";
    }
}
```

- ExamSession è il tipo astratto comune usato dal client.
- WrittenSession/OralSession cambiano il dettaglio concreto ma rispettano lo stesso contratto.
- Il client lavora sul tipo astratto, non sulla sottoclasse concreta.

Factory Method in Java: Creator e ConcreteCreator

```java
abstract class ExamSessionCreator {

    protected abstract ExamSession createSession(Course course, LocalDate
        date);

    // logica comune che usa la sessione creata
    public ExamSession schedule(Course course, LocalDate date) {

        // Chiamata polimorfica: implementata nel concrete creator
        ExamSession session = createSession(course, date);

        // eventuale logica comune:
        // salvataggio, log, notifiche...
        return session;
    }
}
```

```java
class WrittenSessionCreator extends ExamSessionCreator {

    @Override
    protected ExamSession createSession(Course course, LocalDate date) {

        return new WrittenSession(course, date);
    }
}

class OralSessionCreator extends ExamSessionCreator {

    @Override
    protected ExamSession createSession(Course course, LocalDate date) {

        return new OralSession(course, date);
    }
}
```

- ExamSessionCreator definisce il flusso comune e delega il punto variabile a createSession().
- I concrete creator (WrittenSessionCreator, OralSessionCreator) decidono quale prodotto concreto istanziare.
- La creazione resta estendibile: nuovo tipo di appello = nuovo creator, senza modificare il client.

Uso del Factory method

```java
class ExamManagementService {

    private final ExamSessionCreator creator;

    ExamManagementService(ExamSessionCreator creator) {
        this.creator = creator;
    }

    public ExamSession createExamSession(Course course, LocalDate date) {

        // il servizio non conosce
        // WrittenSession / OralSession
        return creator.schedule(course, date);
    }
}
```

```java
// configurazione dell'applicazione
ExamSessionCreator creator = new WrittenSessionCreator();

ExamManagementService service = new ExamManagementService(creator);

// in futuro:
// creator = new OralSessionCreator();
```

- Per cambiare il tipo di sessione basta fornire un ExamSessionCreator diverso.
- Nessuna modifica al codice di ExamManagementService.

Secondo esempio Factory Method: nuova tipologia di appello

```java
class MixedSession extends ExamSession {
    public MixedSession(Course c, LocalDate d) {
        super(c, d);
    }
    @Override
    public String getType() {
        return "mixed";
    }
}

class MixedSessionCreator extends ExamSessionCreator {
    @Override
    protected ExamSession createSession(Course c, LocalDate d) {
        return new MixedSession(c, d);
    }
}
```

```java
ExamSessionCreator creator = new MixedSessionCreator();
ExamManagementService service =
    new ExamManagementService(creator);

ExamSession s = service.createExamSession(corso, data);
```

- Estensione concreta senza toccare ExamManagementService.
- La variabilità resta concentrata nel Creator.

Terzo esempio Factory Method: parser documenti

```java
abstract class Documento {
    abstract String tipo();
}

class DocumentoCsv extends Documento {
    DocumentoCsv(String raw) {/* parse CSV */}
    String tipo() { return "csv"; }
}

class DocumentoJson extends Documento {
    DocumentoJson(String raw) {/* parse JSON */}
    String tipo() { return "json"; }
}
```

```java
abstract class DocumentParser {
    protected abstract Documento createDocumento(String raw);

    public Documento parse(String raw) {
        Documento d = createDocumento(raw);
        // validazioni e normalizzazione comuni
        return d;
    }
}

class CsvParser extends DocumentParser {
    protected Documento createDocumento(String raw) {
        return new DocumentoCsv(raw);
    }
}

class JsonParser extends DocumentParser {
    protected Documento createDocumento(String raw) {
        return new DocumentoJson(raw);
    }
}

DocumentParser parser = new JsonParser();
Documento doc = parser.parse(input);
String tipo = doc.tipo();
```

- Prodotto: Documento; concreti: DocumentoCsv/DocumentoJson.
- Flusso comune in parse(), varia solo la creazione del prodotto concreto.

- Nuovo formato (XmlParser) senza toccare il client.

**Factory Method: vantaggi principali (collegamento a SOLID):**

- *DIP (Dependency Inversion):*
  - il client lavora con ExamSession e ExamSessionCreator, non con WrittenSession/OralSession;
  - la scelta della classe concreta è confinata nel Factory Method.
- *OCP (Open/Closed):*
  - per aggiungere un nuovo tipo di sessione si estende con nuove sottoclassi/creator;
  - il codice che usa il prodotto non va modificato.
- *SRP (Single Responsibility):*
  - la creazione non è sparsa nel codice: è centralizzata nel Creator;
  - il servizio si concentra sulla logica applicativa, non sui dettagli costruttivi.
- *Coerenza del processo di creazione:*
  - eventuali passi comuni (validazioni, log, notifiche) stanno in un punto solo.

**Factory Method: errori tipici (rischi su SOLID):**

- *Creator che diventa un “God object”:*
  - se nel Creator finiscono troppe responsabilità oltre alla creazione, si rischia di violare SRP.
- *Il client continua a conoscere le classi concrete:*
  - se compaiono comunque new WrittenSession(...) in giro, si perde il beneficio su DIP.
- *Esplosione di sottoclassi senza bisogno reale:*
  - troppi Creator “micro” possono aumentare complessità senza vero guadagno;
  - meglio introdurlo quando la creazione varia davvero.

**Strategy vs Factory Method:**

- *Strategy:*
  - incapsula un comportamento variabile;
  - il contesto delega l'esecuzione di un'operazione a una Strategy;
  - esempio UniManager: criterio di ordinamento degli appelli.
- *Factory Method:*
  - incapsula la creazione di oggetti (prodotti);
  - il client delega la scelta della classe concreta al Factory Method;
  - esempio UniManager: creazione di diverse tipologie di appello (ExamSession).
- *In comune:*
  - favoriscono OCP: estendere (nuove strategie / nuovi prodotti) senza modificare i client;
  - favoriscono DIP: dipendenza da astrazioni (SortStrategy, ExamSession, Creator);
  - aiutano SRP: separano responsabilità (comportamenti / creazione) dal resto della logica.

## Design Patterns essenziali: Adapter e Observer

Abbiamo visto:

- Strategy: rendere intercambiabili i comportamenti;
- Factory Method: centralizzare e strutturare la creazione degli oggetti.

Questi pattern agivano soprattutto:

- “dentro” al sistema;
- su logiche di dominio e creazione.

Ora affrontiamo due problemi diversi ma altrettanto comuni:

- come parlare con il mondo esterno senza contaminare il dominio interno (Adapter);
- come reagire a eventi interni senza creare dipendenze rigide tra classi (Observer).

Obiettivo: riconoscere quando questi pattern emergono “da soli” mentre il sistema cresce e si collega ad altri componenti.

**Adapter:** Il nostro sistema è costruito attorno a interfacce interne pulite. Le sorgenti esterne (CSV, JSON, API) espongono interfacce diverse. Effetti tipici:

- formati incompatibili;
- strutture dati grezze;
- gestione I/O o eccezioni non allineata al nostro dominio.

Obiettivo: integrare queste sorgenti senza “sporcare” il codice del dominio.

> **Esempi dal caso UniManager:**
>
> Libreria LegacyCsvLibrary:
>
> - restituisce solo List;
> - formati di data non uniformi;
> - API con metodi statici e dettagli I/O esposti.
>
> Libreria o servizio JSON:
>
> - restituisce strutture grezze (Map);
> - formati temporali diversi (ISO, timestamp, timezone);
> - gestione di errori di rete e codici HTTP.
>
> Queste differenze non devono “trapelare” dentro il dominio.

**Adapter:** Definiamo una interfaccia Target interna (es. ExamSessionImporter). Per ogni sorgente (CSV, JSON, API,…) realizziamo un Adapter dedicato. Ogni Adapter:

- implementa l'interfaccia Target;
- incapsula la libreria esterna (‘Adaptee');
- converte i dati grezzi in oggetti di dominio.

Il dominio usa sempre il Target e resta indipendente dalla tecnologia esterna.

**UniManager: CSV e JSON come sorgenti esterne:** UniManager vuole ottenere sempre una List. Le fonti possono essere diverse:

- file CSV letti da LegacyCsvLibrary;
- file JSON o API con LegacyJsonLibrary.

Problema: senza Adapter, il servizio dovrebbe conoscere formati, parsing, eccezioni, convenzioni di entrambe.
Soluzione: due Adapter:

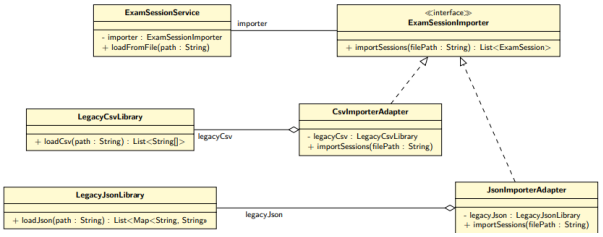
- CsvImporterAdapter;
- JsonImporterAdapter;

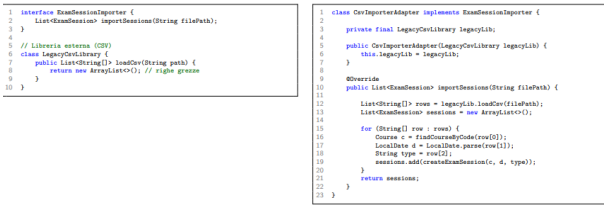
che implementano la stessa interfaccia ExamSessionImporter.

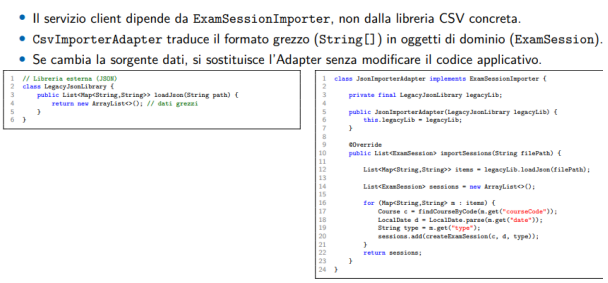
In UniManager il dominio vede sempre e solo:

- un'interfaccia (ExamSessionImporter),
- un tipo di ritorno (List).

Integrazioni diverse (CSV, JSON, API) sono intercambiabili. Possiamo aggiungere nuovi formati senza toccare il dominio. Tutta la logica di mapping e parsing resta confinata negli Adapter.



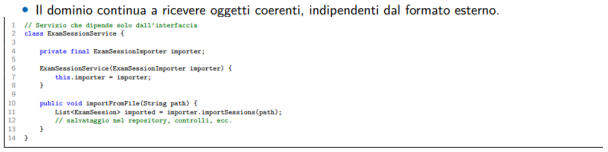




- Il servizio client dipende da `ExamSessionImporter`, non dalla libreria CSV concreta.
- `CsvImporterAdapter` traduce il formato grezzo (`String[]`) in oggetti di dominio (`ExamSession`).
- Se cambia la sorgente dati, si sostituisce l'Adapter senza modificare il codice applicativo.



- L'Adapter JSON applica la stessa interfaccia del caso CSV: cambia la sorgente, non il contratto.
- La mappatura da `Map<String,String>` a `ExamSession` resta confinata nel layer di integrazione.
- Il dominio continua a ricevere oggetti coerenti, indipendenti dal formato esterno.



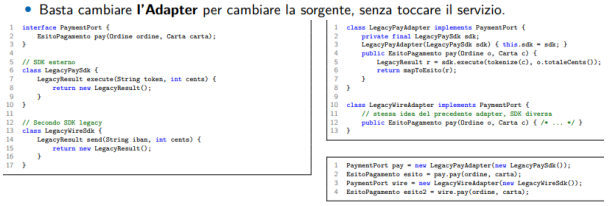
**Configurazione CSV**



**Configurazione JSON**



- `ExamSessionService` non sa se i dati provengono da CSV o JSON.
- Basta cambiare **l'Adapter** per cambiare la sorgente, senza toccare il servizio.



- Il checkout dipende da `PaymentPort`, non dalla SDK esterna.
- Mapping e conversioni restano confinati nell'Adapter.



- Il dominio usa `IdentityProvider` con modello interno stabile.
- Cambio provider SSO: si sostituisce l'Adapter, non il servizio di login.

*Vantaggi:*

- *DIP (Dependency Inversion):*
  - ExamSessionService dipende dall'interfaccia ExamSessionImporter;
  - le librerie esterne restano “dietro” gli Adapter.
- *OCP (Open/Closed):*
  - per supportare un nuovo formato (XML, API diversa,…) basta aggiungere un nuovo Adapter;
  - il servizio che importa non cambia.
- *SRP (Single Responsibility):*
  - parsing, conversione e mapping sono confinati nell'Adapter;
  - il dominio e i servizi non si riempiono di dettagli I/O e formati.
- *Testabilità e manutenzione:*
  - Adapter testabile in isolamento;
  - il servizio può usare importer fittizi nei test.

*Errori tipici:*

- Adapter che fa troppo: se oltre a convertire dati inizia a contenere logica di dominio, si viola SRP.
- Service che dipende comunque dalla libreria esterna: se il dominio usa tipi della libreria (es. String[] o Map ovunque), si indebolisce DIP.
- If/else sui formati dentro il servizio: scegliere CSV/JSON con switch nel servizio porta a modifiche frequenti: peggiora OCP.

**Observer:** Con Adapter abbiamo gestito l'integrazione con l'esterno. Ora il problema è interno: come reagire a un evento? Il contesto tipico è:

- una classe genera un evento (es. nuovo appello);
- più componenti vogliono essere informati.

Soluzione ingenua: la classe chiama direttamente tutti gli altri (sendEmail(), updateUi(), writeLog(),…). Conseguenze:

- la classe finisce per conoscere troppi dettagli su altri componenti;
- il codice diventa più difficile da mantenere e capire;
- aggiungere un nuovo comportamento richiede modifiche al codice esistente.

Il pattern Observer separa:

- il Subject, che produce gli eventi;
- gli Observer, che vogliono essere notificati.

Meccanismo:

- gli Observer si registrano presso il Subject;
- quando qualcosa cambia, il Subject chiama un unico metodo (notifyObservers()) che notifica tutti gli iscritti;
- il Subject non conosce i dettagli di cosa faranno gli Observer.

Effetto:

- possiamo aggiungere/rimuovere osservatori senza modificare il Subject;
- il sistema diventa più estendibile e testabile.

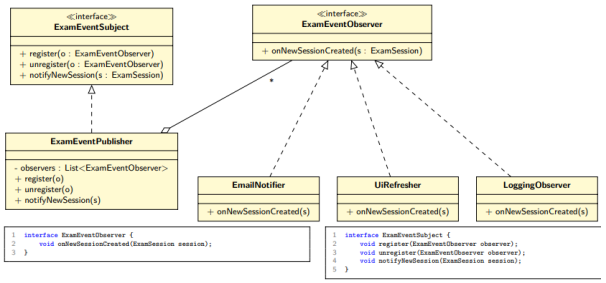
### UniManager: notificare la creazione di un appello

In UniManager, la creazione di un nuovo ExamSession deve attivare più reazioni:

- aggiornare la vista,
- inviare notifiche agli studenti,
- scrivere un log.

Il servizio che crea l'appello non deve conoscere questi dettagli. Con Observer:

- ExamEventPublisher funge da Subject;
- EmailNotifier, UiRefresher, LoggingObserver sono Observer che reagiscono indipendentemente.




```java
interface ExamEventObserver {
    void onNewSessionCreated(ExamSession session);
}
```

```java
interface ExamEventSubject {
    void register(ExamEventObserver observer);
    void unregister(ExamEventObserver observer);
    void notifyNewSession(ExamSession session);
}
```

```java
class ExamEventPublisher implements ExamEventSubject {
    private final List<ExamEventObserver> observers = new ArrayList<>();

    @Override
    public void register(ExamEventObserver observer) {
        observers.add(observer);
    }
    @Override
    public void unregister(ExamEventObserver observer) {
        observers.remove(observer);
    }
    @Override
    public void notifyNewSession(ExamSession session) {
        for (ExamEventObserver o : observers) {
            o.onNewSessionCreated(session);
        }
    }
}
```

- `ExamEventPublisher` gestisce solo il ciclo di notifica (registro observer + dispatch evento).
- Il Subject dipende dall'interfaccia `ExamEventObserver`: nuove reazioni si aggiungono senza modificare il publisher.

```java
class EmailNotifier implements ExamEventObserver {

    @Override
    public void onNewSessionCreated(ExamSession session) {

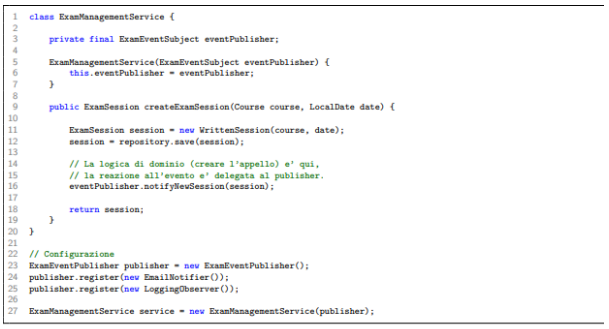
        // invia email agli studenti
        // iscritti al corso dell'appello
        System.out.println("Invio email per: " + session.getCourse().
        getName());
    }
}
```

```java
class LoggingObserver implements ExamEventObserver {

    @Override
    public void onNewSessionCreated(ExamSession session) {

        // scrive su un log applicativo
        System.out.println("LOG: creato appello " + session.getDate());
    }
}
```

- Ogni observer implementa una reazione specifica allo stesso evento, senza accoppiamento diretto tra observer.
- Per aggiungere una nuova reazione basta introdurre una nuova classe observer e registrarla nel publisher.



```
class ExamManagementService {

    private final ExamEventSubject eventPublisher;

    ExamManagementService(ExamEventSubject eventPublisher) {
        this.eventPublisher = eventPublisher;
    }

    public ExamSession createExamSession(Course course, LocalDate date) {

        ExamSession session = new WrittenSession(course, date);
        session = repository.save(session);

        // La logica di dominio (creare l'appello) e' qui,
        // la reazione all'evento e' delegata al publisher.
        eventPublisher.notifyNewSession(session);

        return session;
    }
}

// Configurazione
ExamEventPublisher publisher = new ExamEventPublisher();
publisher.register(new EmailNotifier());
publisher.register(new LoggingObserver());

ExamManagementService service = new ExamManagementService(publisher);
```

- Per aggiungere un nuovo comportamento alla creazione di un appello:
  - basta creare un nuovo `ExamEventObserver` e registrarlo;
  - nessuna modifica al codice di `ExamManagementService`.

*Vantaggi:*

- *DIP (Dependency Inversion):*
  - il Subject dipende dall'interfaccia ExamEventObserver, non dagli observer concreti;
  - le reazioni (email, log, UI, . . . ) restano dettagli sostituibili.
- *OCP (Open/Closed):*
  - nuove reazioni all'evento si aggiungono registrando un nuovo Observer;
  - il codice che genera l'evento non va modificato.
- *SRP (Single Responsibility):*
  - il servizio si concentra sulla logica principale (creare l'appello);
  - le reazioni all'evento stanno in classi separate, ognuna con un compito specifico.
- *Testabilità:*
  - nei test si possono usare observer fake e verificare che la notifica avvenga correttamente.

*Errori tipici:*

- Subject che conosce gli observer concreti: se istanzia direttamente new EmailNotifier(), perde DIP.
- Observer con responsabilità miste: se un singolo observer gestisce email + log + UI, si perde SRP.
- Notifiche “a cascata” difficili da controllare:
  - se gli observer generano altri eventi senza regole chiare, il sistema diventa fragile;
  - spesso si finisce a “patchare” codice esistente, andando contro OCP.

**Adapter vs Observer:**

- *Adapter:*
  - risolve problemi di incompatibilità di interfacce;
  - tipico per integrare librerie, servizi, formati dati esterni;
  - es. UniManager: import di appelli da CSV/JSON con un'interfaccia interna pulita.
- *Observer:*
  - risolve problemi di notifica di eventi;
  - tipico per disaccoppiare chi produce un evento da chi reagisce;
  - es. UniManager: notifiche agli studenti e aggiornamento viste quando viene creato un nuovo appello.
- *In comune:*
  - proteggono il codice da dipendenze rigide;
  - spostano la complessità nei confini del sistema (integrazione, eventi);
  - rendono il design più modulare e evolvibile.

**Design Pattern essenziali:**

| Pattern | Problema che risolve | Esempi in UniManager |
|---|---|---|
| Strategy | Comportamenti che possono variare nel tempo; evita lunghi blocchi `if/else`. | Ordinamento degli appelli secondo criteri diversi (data, corso, iscritti). |
| Factory Method | Creazione di oggetti che può richiedere logiche diverse; centralizza la costruzione. | Creazione di diverse tipologie di ExamSession (Written, Oral). |
| Adapter | Integrazione di librerie/formati esterni incompatibili con il dominio interno. | Import di appelli da CSV/JSON tramite ExamSessionImporter. |
| Observer | Notificare cambiamenti senza accoppiamento rigido tra componenti. | Avvisare studenti, viste e log quando viene creato un nuovo appello. |

I pattern affrontano quattro problemi distinti: comportamenti, creazione, integrazione ed eventi. Insieme costituiscono la base per un design evolvibile.

> **Sintesi:**
>
> - Adapter e Observer affrontano due problemi ricorrenti diversi: integrazione con esterni ed eventi interni tra componenti.
> - Adapter protegge il dominio da formati e librerie incompatibili, spostando la complessità ai confini del sistema.
> - Observer consente di reagire a cambiamenti senza legare direttamente chi produce l'evento a tutti i componenti interessati.
> - Entrambi i pattern favoriscono estendibilità e riduzione dell'accoppiamento, in linea con OCP e DIP.
> - Il beneficio dei pattern dipende da come vengono applicati: migliorano il design se rispondono a una variabilità reale.
> - Gli esempi su UniManager mostrano come questi pattern preparino il sistema a integrazioni e notifiche future.

## Qualità del design: code smells

**Perché parlare di qualità interna?** Un software può funzionare ed essere comunque difficile da mantenere e da far crescere nel tempo. La qualità interna determina in modo diretto:

- quanto è semplice correggere bug senza introdurre effetti collaterali;
- quanto è facile aggiungere nuove funzionalità senza “toccare tutto”;
- quanto si capisce il codice dopo settimane/mesi (anche da parte di chi non lo ha scritto).

Nella pratica, la qualità interna è ciò che rende un progetto:

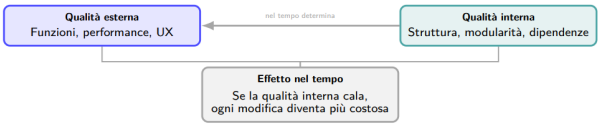
- evolvibile invece che fragile,
- leggibile invece che opaco,
- modulare invece che intrecciato,
- più testabile e quindi più sicuro da modificare.

**Qualità esterna vs qualità interna:**

- *Qualità esterna:* ciò che percepisce l'utente finale (funzionalità offerte, prestazioni, affidabilità, esperienza d'uso).
- *Qualità interna:* proprietà strutturali del codice (chiarezza, organizzazione, modularità, dipendenze sane).

Effetto tipico nei progetti reali: oggi funziona → domani costa troppo cambiarlo o estenderlo. In questa lezione ci concentriamo in particolare su:

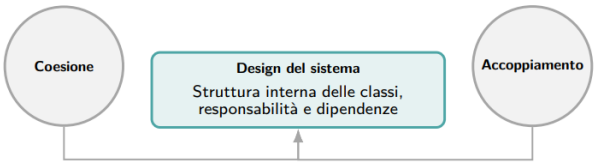
- segnali che indicano un design che sta lentamente peggiorando;
- strumenti concettuali per riconoscerli e intervenire in anticipo.




**Come leggere il design:**

- *Coesione:* quanto gli elementi di uno stesso modulo/classe stanno bene insieme e lavorano per un unico scopo. Se c'è alta coesione allora avrò una classe focalizzata, più facile da capire e testare.
- *Accoppiamento:* quanto un modulo/classe dipende dagli altri. Se ho basso accoppiamento allora ci saranno meno dipendenze rigide, cambiamenti più locali.

Regola pratica: alta coesione + basso accoppiamento.




**Coesione:** misura la “compattezza” delle responsabilità assegnate a un modulo o a una classe. Un modulo realmente coeso:

- ha un obiettivo chiaro e facilmente esplicitabile;
- contiene responsabilità dello stesso tipo e tra loro correlate;
- si può descrivere in modo naturale con una singola frase.

Segnali tipici di bassa coesione:

- una classe che offre molte funzionalità non correlate tra loro;
- metodi che lavorano su concetti diversi e che non condividono un filo logico comune;
- nomi generici (Manager, Utils, Helper) che cercano di “coprire tutto” senza un ruolo preciso.

**Accoppiamento:** misura quanto un modulo dipende da altri moduli e dalle loro decisioni interne. Un accoppiamento elevato significa:

- conoscere e sfruttare dettagli interni di altri componenti;
- vincolarsi a implementazioni concrete invece che a contratti astratti;
- amplificare l'effetto di una modifica anche lontano dal punto di intervento.

Segnali tipici di alto accoppiamento:

- accesso diretto a campi interni e strutture dati di altre classi;
- catene di chiamate lunghe (train wreck);
- dipendenze diffuse “ovunque” verso componenti instabili o tecnologici.

Nei progetti reali emergono spesso due problemi ricorrenti, che tendono a rafforzarsi a vicenda se non vengono controllati:

1. classi che crescono troppo nel tempo, accumulando responsabilità diverse → bassa coesione;
2. componenti sempre più intrecciati tra loro, con dipendenze diffuse → alto accoppiamento.

Una regola pratica utile per diagnosticare la situazione è la seguente:

- se per cambiare una parte A è necessario capire anche B, C, D, allora l'accoppiamento è probabilmente eccessivo;
- se una classe ha “troppe ragioni per cambiare” in contesti diversi, allora la sua coesione è probabilmente insufficiente.

L'obiettivo del buon design è mantenere questi due aspetti in equilibrio, così che ogni modulo possa evolvere in modo il più possibile indipendente.




Spesso il problema non è che il codice sia “sbagliato” o non funzionante, ma che risulti scomodo da leggere, capire e modificare. Situazioni tipiche in cui emerge questa sensazione:

- metodi troppo lunghi, con molti passaggi e livelli di dettaglio mescolati;
- classi che crescono a ogni nuova feature senza una struttura chiara;
- logiche di dominio mescolate con dettagli tecnici o di presentazione.

Questa sensazione di disagio è in realtà preziosa, non è una prova matematica di errore, ma è un segnale da prendere sul serio e verificare usando criteri progettuali e strumenti concettuali.

**Code smells:** sono indicatori di possibili problemi nella struttura e nell'organizzazione del design. Non sono bug né errori di funzionamento:

- il programma continua a funzionare correttamente;
- ma il design mostra segnali di tensione e perdita di chiarezza.

Sono particolarmente utili perché:

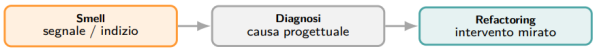
- aiutano a individuare il degrado del design prima che diventi evidente o ingestibile;
- suggeriscono punti del codice in cui il refactoring può portare il maggior beneficio.

**Smells e refactoring:**

- Smell → ipotesi di problema nel design: qualcosa suggerisce che la struttura non sia ottimale.
- Refactoring → intervento disciplinato per migliorare la struttura senza cambiare il comportamento osservabile.

L'idea chiave è che gli smells non dicono “cambia qui” in modo automatico, ma dicono “qui vale la pena guardare meglio e fare una verifica”. In pratica, il flusso di lavoro tipico è:

- si osserva il codice e si riconoscono possibili segnali;
- si formula una diagnosi (quale problema progettuale c'è sotto?);
- si applicano refactoring mirati, piccoli e controllati.




Gli smell che vedremo sono:

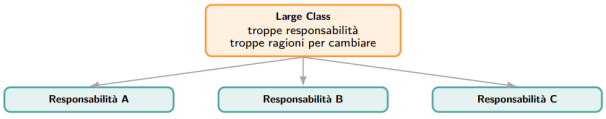
- *Large Class:* troppe responsabilità nella stessa classe.
- *Long Method:* metodo troppo lungo e difficile da manutenere.
- *Feature Envy:* metodo che usa più dati di un'altra classe che della propria.
- *Data Clumps:* gruppi di parametri che ricorrono sempre insieme.
- *God Object:* classe centrale che coordina, decide e conosce troppo.
- *Duplicated Code:* stessa logica copiata in più punti.
- *Primitive Obsession:* uso eccessivo di tipi primitivi al posto di concetti di dominio.
- *Divergent Change:* una classe cambia per motivi eterogenei.
- *Shotgun Surgery:* una modifica richiede piccoli cambi sparsi in molti file.

**Large Class:** Una Large Class è una classe che accumula troppe responsabilità e, di conseguenza, troppe ragioni diverse per cambiare nel tempo. Effetti tipici che emergono in questi casi:

- bassa coesione, perché le responsabilità non sono omogenee;
- maggiore difficoltà di test, poiché è complesso isolare i comportamenti;
- modifiche rischiose, perché molte funzionalità “passano tutte di lì”.

Questo smell si accompagna spesso a:

- nomi generici ( `*Manager` , `*Service` ) che non chiariscono il ruolo reale;
- una crescita “per accumulo” di metodi, senza una struttura progettuale chiara.




**Example**

```
class ExamManagementService {
    // ERRORE: responsabilita' studenti nella stessa classe
    void registerStudent(...) { ... }

    // ERRORE: responsabilita' appelli nella stessa classe
    void createSession(...) { ... }
    void enrollStudent(...) { ... }
    void publishResults(...) { ... }

    // ERRORE: persistenza mescolata con dominio
    void saveOnFile(...) { ... }
    void loadFromFile(...) { ... }

    // ERRORE: presentazione dentro il service
    String formatCliMessage(...) { ... }
}
```

- Una sola classe copre dominio, persistenza e presentazione.

- Ogni nuova richiesta tende a finire nello stesso punto.

```java
// FIX: solo regole di business sugli appelli
class ExamDomainService {
    void createSession(...) { ... }
    void enrollStudent(...) { ... }
    void publishResults(...) { ... }
}

// FIX: persistenza isolata in un componente dedicato
class ExamRepository {
    void save(ExamSession s) { ... }
    ExamSession findById(String id) { ... }
}

// FIX: presentazione separata ai margini (CLI/UI)
class ExamPresenter {
    String formatCliMessage(...) { ... }
}
```

**Large class (refactoring)**

- Obiettivo: separare per responsabilità e per motivo di cambiamento.
- Decomposizione esplicita: dominio, persistenza e presentazione.
- Effetto: classi più coese e modifiche più locali.

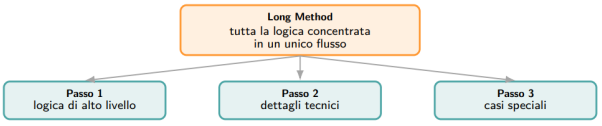
**Long Method:** Un Long Method è un metodo che cresce oltre misura e tende a concentrare troppa logica in un unico punto del codice. Tipicamente mescola tra loro:

- livelli di astrazione diversi (alto livello e dettagli operativi);
- logica di business e dettagli tecnici o infrastrutturali;
- casi normali ed eccezioni gestiti nello stesso flusso lineare.

Effetti tipici di questo smell:

- è difficile capire “che cosa fa” davvero il metodo nel suo insieme;
- è difficile riusare o testare singole parti della logica;
- aumenta la probabilità di introdurre bug quando si modifica anche solo un passaggio.

Refactoring tipico e consigliato: **Extract Method**.




**Example**

```java
void processEnrollment(String studentId, String examId) {
    // ERRORE: metodo lungo con troppi dettagli interni
    Student s = studentRepo.find(studentId);
    ExamSession e = examRepo.find(examId);
    if (s == null || e == null) throw new IllegalArgumentException();

    // ERRORE: regole business mescolate nel metodo orchestratore
    if (!e.isOpen()) throw new IllegalStateException();
    if (!s.hasPrerequisitesFor(e)) throw new IllegalStateException();
    if (s.hasScheduleConflict(e)) throw new IllegalStateException();

    // ERRORE: persistenza nello stesso metodo
    enrollmentRepo.save(new Enrollment(s, e));
    // ERRORE: audit e notifica nello stesso flusso
    audit.log("ENROLL", studentId, examId, e.getDate().toString());
    notifier.notifyStudent(studentId, "Iscrizione completata");
    // ERRORE: output UI/CLI dentro logica applicativa
    System.out.println("OK " + studentId + " -> " + e.getCourse().getCode());
}
```

- Validazione, regole, persistenza, audit e output nello stesso metodo.

```java
void processEnrollment(String studentId, String examId) {
    // FIX: orchestrazione ad alto livello, leggibile
    EnrollmentContext ctx = loadContext(studentId, examId);
    // FIX: regole business isolate
    validateEnrollment(ctx);
    // FIX: persistenza isolata
    persistEnrollment(ctx);
    // FIX: side-effect isolati
    logEnrollment(ctx);
    notifyEnrollment(ctx);
}
```

**Long method (refactoring)**

- Refactoring applicato: **Extract method**
- Ogni step ha nome esplicito e responsabilità unica
- Il metodo principale torna leggibile a livello di processo

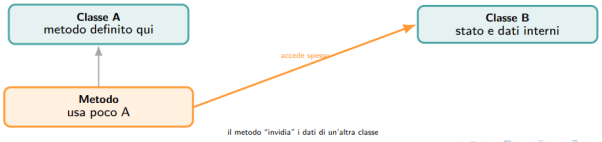
**Feature Envy:** Un Feature Envy si verifica quando un metodo utilizza più dati e comportamenti di un'altra classe che non quelli della classe in cui è definito. Indicazione progettuale tipica:

- la responsabilità è probabilmente posizionata nel posto sbagliato;
- il modello concettuale risulta sbilanciato o mal distribuito.

Sintomi comuni di questo smell:

- catene di accessi del tipo `a.getB().getC().getX()`, che rivelano una conoscenza eccessiva della struttura interna altrui;
- un metodo che “conosce troppo” lo stato e i dettagli di un altro oggetto.

Refactoring tipici: **Move Method** oppure **Extract Class**.




### Example

```java
class ExamSessionPrinter {
    String formatForStudent(ExamSession s) {
        // ERRORE: il metodo naviga troppo dentro altri oggetti
        return s.getCourse().getCode() + " - "
            + s.getCourse().getName() + " - "
            + s.getDate().toString() + " - "
            + s.getRoom().getName() + " - "
            + s.getTeacher().getSurname();
    }
}
```

- Il metodo usa quasi solo dati interni di ExamSession, Course, Room, Teacher.
- La responsabilità di formattazione è nel posto sbagliato.

```java
class ExamSession {
    // FIX: il comportamento vive vicino ai dati che usa
    String summaryForStudent() {
        return course.getCode() + " - "
            + course.getName() + " - "
            + date.toString() + " - "
            + room.getName() + " - "
            + teacher.getSurname();
    }
}

class ExamSessionPrinter {
    // FIX: il printer delega, non conosce dettagli interni
    String formatForStudent(ExamSession s) {
        return s.summaryForStudent();
    }
}
```

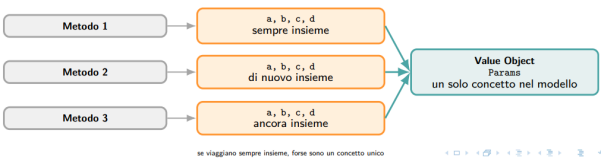
**Long method (refactoring)**

- Refactoring applicato: **Move method**
- Il comportamento torna vicino ai dati che usa davvero

**Data Clumps:** I Data Clumps sono gruppi di variabili che compaiono sempre insieme e vengono passati in giro come un unico “pacchetto”, anche se nel codice restano separati. Situazioni tipiche:

- gli stessi parametri ripetuti in molti metodi e costruttori;
- le stesse variabili che “viaggiano insieme” tra classi e servizi.

Questo smell spesso indica che nel modello manca un concetto: un'entità o, più spesso, un Value Object che rappresenti quel gruppo in modo esplicito. Il refactoring tipico è quello di introdurre un **Value Object** (es. StudentId, ExamInfo, ExamDescriptor,…).




### Example

```java
// ERRORE: stesso gruppo di parametri ripetuto in piu' metodi
void enroll(
    String studentId,
    String studentName,
    String courseCode,
    String courseName,
    LocalDate examDate,
    String roomName
) { ... }

void publishResult(
    String studentId,
    String studentName,
    String courseCode,
    String courseName,
    LocalDate examDate,
    String roomName,
    int grade
) { ... }
```

- Lo stesso blocco di parametri si ripete in più metodi.
- Manca un concetto esplicito nel modello.

```java
// FIX: i campi che viaggiano insieme diventano un Value Object
final class StudentDescriptor {
    private final StudentId studentId;
    private final StudentName studentName;
    // costruttore + getter
}

final class ExamDescriptor {
    private final CourseCode courseCode;
    private final CourseName courseName;
    private final LocalDate examDate;
    private final String roomName;
    // costruttore + getter
}

// FIX: firme piu' corte e semanticamente piu' chiare
void enroll(StudentDescriptor student, ExamDescriptor exam) { ... }
void publishResult(StudentDescriptor student, ExamDescriptor exam, int grade) { ... }
```

**Data clumps (refactoring)**

- Refactoring applicato: **Introduce value object**
- Firma più corta, semantica più chiara, validazioni centralizzabili
- In questo esempio introduciamo anche StudentDescriptor per accorpare i dati studente che viaggiano insieme

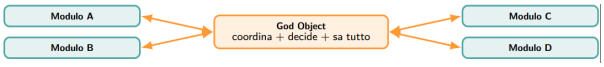
**God Object:** Un God Object è una classe “onnipotente” che finisce per diventare il centro operativo del sistema e concentra decisioni che dovrebbero essere distribuite. In pratica è una classe che:

- coordina tutto (orchestrazione di molte parti),
- decide tutto (regole e policy sparse),
- sa tutto (conoscenza di dettagli interni di molti moduli).

Tipicamente produce effetti molto negativi:

- accoppia tra loro parti del sistema che dovrebbero restare separate;
- diventa un collo di bottiglia cognitivo: per capire il sistema bisogna passare da lì;
- impedisce modularità e testabilità, perché è difficile isolarla e simulare dipendenze.

È spesso la forma estrema di Large Class + responsabilità non distribuite (tutto viene “assorbito” in un punto).




Example

```java
class UniManagerService {
    // ERRORE: dominio concentrato qui
    void createSession(...) { ... }
    void enrollStudent(...) { ... }
    void publishGrades(...) { ... }
    // ERRORE: integrazione/export nello stesso punto
    void exportCsv(...) { ... }
    void importCsv(...) { ... }
    // ERRORE: persistenza nello stesso punto
    void saveOnFile(...) { ... }
    void loadFromFile(...) { ... }
    // ERRORE: notifiche + presentazione nello stesso punto
    void sendNotifications(...) { ... }
    void buildDashboard(...) { ... }
}
```

- Una sola classe concentra dominio, I/O, reportistica e presentazione.
- Ogni modulo finisce per dipendere da questo punto centrale.

```
// FIX: separazione esplicita delle responsabilita'
class ExamDomainService {
    private final ExamRepository repo;

    ExamDomainService(ExamRepository repo) { this.repo = repo;
    void closeSession(String examId) { ... }
}

class ExamRepository {
    ExamSession findById(String examId) { /* query */ return null; }
    void save(ExamSession session) { /* persistenza */ }
}

class NotificationService {
    void notifyClosure(String examId) {
        // invio email / evento verso UI
    }
}

class ExamClosureUseCase {
    private final ExamDomainService domain;
    private final NotificationService notifications;

    ExamClosureUseCase(ExamDomainService domain, NotificationService notifications) { ... }
    void execute(String examId) { ... }
}
```

**God object (refactoring)**

- I ruoli sono separati in servizi distinti
- Il caso d'uso non passa più da una classe onnicomprensiva

**Duplicated Code:** Il Duplicated Code compare quando la stessa logica è copiata in più punti. Effetti tipici:

- ogni correzione va ripetuta in più file/metodi;
- alto rischio di inconsistenza tra copie quasi uguali.

Indizio operativo: “copia e incolla” usato per introdurre varianti minime. Refactoring tipici: **Extract Method, Extract Class, Template Method**.

### Example

```
class ExamService {
    boolean canEnroll(Student s, ExamSession e) {
        // ERRORE: questa sequenza sara' copiata anche altrove
        if (!e.isOpen()) return false;
        if (!s.hasPrerequisitesFor(e)) return false;
        if (s.hasScheduleConflict(e)) return false;
        return true;
    }
}

class RetakeService {
    boolean canEnrollRetake(Student s, ExamSession e) {
        // ERRORE: duplicazione delle stesse tre regole
        if (!e.isOpen()) return false;
        if (!s.hasPrerequisitesFor(e)) return false;
        if (s.hasScheduleConflict(e)) return false;
        return s.hasPreviousFail(e.getCourse());
    }
}
```

- Le prime tre regole sono duplicate.
- La variante reale è solo l'ultima condizione.

```
// FIX: regole comuni estratte in un metodo unico
class EnrollmentPolicy {
    boolean baseChecks(Student s, ExamSession e) {
        return e.isOpen()
            && s.hasPrerequisitesFor(e)
            && !s.hasScheduleConflict(e);
    }
}

class RetakePolicy extends EnrollmentPolicy {
    // FIX: qui resta solo la variante specifica
    boolean canEnrollRetake(Student s, ExamSession e) {
        return baseChecks(s, e) && s.hasPreviousFail(e.getCourse());
    }
}
```

**Duplicated code (refactoring)**

- Regole comuni in un punto solo
- Le variabili restano piccole e controllabili

**Primitive Obsession:** Riguarda l'uso eccessivo di tipi primitivi (String, int, double) al posto di concetti di dominio. Effetti tipici:

- validazioni sparse e duplicate;
- parametri poco autoesplicativi;
- errori di scambio tra campi semanticamente diversi.

Refactoring tipici: **Replace Data Value with Object, Introduce Value Object**.

### Example

```
void registerGrade(
    String studentId,
    String courseCode,
    int grade,
    String examDate
) {
    // ERRORE: validazioni sparse su primitivi
    if (studentId == null || studentId.isBlank()) throw new IllegalArgumentException();
    if (courseCode == null || courseCode.isBlank()) throw new IllegalArgumentException();
    if (grade < 18 || grade > 30) throw new IllegalArgumentException();
    // ...
}
```

- I parametri sono primitivi, ma rappresentano concetti di dominio.
- Le regole di validazione non sono incapsulate.

```
// FIX: i vincoli entrano in tipi di dominio dedicati
final class StudentId { /* validazione nel costruttore */ }
final class CourseCode { /* validazione nel costruttore */ }
final class GradeValue { /* range 18..30 */ }
final class ExamDate { /* formato/data valida */ }

void registerGrade(
    StudentId studentId,
    CourseCode courseCode,
    GradeValue grade,
    ExamDate examDate
) { /* ... */ }
```

**Primitive obsession (refactoring)**

- I vincoli diventano parte del modello
- La firma del metodo è più espressiva e robusta

**Divergent Change:** Divergent Change si ha quando una singola classe viene modificata per motivi diversi e non correlati. Il segnale tipico è che ogni nuova richiesta (report, validazione, persistenza, UI) tocca sempre la stessa classe. L'effetto è quello di forte instabilità locale e regressioni frequenti. Il refactoring tipico: separare responsabilità per asse di cambiamento.

Example

```
class ExamManagementService {
    // ERRORE: cambia se cambiano regole business
    void createSession(...) { ... }

    // ERRORE: cambia se cambia formato export
    String exportCsv(...) { ... }

    // ERRORE: cambia se cambia storage
    void saveOnFile(...) { ... }

    // ERRORE: cambia se cambia interfaccia utente
    String formatUiMessage(...) { ... }
}
```

- Quattro motivi di cambiamento nella stessa classe

```
// FIX: una classe per ciascun asse di cambiamento
class ExamDomainService { void createSession(...) { ... } }
class ExamExporter { String exportCsv(...) { ... } }
class ExamRepository { void saveOnFile(...) { ... } }
class ExamPresenter { String formatUiMessage(...) { ... } }
```

**Divergent change (refactoring)**

- Ogni classe di cambiamento ha una classe dedicata
- Le modifiche eterogenee non ricadono più nello stesso file

**Shotgun Surgery:** Shotgun Surgery è il caso opposto del Divergent Change: una modifica concettualmente unica richiede tanti cambi minori in classi diverse. Segnali tipici:

- pull request con molti file toccati per una feature piccola;
- rischio elevato di dimenticare un punto.

Refactoring tipico: accorpare comportamento correlato e chiarire i confini.

Example

- Nuova regola: “uno studente non può iscriversi a due appelli nello stesso slot orario”.
- Implementazione dispersa:

```
// ERRORE: una sola regola richiede cambi sparsi
StudentService.java
ExamSessionService.java
EnrollmentFacade.java
CalendarService.java
CliController.java
CsvExporter.java
```

- Una sola regola di dominio impatta troppi punti diversi.

```java
// FIX: centralizzare la regola in una policy unica
class SchedulingPolicy {
    boolean canEnroll(Student s, ExamSession e) {
        return e.isOpen() && !s.hasScheduleConflict(e);
    }
}

class EnrollmentService {
    private final SchedulingPolicy policy;
    // FIX: il service delega la regola e resta stabile
    boolean enroll(Student s, ExamSession e) {
        if (!policy.canEnroll(s, e)) return false;
        // save + notify
        return true;
    }
}
```

**Shotgun surgery (refactoring)**

- La regola è centralizzata in una policy unica
- I moduli delegano: una modifica tocca pochi punti

**Riepilogo:**

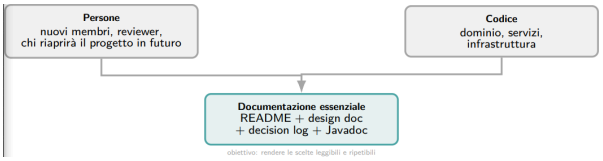
- La qualità interna del design incide direttamente sulla capacità di mantenere, testare ed evolvere il software nel tempo.
- Coesione e accoppiamento sono criteri pratici per leggere la “salute” del codice oltre il solo comportamento corretto.
- I code smells non sono prove definitive di errore, ma segnali utili per individuare zone a rischio del design.
- Riconoscere presto gli smells aiuta a intervenire con refactoring prima che il degrado diventi architetturale.
- I miglioramenti di design spesso non cambiano il comportamento esterno, ma riducono fragilità e costo delle modifiche future.
- Gli esempi su UniManager mostrano come collegare osservazione degli smells e refactoring concreto.

## Documentazione essenziale

La documentazione viene talvolta trattata come attività accessoria. In realtà è uno strumento di ingegneria:

- riduce incomprensioni;
- accelera onboarding e passaggi di consegne;
- rende il progetto manutenibile e accogliente.

Obiettivo pratico: meno attrito tra persone e codice ⇒ più qualità e meno errori.




**A chi serve davvero la documentazione?**

1. A chi ha già lavorato sul progetto, quando:
   - lo riapre dopo settimane o mesi;
   - deve intervenire rapidamente su un bug o una modifica senza ricordare tutti i vincoli progettuali.
2. A chi entra successivamente nel progetto:
   - un onboarding più rapido riduce domande ricorrenti, incomprensioni e interventi correttivi.
3. A chi valuta, estende o integra il progetto:
   - consente di comprendere scelte architetturali e trade-off senza dover analizzare l'intero codice sorgente.

**Documentazione essenziale:** Non è necessario “documentare tutto” né descrivere ogni dettaglio del sistema. È invece fondamentale mantenere un insieme ridotto ma significativo di documenti sempre coerenti con il codice:

- *README:* punto di ingresso al progetto, guida all'onboarding e all'avvio rapido per chi lo utilizza o lo esplora per la prima volta.
- *Design document:* descrizione delle principali scelte architetturali, dei vincoli e delle motivazioni che le hanno guidate.
- *Decision log:* tracciamento delle decisioni tecniche rilevanti, utile per comprendere perché è stata scelta una soluzione rispetto ad altre.
- *Documentazione API mirata (Javadoc, docstring, godoc,…):* descrive i contratti delle API pubbliche e i comportamenti rilevanti per l'uso corretto del codice.

Regola pratica di riferimento:

- poco, chiaro e aggiornato → più utile di una documentazione ampia ma obsoleta.
- **Terminologia: ADR** (Architecture Decision Record) indica una nota breve che registra una singola decisione architetturale.

**Documentazione come codice (docs-as-code):** La documentazione essenziale non vive "fuori" dal progetto, ma sta nel repository, versionata insieme al codice. Una scelta pratica ricade su:

- Markdown (.md) per documenti operativi brevi (README, ADR, note tecniche);
- LaTeX per documenti più strutturati e versionati (relazioni, report tecnici, slide, documenti di progetto).

Quando il codice cambia, la documentazione "si vede" che va aggiornata (stesso commit, stessa responsabilità). Un anti-pattern ricorrente è: documento Word / Drive separato ⇒ divergenza rapida ⇒ perdita di fiducia. L'effetto pratico è che la documentazione si aggiorna insieme al codice e resta allineata nel tempo.

**README, il primo documento che si legge:** Rappresenta la porta di ingresso al repository ed è spesso il primo punto di contatto con il progetto. Si tratta di un documento operativo orientato all'uso (uno strumento di onboarding tecnico). Consente di ottenere rapidamente un primo risultato funzionante e verificabile. Un README ben scritto contribuisce a ridurre:

- tempo perso nella fase di setup e configurazione;
- errori di esecuzione dovuti a istruzioni incomplete;
- domande ripetitive e interventi di supporto non necessari.

**Le 4 domande a cui deve rispondere:**

1. *Che cos'è il progetto?*
   - fornisce il contesto generale e l'ambito applicativo;
   - permette di capire subito se il progetto è rilevante per chi lo sta esplorando.
2. *Cosa fa il progetto?*
   - descrive le funzionalità principali in modo sintetico;
   - evita descrizioni prolisse: poche righe sono più efficaci di una spiegazione lunga e dispersiva.
3. *Come si esegue?*
   - spiega come ottenere un'installazione funzionante;
   - include un quickstart riproducibile, verificabile passo dopo passo.
4. *Come si contribuisce?*
   - chiarisce regole minime di collaborazione;
   - riduce errori dovuti a stili incoerenti o test mancanti.




**Struttura consigliata (template mentale):**

- *Overview:*
  - introduce il progetto in un paragrafo chiaro;
  - evidenzia obiettivi e funzionalità principali.
- *Quickstart:*
  - fornisce comandi reali e immediatamente eseguibili;
  - porta rapidamente a un primo risultato funzionante.
- *Usage:*
  - mostra 1–2 esempi rappresentativi di utilizzo;
  - chiarisce il comportamento senza aprire il codice.
- *Configurazione:*
  - documenta variabili, file e dipendenze rilevanti;
  - chiarisce cosa è personalizzabile e come.
- *Test:*
  - spiega come eseguire la suite di test;
  - rende verificabile il corretto funzionamento.
- *Contributing:*
  - definisce regole minime per contribuire in modo coerente;
  - riduce attriti su branch, review e stile del codice.

**README anti-pattern (cose che lo rovinano):**

- *Wall of text:* assenza di sezioni chiare e di un quickstart immediato, che rende difficile individuare le informazioni essenziali.
- *Istruzioni vaghe:* indicazioni generiche come “installare le dipendenze” senza specificare strumenti, versioni o comandi concreti.
- *Comandi non verificati o incompleti:* esempi che non funzionano o che producono errori non documentati.
- *Duplicazioni e incoerenze:* stesse informazioni ripetute in più punti del README con contenuti non allineati tra loro.
- *README usato come contenitore indistinto:* accumulo di informazioni eterogenee che lo rende lungo, confuso e poco leggibile.

**README leggibile, Markdown essenziale:** Il README è quasi sempre scritto in Markdown (non serve conoscerlo tutto, basta un sottoinsieme essenziale). Elementi fondamentali per la leggibilità sono:

- `# Titolo principale`: identifica chiaramente il progetto.
- `## Sezioni`: struttura logica (Quickstart, Usage, Contributing).
- `### Sottosezioni`: dettagli senza appesantire.

Enfasi testuale:

- `**grassetto**` per concetti chiave e parole guida;
- `*corsivo*` per chiarimenti o termini secondari;
- `‘codice‘` per comandi, file, classi, opzioni CLI.

Regola pratica: se una sezione è lunga più di una schermata, probabilmente va spezzata con un titolo `##`.

```
# UniManager

**UniManager** is a *small teaching project* used to discuss
software design, architecture boundaries, and documentation practices.

## Overview
- Manage courses, exam sessions, enrollments, and grades.
- Keep 'domain/' independent from CLI and persistence.
- Provide a simple baseline for discussing architecture in class.

### Main goals
1. clear architecture boundaries
2. testability-first
3. evolvability over time

## Quickstart
Requirements: JDK 17+, Maven 3.8+

$ git clone https://github.com/<org>/unimanager.git
$ cd unimanager
$ mvn test
$ mvn exec:java -Dexec.mainClass="unimanager.cli.Main"

### Project structure
- 'src/main/java/unimanager/domain'
- 'src/main/java/unimanager/application'
- 'src/main/java/unimanager/interfaces'
```

```

## Usage (CLI)
### Record an exam grade
$ unimanager exam record \
    --student s123 \
    --course INF101 \
    --date 2025-10-10 \
    --room A1 \
    --grade 28

### Help
$ unimanager --help

## Docs
- [Design doc](docs/design/unimanager-architecture.md)
- [ADR index](docs/decisions/README.md)
- [API notes](docs/api/README.md)

## Contributing
- Run 'mvn test' before pushing.
- Keep 'domain/' independent from CLI and persistence.
- Add/update ADR for non-trivial decisions.
- Update README if setup or commands change.
```

**Regola pratica per il README:** Se una persona che entra nel progetto non riesce a:

- clonare il repository;
- eseguire il progetto senza ambiguità;
- ottenere un output minimo e verificabile;

allora il README non sta svolgendo correttamente la funzione di onboarding. L'obiettivo concreto del README è di ridurre al minimo il time-to-first-success, ovvero il tempo necessario per arrivare a un primo risultato funzionante.

**Design document:** È un documento breve (indicativamente 1–3 pagine) che descrive:

- le principali scelte architetturali del sistema;
- le alternative progettuali considerate durante la progettazione;
- i vincoli e i trade-off che hanno guidato le decisioni.

Non è una descrizione esaustiva di ogni classe, metodo o dettaglio implementativo. Il valore principale del design document è di preservare il ragionamento progettuale alla base delle scelte, rendendolo comprensibile anche a chi arriverà sul progetto in un secondo momento.

**Quando serve davvero un design doc?** Un design document è particolarmente utile quando una scelta progettuale è:

- difficile o costosa da modificare in seguito;
- rilevante per più moduli o parti del sistema;
- soggetta a discussioni ricorrenti nel tempo.

Esempi tipici di decisioni da documentare:

- organizzazione dei layer e definizione dei boundary;
- strategie di persistenza (in-memory vs database);

- formati di input e output e loro stabilità nel tempo;
- policy di gestione degli errori ed eccezioni.

**Struttura di un design doc leggero:**

- *Contesto e obiettivi:* descrivono il problema da risolvere e i risultati che il progetto intende ottenere.
- *Non-obiettivi:* chiariscono esplicitamente cosa il design document non intende affrontare o risolvere.
- *Proposta:* presenta la soluzione architetturale adottata, idealmente supportata da una singola figura riassuntiva.
- *Alternative considerate:* elencano le opzioni valutate e i motivi per cui non sono state scelte.
- *Conseguenze:* analizzano vantaggi, svantaggi e rischi introdotti dalla soluzione proposta.
- *Piano di adozione:* descrive come e quando la soluzione viene introdotta, se è prevista una migrazione o un'evoluzione graduale.

**Esempio design doc:**

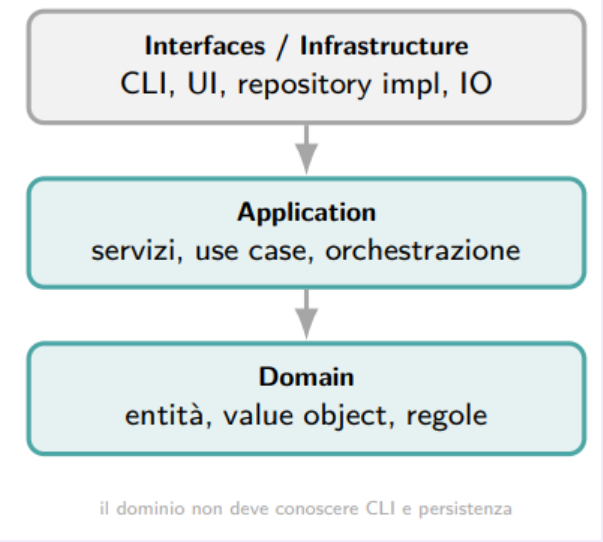
Estratto sintetico di un design document applicato a UniManager.

- *Contesto:* progetto didattico utilizzato per discutere aspetti di design, organizzazione dei servizi applicativi e definizione dei boundary architetturali.
- *Obiettivo principale:* mantenere il dominio applicativo indipendente sia dall'interfaccia utente (CLI) sia dai meccanismi di persistenza.
- *Vincoli progettuali:*
  - **semplicità:** struttura leggibile e comprensibile dagli studenti;
  - **testabilità:** utilizzo di repository in-memory per facilitare i test automatici;
  - **estendibilità:** supporto a interfacce diverse nel tempo (CLI iniziale, GUI o API in una fase successiva).

Proposta architetturale: separazione a livello di responsabilità:

- **Domain:** entità e regole del dominio.
- **Application:** servizi (use case) e orchestrazione.
- **Infrastructure/Interfaces:** repository concreti, CLI/UI.

Principio: dipendenze verso l'interno (dominio stabile).




**UML minimale:** Un diagramma UML inserito in un design document deve essere:

- minimale, con un numero limitato di elementi (circa 6–10);
- leggibile a colpo d'occhio, senza sovraccaricare la vista;
- focalizzato su relazioni, responsabilità e dipendenze principali.

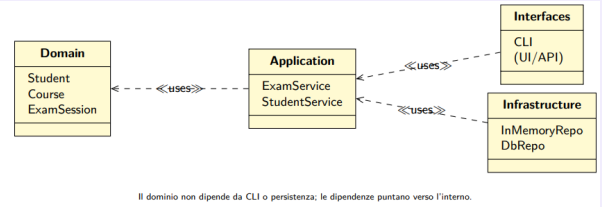
Non è necessario rappresentare:

- ogni campo, getter o dettaglio implementativo;
- aspetti che non contribuiscono a chiarire la struttura complessiva.

**Conmtinuo dell'esempio precedente con UML minimale:**

Rappresentazione essenziale per chiarire:

- boundary tra Domain, Application e Interfaces/Infrastructure;
- dipendenze consentite (verso l'interno).




**Decision log:** Il codice descrive che cosa è stato realizzato e come il sistema funziona oggi. Spesso però non esplicita perché sono state prese certe decisioni:

- quali alternative erano disponibili al momento della scelta?
- quali vincoli (tecnici o organizzativi) hanno guidato la decisione?
- quali conseguenze e compromessi vengono accettati nel tempo?

Il decision log affronta il problema con note brevi, tracciabili e cronologiche, utili per ricostruire il contesto anche a distanza di mesi.

**Formato semplice (stile ADR):** Ogni decisione viene registrata in un file autonomo, che include:

- **ID e data** (per riferimenti e ordine cronologico);
- *decisione* presa, formulata in modo chiaro;
- *contesto* che ha portato alla scelta;
- **alternative considerate** e scartate;
- **conseguenze** accettate (pro e contro).

Un buon ADR dovrebbe essere:

- **compatto** (mezza pagina o una pagina al massimo);
- facilmente **linkabile** da task e revisioni tecniche;
- semplice da **aggiornare** se la decisione evolve.

**Esempio ADR:**

```
ADR-001 (2025-10-01)
Title: Repository interfaces + InMemory implementation

Decision
Define StudentRepository and ExamRepository as interfaces.
Provide an InMemory*Repository implementation for the course.

Context
The system requires testability and a clear separation between
domain/application logic and infrastructure concerns.

Alternatives
A) Use a database immediately (e.g., JDBC / ORM).
B) Use file-based persistence (CSV / JSON).
C) Keep everything in memory.

Consequences
Fast tests, simple setup, clean architectural boundaries.
Not persistent across runs.
Future work: introduce a database adapter.
```

**Regola pratica:** mettere l'informazione dove serve a chi la userà. Per questo è bene usare una gerarchia (dal più “vicino al codice” al più “panoramico”):

- **Javadoc** accanto a classi e metodi: descrive contratti e comportamento osservabile delle API.
- **README** alla radice del repository: è il punto di ingresso (quickstart, usage, contributing).
- **Design doc** in /docs/design/: chiarisce architettura, vincoli e trade-off.

- **Decision log (ADR)** in /docs/decisions/ADR-001.md, ADR-002.md,…: conserva la memoria storica delle decisioni non banali.

**Documentazione del codice, Javadoc mirata:** Documentare il codice non significa commentare ogni riga né spiegare ciò che è già evidente dalla lettura. Una Javadoc realmente utile si concentra su:

- **uso:** come e quando chiamare l'API correttamente;
- **contratto:** che cosa il metodo o la classe garantisce;
- **effetti:** side effects rilevanti e interazioni con la persistenza;
- **casi di errore:** condizioni anomale ed eccezioni significative.

L'obiettivo pratico è quello di comprendere il comportamento dell'API senza dover aprire o analizzare l'implementazione.

**Priorità nella documentazione:**

1. **API pubbliche e stabili:** punti di accesso pensati per essere usati da altri moduli o da codice esterno.
2. **Classi con responsabilità non banale:** componenti che incarnano regole di dominio, coordinano più oggetti o svolgono ruoli centrali nel sistema.
3. **Metodi che presentano almeno una delle seguenti caratteristiche:**
   - effetti collaterali importanti (persistenza, I/O, modifiche di stato);
   - invarianti o precondizioni che devono essere rispettate;
   - eccezioni significative che il chiamante deve gestire;
   - regole di business non ovvie o non immediatamente deducibili dal nome.

**Cosa evitare:**

- Commenti che ripetono il codice senza aggiungere informazione, ad esempio spiegazioni ovvie come “incrementa i” sopra i++.
- Commenti di tipo procedurale, che descrivono ogni singolo passo quindi che diventano rapidamente falsi o fuorvianti non appena l'implementazione cambia.
- La “bugia della documentazione”, un commento non aggiornato è un bug informativo e per questo induce chi legge a fare scelte sbagliate o a perdere tempo. La Javadoc deve quindi privilegiare i contratti (più stabili), non i dettagli interni (più volatili).
- Javadoc eccessivamente lunga o verbosa. Se entra nei dettagli interni del codice, smette di descrivere un contratto.

### Javadoc su ExamService (mirata all'uso)

```java
package unimanager.service;

import unimanager.domain.ExamDescriptor;

/**
 * Application service for exam registration and grade recording.
 * <p>
 * This service implements a single use case: ensure a student exists,
 * register them to an exam session, and optionally record a passing grade.
 * </p>
 */
public class ExamService {

    // repositories injected (interfaces)

    /**
     * Registers a student to the exam described by {@code exam} and records
     * the grade if it is passing.
     *
     * @param studentId unique student identifier (non-null, non-empty)
     * @param studentName student name (required if the student does not exist yet)
     * @param exam exam descriptor (non-null)
     * @param grade grade to record; passing threshold is 18
     * @return a human-readable outcome message for the caller (CLI/UI)
     * @throws IllegalArgumentException if input parameters are invalid
     */
    public String registerAndRecord(String studentId, String studentName, ExamDescriptor exam, int grade) {

        // implementation
        return "OK";
    }
}
```

La documentazione dell'API deve rendere esplicito che il servizio:

- ritorna un esito (valore o risultato strutturato),
- non si occupa della presentazione dell'output.

La Javadoc dovrebbe chiarire:

- che cosa viene restituito e con quale significato;
- perché la formattazione e la stampa sono responsabilità della CLI/UI.

Benefici progettuali:

- separazione chiara tra logica applicativa e interfaccia;
- maggiore testabilità (nessun output nel mezzo del caso d'uso);
- riuso della stessa logica in contesti diversi: CLI oggi, GUI o API domani.

**Come si integra nel ciclo di vita del progetto:**

- **README:** viene aggiornato quando cambiano il setup, i comandi di esecuzione o le modalità di utilizzo.
- **Design document:** viene aggiornato quando cambiano i boundary, l'architettura complessiva o i vincoli principali.
- **Decision log:** viene aggiornato ogni volta che viene presa una decisione non banale e difficilmente reversibile.
- **Javadoc:** viene aggiornata quando cambia il contratto delle API pubbliche o il loro comportamento osservabile.

Tutti questi elementi non sono documentazione “aggiuntiva” o burocratica, ma fanno parte integrante del processo di sviluppo del software.

**Definition of Done della documentazione:** Una feature non è davvero completata se il codice cambia ma la documentazione resta indietro. Regola operativa per ogni modifica rilevante:

- se cambia setup/uso → aggiornare **README**;
- se cambiano confini o scelte architetturali → aggiornare **design doc**;
- se c'è una decisione non banale → aggiungere/aggiornare **ADR**;
- se cambia il contratto API → aggiornare **Javadoc**.

DoD minima: codice, test e documentazione coerenti nello stesso merge.

**Ownership, chi mantiene cosa:**

- **README:** ownership del team, revisione obbligatoria quando cambiano uso/setup.
- **Design doc:** ownership dei responsabili architetturali del modulo.
- *ADR:* ownership di chi propone la decisione, con review tecnica.
- **Javadoc:** ownership di chi modifica API pubbliche e servizi applicativi.

Principio: ogni documento deve avere un proprietario esplicito, non “di nessuno”.

**Errori frequenti in fase di consegna:**

- README con quickstart non riproducibile (comandi mancanti o versioni assenti).
- Design doc descrittivo ma senza alternative e trade-off.
- ADR che registra solo la decisione finale senza contesto.
- Javadoc che copia l'implementazione invece di esplicitare il contratto.
- Documenti aggiornati in momenti diversi: codice e testo divergono.

> **Riepilogo:**
>
> - La documentazione essenziale riduce attrito operativo: aiuta onboarding, manutenzione e passaggi di consegne.
> - README, design document, decision log e Javadoc rispondono a bisogni diversi e complementari di comunicazione tecnica.
> - Il valore della documentazione dipende dalla sua utilità pratica e dall'allineamento con il codice, non dalla quantità di pagine.
> - README e quickstart supportano l'uso del progetto; design document e decision log spiegano perché il sistema è fatto così.
> - La Javadoc è più utile quando chiarisce contratto e comportamento osservabile delle API, non i dettagli interni.
> - Tenere la documentazione aggiornata è parte del lavoro di sviluppo e prepara meglio testing, CI e manutenzione.

## Testing: principi e piramide del testing

Una convinzione molto diffusa nello sviluppo software è: "Prima scrivo il codice e alla fine faccio un test per controllare se funziona". In un progetto reale questo approccio tende a produrre effetti sistematici:

- il feedback arriva tardi, quando il codice è già cambiato più volte e il contesto iniziale si è perso;
- gli errori diventano più difficili da isolare, perché entrano in gioco più componenti e più ipotesi non esplicitate;
- la correzione richiede più tempo, perché si deve ricostruire a posteriori cosa ci si aspettava davvero.

Il risultato più dannoso spesso non è il bug in sé, ma la conseguenza organizzativa:

- ogni modifica viene percepita come rischiosa,
- il refactoring viene rimandato,
- si sviluppa una reale paura di cambiare il codice.

Nel contesto dello sviluppo moderno, il testing non può più essere considerato un semplice “controllo qualità finale” eseguito a valle dell'implementazione. Il testing diventa invece una pratica continua, integrata nel processo di sviluppo, che accompagna le scelte progettuali e contribuisce a orientarle nel tempo. Un insieme ben costruito di test automatici svolge il ruolo di rete di sicurezza:

- riduce il rischio associato alle modifiche e agli interventi correttivi;
- rende praticabili refactoring frequenti e una progressiva pulizia del design;
- facilita il lavoro collaborativo, perché rende esplicito il comportamento atteso del sistema.

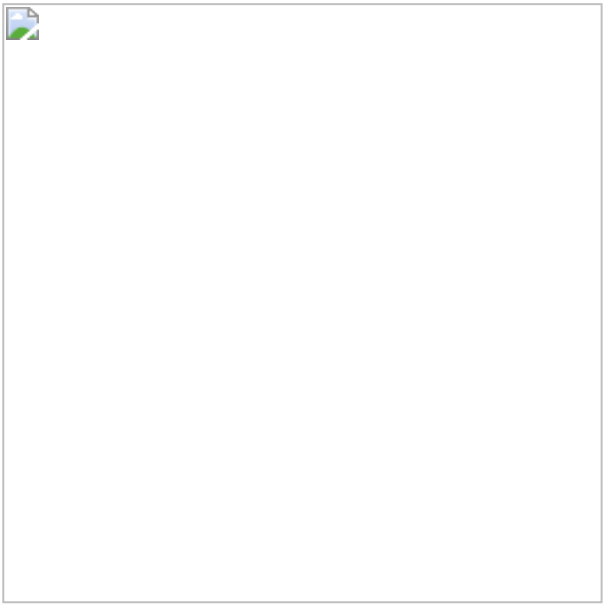
In assenza di test automatici, il software tende progressivamente a irrigidirsi: ogni cambiamento diventa una scommessa e l'evoluzione del sistema rallenta.

**Due obiettivi del testing:** I test possono essere letti attraverso due obiettivi principali, entrambi necessari:

- *Dimostrare che il sistema soddisfa i requisiti (validazione):* verificare che i comportamenti richiesti siano presenti e coerenti con quanto promesso.
- *Scoprire difetti e comportamenti indesiderati (defect testing):* far emergere errori, incoerenze e casi non gestiti prima che arrivino in produzione.

Non sono due mondi separati: molto spesso un test nato per validare un requisito finisce per rivelare un difetto nascosto, oppure un caso ambiguo nella specifica. Cambia però lo “spirito” con cui si scelgono i casi di test:

- test con input realistici e attesi, che rappresentano l'uso normale del sistema;
- test con input anomali o ai limiti, che esplorano condizioni rare, errori e vincoli.



**Perché oggi le regressioni sono inevitabili?** Un sistema software reale non è statico: cambia continuamente nel tempo, per motivi diversi:

- nuove funzionalità e miglioramenti richiesti dagli utenti,
- correzioni di bug e interventi di manutenzione,
- refactoring per mantenere il design leggibile e sostenibile,
- aggiornamenti di librerie, framework e dipendenze esterne.

Ogni cambiamento, anche piccolo, può rompere un comportamento che prima era corretto: questa situazione prende il nome di **regressione**. Il punto centrale non è chiedersi “se succederà”, ma accettare che “prima o poi succederà”, perché la complessità cresce e le interazioni tra componenti non sono sempre prevedibili. La difesa più efficace è una suite di test automatica:

- ripetibile,
- eseguita spesso,
- capace di segnalare rapidamente quando un comportamento atteso non è più garantito.

**Manuale vs automatico:** Il testing manuale rimane necessario in alcune situazioni:

- valutazione dell'esperienza d'uso e dell'interfaccia grafica,
- flussi complessi che coinvolgono più passaggi e decisioni,
- verifiche che richiedono giudizio umano o interpretazione contestuale.

Tuttavia, il testing manuale non scala bene: richiede tempo, è costoso da ripetere e tende a essere applicato solo a una parte dei casi possibili.

Il testing automatico costituisce la base della verifica delle regressioni:

- è ripetibile in modo affidabile,
- è veloce da eseguire,
- può essere lanciato ad ogni modifica del codice senza costi aggiuntivi significativi.

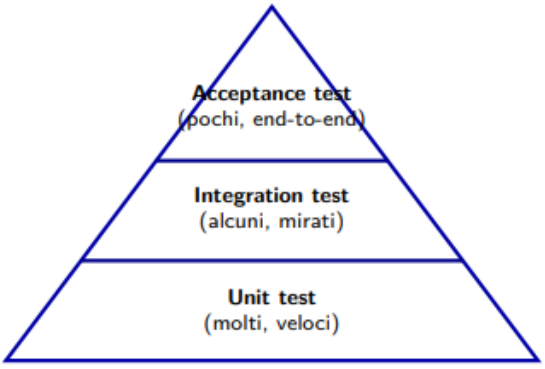
L'obiettivo pratico non è scegliere uno solo dei due approcci, ma automatizzare tutto ciò che è stabile e verificabile, lasciando al testing manuale gli aspetti che richiedono valutazioni qualitative.

**La piramide del testing:** La piramide del testing è un modello semplice per distribuire bene i test:

- molti test piccoli e veloci alla base,
- alcuni test di collaborazione a metà,
- pochi test end-to-end in cima.

L'idea non è “fare tanti test a caso”, ma ottenere:

- feedback veloce,
- diagnosi semplice,
- fiducia sul comportamento complessivo.




**Unit test:** Gli unit test verificano una singola unità di comportamento del sistema, considerata in modo isolato rispetto al resto dell'applicazione. Tipicamente si concentrano su:

- un metodo specifico,
- una classe con una responsabilità ben definita,
- una singola regola di business o di validazione.

Per essere davvero efficaci, gli unit test condividono alcune caratteristiche fondamentali:

- sono **rapidi**, perché si eseguono in pochi millisecondi e possono essere lanciati molto spesso;
- sono **isolati**, cioè non dipendono da database, rete, filesystem o tempo reale;
- sono **deterministici**, nel senso che a parità di input producono sempre lo stesso risultato.

Queste proprietà rendono gli unit test la base della piramide del testing, perché permettono un feedback continuo e affidabile durante lo sviluppo quotidiano.

I candidati migliori per gli unit test sono le regole di business, perché rappresentano il cuore logico del sistema e concentrano molte decisioni importanti. In particolare, vale la pena testare con attenzione:

- validazioni di input e di stato,
- vincoli che devono essere sempre rispettati,
- calcoli e trasformazioni dei dati,
- transizioni di stato tra fasi diverse del ciclo di vita di un oggetto.

Questi test sono in genere facili da scrivere e offrono un grande valore, perché intercettano regressioni “costose”, che altrimenti emergerebbero solo molto più avanti. Un buon unit test è anche leggibile:

- descrive chiaramente cosa deve succedere in una certa situazione,
- evita di raccontare come il codice realizza internamente quel comportamento.

**Integration test:** Gli integration test verificano la collaborazione tra più componenti del sistema, osservando come parti diverse funzionano insieme in uno scenario realistico. Tipicamente riguardano situazioni come:

- servizi applicativi che interagiscono con repository o componenti di persistenza;
- flussi che attraversano più layer dell'architettura (ad esempio dominio, applicazione, infrastruttura);
- integrazioni con risorse reali o semi-reali, come database, file system o servizi esterni.

Rispetto agli unit test, questi test sono più costosi da gestire perché richiedono:

- un setup più articolato dell'ambiente di esecuzione,
- la presenza di dipendenze esterne correttamente configurate,
- dati coerenti e consistenti tra i diversi componenti coinvolti.

Per questo motivo gli integration test non dovrebbero essere numerosi, ma selezionati con attenzione e mantenuti pochi e mirati sui punti di collaborazione più critici.

Gli integration test risultano particolarmente utili in quei punti del sistema in cui il comportamento corretto dipende dall'interazione tra più componenti. In pratica, i casi più critici sono spesso legati a:

- il mapping tra oggetti del dominio e il livello di persistenza;
- la serializzazione e il parsing dei dati, insieme alla gestione dei formati;
- le transazioni e la garanzia di consistenza dei dati tra operazioni successive;
- le configurazioni dell'applicazione e il corretto wiring delle dipendenze.

In questi punti gli unit test non sono sufficienti, perché l'errore non nasce da una singola unità, ma dal modo in cui più parti del sistema collaborano. Un integration test ben scelto e mirato su uno di questi punti critici vale spesso più di molti test generici che coprono solo casi superficiali.

**Acceptance test:** Gli acceptance test verificano il comportamento del sistema dal punto di vista di chi lo utilizza, senza entrare nei dettagli interni dell'implementazione. Si basano su descrizioni ad alto livello, come:

- scenari completi di utilizzo,
- user stories rappresentative,
- criteri di accettazione che definiscono quando una funzionalità può dirsi soddisfacente.

Il loro valore principale è la validazione funzionale: permettono di verificare che il sistema, nel suo insieme, faccia davvero ciò che serve rispetto alle esigenze espresse. Allo stesso tempo presentano alcune limitazioni strutturali:

- sono più lenti da eseguire rispetto agli altri tipi di test,
- risultano più fragili, perché sensibili a cambiamenti anche minimi,
- rendono più complessa l'analisi delle cause quando qualcosa va storto.

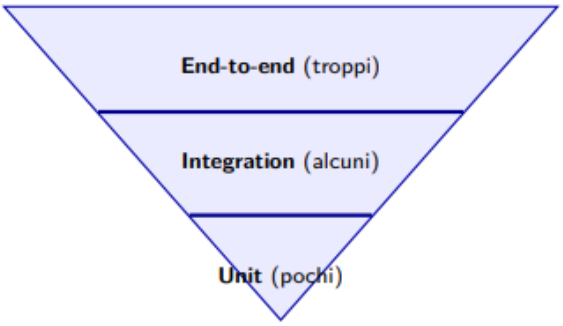
Per questi motivi gli acceptance test sono collocati in cima alla piramide: sono pochi, selezionati con cura e rappresentano solo i flussi principali e più significativi del sistema.

**Un anti-pattern, la piramide rovesciata:** Errore comune: basarsi quasi solo su test end-to-end. Effetti tipici:

- suite lenta,
- fallimenti difficili da interpretare,

- test che si rompono per cambiamenti minimi,
- persone che smettono di eseguire i test.

Questa forma viene spesso chiamata “ice-cream cone”.




**Criteri pratici per una suite sana:** Una suite di test “sana” tende a mostrare alcune proprietà ricorrenti:

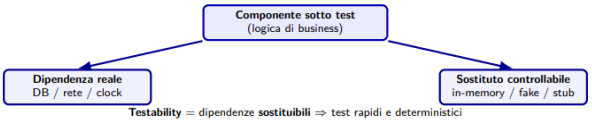
- è abbastanza veloce da poter essere eseguita spesso, idealmente durante lo sviluppo quotidiano;
- quando fallisce, produce errori comprensibili e facili da collegare alla causa;
- fallisce in modo deterministico, cioè non “a caso” e non dipendendo dall'ambiente;
- copre soprattutto le regole di business e i flussi principali, cioè ciò che dà valore al sistema.

Non esiste una percentuale magica o una ricetta universale: l'obiettivo pratico è ottenere feedback continuo e mantenere alta la fiducia mentre il software evolve. Se la suite diventa lenta, la conseguenza è quasi sempre la stessa: viene eseguita sempre meno, fino a essere ignorata nei momenti in cui servirebbe di più.

**Testability:** Non tutto il codice è ugualmente facile da testare: alcune scelte progettuali rendono la verifica semplice, altre la complicano inutilmente. Dire che un codice è “testabile” significa, in pratica, che è possibile:

- verificare il comportamento di una parte del sistema senza dover costruire o avviare mezzo sistema intorno;
- controllare l'ambiente del test, incluse dipendenze esterne, dati di input e fattori come il tempo;
- ottenere test piccoli, veloci da eseguire e stabili nel tempo.

La testabilità è quindi un indicatore importante della qualità del design: spesso coincide con alta coesione, basso accoppiamento e con una struttura che favorisce l'evoluzione del software.




**Segnali di bassa testability:** Alcune caratteristiche del codice rendono la scrittura dei test più difficile del necessario e sono spesso segnali di problemi di design. Tra i segnali più comuni rientrano:

- dipendenze nascoste, come la creazione interna di client per database, servizi HTTP o altre risorse esterne;
- side effect diffusi, ad esempio scritture su file o rete mescolate direttamente alla logica di business;
- classi troppo grandi, che accumulano responsabilità diverse e diventano difficili da comprendere e isolare.

Un'altra fonte frequente di problemi è l'uso diretto di elementi globali o non controllabili:

- tempo di sistema,
- generatori di numeri casuali,
- singleton o stati globali condivisi.

Un sintomo pratico molto chiaro è questo: per testare anche una singola regola di business diventa necessario costruire un setup enorme e complesso.

**SRP:** Il principio di **Single Responsibility** (SRP) afferma che una classe dovrebbe avere una sola responsabilità, cioè un solo motivo valido per cambiare. Quando questo principio è rispettato, anche i test diventano più naturali e diretti:

- ogni test verifica un comportamento preciso e ben delimitato;
- il setup necessario è minimo, perché ci sono poche dipendenze coinvolte;
- in caso di errore, la causa è più facile da individuare e isolare.

Al contrario, classi “tuttofare”, che mescolano logica diversa, costringono a scrivere test grandi, complessi e fragili, difficili da capire e costosi da mantenere. Una regola empirica utile per riconoscere il problema è la seguente: se un singolo test deve controllare troppe cose insieme, molto probabilmente la classe sotto test fa troppe cose.

**DIP:** Il **Dependency Inversion Principle (DIP)** afferma che il codice dovrebbe dipendere da astrazioni e non da dettagli concreti di implementazione. Questo principio ha un impatto diretto sulla testabilità, perché rende possibile sostituire facilmente le dipendenze esterne durante i test. In pratica, nei test diventa naturale usare alternative controllabili:

- un repository in memoria al posto di un database reale,
- un clock controllabile invece del tempo di sistema,
- un gateway finto o simulato invece di una comunicazione di rete.

Quando le dipendenze sono progettate per essere sostituibili, i test tornano ad essere rapidi, deterministici e focalizzati sul comportamento, senza effetti collaterali indesiderati.

**UniManager: due casi ideali da testare:**

All'interno di UniManager sono presenti alcune regole di business particolarmente chiare e ben delimitate, che si prestano molto bene alla scrittura di unit test. Due esempi rappresentativi sono:

- la logica di iscrizione a un appello d'esame,
- la validazione di un voto inserito dal docente.

Questi casi sono buoni esempi dal punto di vista didattico perché:

- presentano input e output ben definiti, facilmente verificabili;
- includono diversi casi limite e condizioni di errore interessanti;
- non dovrebbero dipendere direttamente dall'interfaccia utente o dal database per essere validate correttamente.

**Iscrizione a un appello:** Scenario di riferimento: un utente con ruolo di studente tenta di iscriversi a un appello d'esame disponibile nel sistema. Questo scenario è governato da alcune regole di business tipiche, realistiche e facilmente comprensibili anche a fini didattici:

- lo studente deve risultare regolarmente iscritto al corso associato all'appello;
- non è consentita una doppia iscrizione allo stesso appello;
- l'iscrizione non è possibile oltre una certa data di scadenza;
- l'appello può prevedere un numero massimo di posti disponibili.

In un contesto del genere, gli unit test risultano particolarmente efficaci perché permettono di verificare in modo diretto:

- i casi validi, in cui l'iscrizione deve andare a buon fine;
- i casi invalidi, in cui il sistema deve impedire l'operazione;
- la coerenza dei messaggi di errore o delle eccezioni prodotte.

**Validazione del voto:** Scenario di riferimento: un utente con ruolo di docente registra il voto di uno studente al termine di un appello d'esame. Anche in questo caso il comportamento del sistema è regolato da un insieme di regole di business ben definite:

- il voto deve rientrare nell'intervallo previsto dal regolamento (ad esempio 18–30), con l'eventuale gestione della lode;
- il voto può essere registrato solo per studenti effettivamente iscritti all'appello;
- non è consentito registrare o modificare un voto se il verbale dell'appello risulta chiuso;
- il processo può prevedere stati distinti, come “inserito”, “pubblicato” e “verbalizzato”.

Anche in questo scenario il nucleo logico è facilmente testabile senza coinvolgere infrastrutture esterne: la validazione del voto dovrebbe risiedere in un punto chiaro e ben isolato del dominio, rendendo possibile la verifica tramite unit test semplici e mirati.

**Come scegliere casi di test:** La scelta dei casi di test non è casuale, ma segue un ragionamento preciso. Un approccio semplice ed efficace consiste nel considerare:

- un caso “tipico”, che rappresenta l'uso normale e che deve funzionare senza problemi;
- casi ai limiti, come inizio e fine di un intervallo, scadenze temporali, valori massimi e minimi ammessi;
- casi di errore, legati a input non validi, stati non consentiti o tentativi di operazioni duplicate.

L'obiettivo non è testare ogni possibile combinazione, ma concentrare gli sforzi su ciò che statisticamente tende a rompersi:

- i confini delle regole,
- le condizioni eccezionali,
- le combinazioni meno ovvie ma più rischiose.

**Test-first:** L'approccio test-first non va inteso come l'adozione rigida di un rituale formale o di una metodologia prescrittiva. In pratica, significa usare i test come strumento per chiarire il comportamento atteso prima di scrivere l'implementazione:

- cosa deve succedere in condizioni corrette;
- cosa non deve succedere mai;
- quali situazioni di errore devono essere intercettate e gestite.

L'effetto pratico di questo modo di lavorare è spesso immediato:

- interfacce e API più pulite e coerenti;
- meno ambiguità nelle responsabilità del codice;
- implementazioni più semplici, perché guidate dal comportamento atteso.

Un modo pragmatico e leggero di applicare il test-first può essere visto come un micro-processo iterativo:

1. si sceglie una regola di business piccola e ben delimitata;
2. si scrive un test che ne descrive il comportamento atteso;
3. si esegue il test e si osserva il fallimento, che è il risultato previsto (Red);
4. si implementa il minimo indispensabile per far passare il test (Green);
5. si refactorizza il codice mantenendo i test sempre verdi (Refactor).

Il valore principale non è semplicemente “scrivere il test prima del codice”, ma il fatto che questo approccio costringe a ragionare sull'uso, sul comportamento e sui confini della regola prima di concentrarsi sull'implementazione.

**Takeaway, come leggere la piramide:** La piramide non è una moda: è un modo pratico per bilanciare costo e feedback. Salendo nella piramide, in generale:

- il **costo** aumenta (setup, dipendenze, manutenzione),
- la **velocità** diminuisce (da millisecondi a secondi/minuti),
- l'**isolamento** diminuisce (da unità isolate a flussi end-to-end).

Obiettivo operativo: tanti unit test per feedback veloce, integration test mirati sui punti critici, pochi acceptance sui flussi principali.




**Riepilogo:**

- Il testing è parte del processo di progettazione: rende esplicito il comportamento atteso e riduce il rischio delle modifiche.
- La piramide del testing aiuta a bilanciare velocità di feedback, costo e granularità dei controlli automatici.
- Unit, integration e acceptance test rispondono a domande diverse e vanno combinati, non sostituiti l'uno con l'altro.
- La testabilità dipende fortemente dal design: buon isolamento e dipendenze sane rendono i test più semplici e utili.

- La rete di test automatici è ciò che rende più praticabili refactoring ed evoluzione continua del sistema.
- Il test-first è utile soprattutto perché costringe a chiarire comportamento e confini prima dell'implementazione.

## JUnit 5: test

Nel ciclo di sviluppo moderno, implementazione e verifica non sono fasi separate, ma attività che procedono in modo intrecciato. Il codice viene scritto insieme ai test che ne descrivono il comportamento atteso, rendendo esplicite ipotesi e decisioni che altrimenti rimarrebbero implicite. L'esecuzione frequente dei test fornisce un feedback immediato sugli effetti delle modifiche introdotte. Questo feedback consente di individuare rapidamente errori e regressioni, prima che il contesto diventi troppo complesso o difficile da ricostruire. Nel tempo, la presenza dei test influenza il design del sistema, favorendo soluzioni più semplici, modulari e facilmente evolvibili.

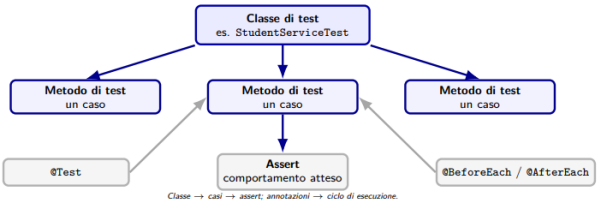
**JUnit 5:** è il framework di riferimento per la scrittura ed esecuzione di test automatici in Java, progettato per supportare in modo esplicito il testing di unità. Fornisce un insieme di annotazioni, asserzioni e meccanismi di esecuzione che permettono di descrivere in modo preciso il comportamento atteso del codice. L'obiettivo principale non è “verificare che il codice funzioni” in senso generico, ma rendere eseguibili e verificabili le regole e le ipotesi su cui il sistema è costruito. L'integrazione nativa con gli ambienti di sviluppo consente l'esecuzione frequente dei test come parte naturale del lavoro quotidiano. In questo senso, JUnit 5 rappresenta lo strumento operativo che rende praticabili i principi di testing discussi finora.

L'obiettivo non è esplorare tutte le funzionalità del framework, ma mostrare come tradurre in pratica i concetti visti nelle lezioni precedenti. L'attenzione sarà quindi concentrata su un sottoinsieme essenziale:

- struttura di una classe di test,
- gestione delle fixture,
- uso consapevole degli assert,
- leggibilità e naming dei test.

Questo è sufficiente per scrivere test di unità chiari, efficaci e coerenti con gli obiettivi del corso.

**Struttura di base di un test JUnit:** Un test JUnit è organizzato in una **classe di test**, separata dal codice di produzione. Ogni **metodo di test** rappresenta un singolo caso o una singola regola da verificare. Le annotazioni definiscono ruolo e momento di esecuzione dei metodi (*@Test, @BeforeEach, @AfterEach*). La struttura favorisce test piccoli, leggibili e indipendenti, con risultati facilmente interpretabili.




**La classe di test:** Una classe di test è una normale classe Java, collocata nel codice di test e dedicata esclusivamente alla verifica del comportamento. Per convenzione, il nome della classe di test riflette quello della classe sotto test, rendendo immediato il collegamento tra codice di produzione e verifica:

- *StudentServiceTest*
- *ExamServiceTest*

Ogni classe di test si concentra su una singola classe reale del sistema, evitando di mescolare responsabilità e comportamenti non correlati. Questa corrispondenza uno-a-uno favorisce la leggibilità, semplifica la navigazione del progetto e rende più chiaro cosa viene verificato e dove.

**Il metodo di test:** Un metodo di test è un normale metodo Java, annotato con @Test, che descrive un singolo comportamento atteso. Ogni metodo dovrebbe concentrarsi su una sola regola o condizione, evitando di mescolare più verifiche non correlate. Un buon metodo di test è tipicamente:

- piccolo, perché verifica un solo aspetto;
- focalizzato, perché ha uno scopo chiaro;
- indipendente, perché non dipende da altri test.

fail: test rosso → correggere il codice

```java
@Test
void shouldNotEnrollStudentTwice() {
    examService.enroll(student, exam);

    try {
        examService.enroll(student, exam);
        fail("Second enrollment should not be allowed");
    } catch (IllegalStateException e) {
        // expected behavior
    }
}
```

**Fixture:** Una fixture rappresenta lo stato iniziale in cui un test viene eseguito, costruito in modo controllato e intenzionale. Il suo scopo principale è garantire che ogni test parta da condizioni note e ben definite. L'uso di fixture permette di ottenere test che risultano:

- **puliti**, perché non dipendono da esecuzioni precedenti;
- **ripetibili**, perché producono sempre lo stesso risultato a parità di condizioni;
- **prevedibili**, perché falliscono solo quando il comportamento è realmente errato.

In assenza di una fixture esplicita, i test tendono a dipendere implicitamente dallo stato lasciato da altri test, diventando fragili e difficili da interpretare.

**@BeforeEach:** @BeforeEach identifica un metodo eseguito automaticamente prima di ogni metodo di test. Il suo ruolo è costruire la fixture, inizializzando lo stato necessario all'esecuzione dei test. In questo modo ogni test parte da condizioni identiche, senza dipendere dall'ordine di esecuzione o da altri test.

```java
class ExamServiceTest {

    private ExamService examService;
    private Student student;
    private Exam exam;

    @BeforeEach
    void setUp() {
        examService = new ExamService();
        student = new Student("12345");
        exam = new Exam("Software Engineering");
    }
}
```

**@AfterEach:** @AfterEach identifica un metodo eseguito automaticamente dopo ogni metodo di test. Serve soprattutto a ripristinare lo stato esterno modificato dal test, quando questo è necessario. Esempi tipici sono:

- cancellare file temporanei creati dal test;
- ripulire directory di lavoro;
- riportare il database di test a uno stato noto.

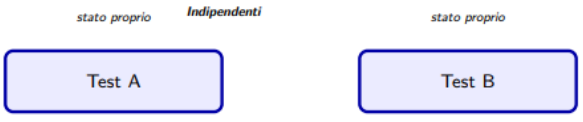
Nei test di unità semplici, il suo utilizzo è spesso limitato, perché la fixture viene ricostruita da zero per ogni test.

```java
class ImportJobTest {

    private Path tempCsv;
    private TestDatabase db;

    @AfterEach
    void tearDown() throws IOException {
        Files.deleteIfExists(tempCsv); // cleanup file temporaneo
        db.reset();                    // reset stato DB di test
    }
}
```

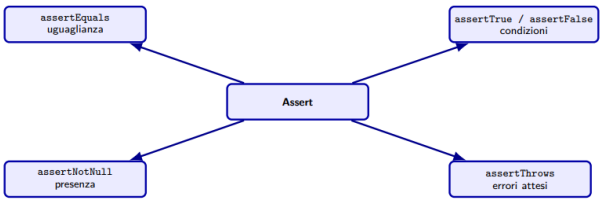
**Perché i test devono essere indipendenti?** L'ordine di esecuzione dei test non è garantito e non deve influenzarne il risultato. Ogni test dovrebbe costruire e usare il proprio stato, senza dipendere da effetti prodotti da altri test. Un test ben progettato fallisce solo quando il comportamento che verifica è effettivamente errato.




**Assert:** Un **assert** esprime in forma eseguibile una proprietà che deve risultare vera al termine dell'azione verificata dal test. La sua funzione non è “controllare un valore a caso”, ma rendere esplicito il

comportamento atteso del sistema in una certa condizione. Un test è quindi, in larga parte, una sequenza di scelte che portano a un punto in cui l'assert stabilisce cosa costituisce correttezza e cosa costituisce errore. Quando un test fallisce, la qualità dell'assert determina la qualità del feedback: un buon assert rende il fallimento comprensibile e direttamente collegabile alla causa.

JUnit mette a disposizione diverse famiglie di assert, ciascuna pensata per esprimere un tipo specifico di proprietà. La scelta dell'assert corretto contribuisce in modo diretto alla leggibilità del test e alla chiarezza del fallimento.




**assertEquals:** assertEquals verifica che il valore prodotto dal sistema coincida con il valore atteso in una determinata condizione. Un assert di questo tipo va letto come una frase dichiarativa: “dato questo scenario, il risultato deve essere questo”. Quando è ben scelto, assertEquals rende il test immediatamente comprensibile e facilita l'interpretazione dei fallimenti.

**Esempio 1**

```
@Test
void totalPriceShouldIncludeAllItems() {
    Order order = new Order();
    order.addItem(new Item(10));
    order.addItem(new Item(15));
    assertEquals(25, order.totalPrice());
}
```

**Esempio 2**

```
@Test
void shouldStartWithZeroEnrollments() {
    Exam exam = new Exam("SE");
    assertEquals(0, exam.enrollmentCount());
}
```

**assertTrue:** assertTrue verifica che una condizione booleana sia vera. È utile quando si vuole controllare un vincolo espresso naturalmente come predicato (*isOpen(), isValid(), contains(...)*).

**Esempio 1**

```
@Test
void examShouldBeOpenForEnrollment() {
    Exam exam = new Exam("SE");
    exam.openEnrollment();
    assertTrue(exam.isEnrollmentOpen());
}
```

**Esempio 2**

```
@Test
void enrolledListShouldContainStudent() {
    examService.enroll(student, exam);
    assertTrue(exam.enrolledStudents()
        .contains(student));
}
```

**assertFalse:** assertFalse verifica che una condizione booleana sia falsa. È il complemento naturale di assertTrue per esprimere in modo esplicito ciò che il sistema non deve permettere.

**Esempio 1**

```
@Test
void examShouldNotBeOpenAfterClosing() {
    Exam exam = new Exam("SE");
    exam.openEnrollment();
    exam.closeEnrollment();
    assertFalse(exam.isEnrollmentOpen());
}
```

**Esempio 2**

```
@Test
void shouldRejectExpiredEnrollmentWindow() {
    exam.setEnrollmentDeadline(yesterday);
    assertFalse(exam.canEnroll(today));
}
```

**assertNotNull:** assertNotNull verifica che un oggetto sia stato creato o restituito correttamente. È utile per controllare precondizioni minime prima di assert più specifici.

**Esempio 1**

```
@Test
void shouldCreateStudentWithValidId() {
    Student student = studentService.register("12345");
    assertNotNull(student);
}
```

**Esempio 2**

```
@Test
void shouldFindExamByCode() {
    Exam exam = examRepository.findByCode("SE-2026");
    assertNotNull(exam);
}
```

**assertThrows:** Le eccezioni non rappresentano solo errori tecnici, ma spesso esprimono regole di business violate. Testare che un'eccezione venga sollevata significa verificare che il sistema impedisca comportamenti non consentiti. In questo senso, un test può specificare in modo esplicito anche ciò che il sistema non deve permettere. Nella chiamata a assertThrows, `() -> ...` è una **funzione lambda**: rappresenta il blocco di codice che ci aspettiamo lanci l'eccezione (una funzione anonima senza argomenti).

**Esempio 1**

```
@Test
void shouldNotEnrollStudentTwice() {
    examService.enroll(student, exam);
    assertThrows(IllegalStateException.class,
        () -> examService.enroll(student, exam));
}
```

**Esempio 2**

```
@Test
void shouldRejectInvalidGrade() {
    assertThrows(IllegalArgumentException.class,
        () -> examService.recordGrade(student, 31));
}
```

**Framework di testing in altri linguaggi:** JUnit 5 è il riferimento in Java, ma il concetto è trasversale: quasi ogni linguaggio ha framework analoghi per unit test, fixture e assert. Esempi comuni sono:

- Python: pytest, unittest;
- C++: GoogleTest, Catch2;
- JavaScript/TypeScript: Jest, Vitest.

Cambia la sintassi, ma resta uguale l'obiettivo: verificare in modo automatico il comportamento atteso.

**Python: esempio con pytest:**

```python
import pytest

def test_should_not_enroll_student_twice(exam_service, student, exam):
    exam_service.enroll(student, exam)

    with pytest.raises(ValueError):
        exam_service.enroll(student, exam)
```

- `pytest.raises(...)` è l'analogo pratico di `assertThrows(...)`.

### C++: esempio con GoogleTest:

```cpp
#include <gtest/gtest.h>

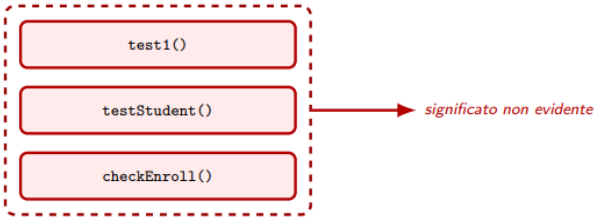
TEST(ExamServiceTest, ShouldNotEnrollStudentTwice) {
    ExamService service;
    service.enroll("s1", "se-2026");

    EXPECT_THROW(service.enroll("s1", "se-2026"),
                 std::logic_error);
}
```

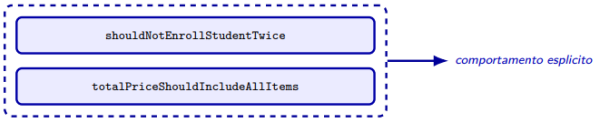
- `TEST(...)` è una **macro** di GoogleTest che definisce un caso di test.
- `EXPECT_THROW(...)` esprime l'attesa di eccezione in stile C++.

**I test sono documentazione:** I test automatici vengono spesso letti più frequentemente del codice di produzione, soprattutto quando si deve capire come il sistema dovrebbe comportarsi. Un buon test non si limita a verificare un risultato, ma racconta una storia: descrive uno scenario, un'azione e un comportamento atteso. In questo contesto, il nome del test diventa parte integrante della documentazione del sistema, perché fornisce una descrizione sintetica e verificabile del comportamento.

**Naming sbagliato:** Alcuni nomi di test sono formalmente corretti, ma non comunicano quale comportamento venga verificato. Il risultato è che, quando un test fallisce, diventa necessario aprire e leggere il codice per capirne il senso.




**Naming corretto:** Un buon nome di test descrive in modo sintetico lo scenario e il comportamento atteso. Questo rende la suite più leggibile e riduce la necessità di aprire il codice per capire cosa viene verificato.




**Naming e responsabilità:** Ogni test dovrebbe essere responsabile della verifica di un singolo comportamento ben definito. Un nome vago è spesso il sintomo di un test che verifica troppe cose insieme o non ha uno scopo chiaro. Al contrario, un nome esplicito e anche relativamente lungo indica che il comportamento sotto test è stato chiarito prima ancora di guardare l'implementazione.

**Struttura del progetto:** Nei progetti Java moderni, il codice di produzione e il codice di test sono mantenuti separati in modo esplicito. Questa separazione riflette una distinzione concettuale:

- il codice in src/main/java definisce cosa fa il sistema;
- il codice in src/test/java definisce come quel comportamento viene verificato.

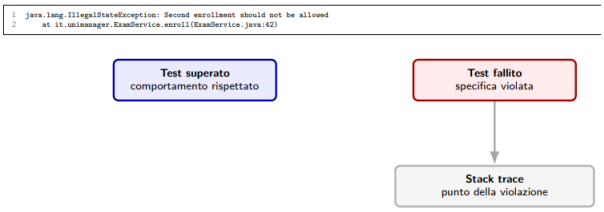
I test non sono codice “di supporto” o temporaneo, ma una parte strutturale del progetto, con pari dignità rispetto al codice di produzione. Mantenere questa distinzione rende più chiaro cosa appartiene al dominio del sistema e cosa appartiene alla sua specifica verificabile.

**Eseguire i test dall'IDE:** Gli ambienti di sviluppo moderni, come IntelliJ IDEA, integrano l'esecuzione dei test direttamente nel flusso di lavoro quotidiano. I test possono essere eseguiti a diversi livelli di granularità, a seconda del tipo di feedback che si vuole ottenere:

- un singolo metodo di test, per verificare rapidamente una regola specifica;
- un'intera classe di test, per controllare un componente nel suo insieme;
- l'intera suite, per avere fiducia globale prima di un cambiamento più ampio.

Questa integrazione rende naturale l'uso frequente dei test, riducendo l'attrito tra scrittura del codice e verifica del comportamento.

**Interpretare i risultati:** L'esito di un test fornisce un feedback immediato sul rispetto o meno del comportamento specificato. Un fallimento indica una violazione della specifica, non un errore generico del programma. Lo stack trace è uno strumento per risalire al punto in cui il comportamento atteso non è stato rispettato.




**Debug guidato dai test:** Quando un test fallisce, il debug non parte “a caso”: parte da una specifica violata. Flusso pratico consigliato:

1. leggere nome del test e messaggio di fallimento;
2. individuare il punto utile nello stack trace;
3. riprodurre il caso in debug con breakpoint mirato;
4. osservare stato e variabili nel punto della violazione;
5. correggere, rilanciare il test, poi la suite.

Obiettivo: correggere la causa del fallimento, non solo il sintomo.

**Test rosso -> debug -> fix**

```
org.opentest4j.AssertionFailedError:
Expected :2
Actual   :1
```

- Il test attende 2 iscritti, ma il servizio ne restituisce 1.
- Si imposta un breakpoint nel metodo enroll(...) e si verifica:
  - se il secondo inserimento viene davvero eseguito;
  - se una condizione blocca in anticipo il flusso;
  - se la collection usata per tracciare gli iscritti viene aggiornata.
- Correzione tipica: correzione della condizione o dell'aggiornamento stato.
- Verifica finale: test rosso diventa verde, poi riesecuzione della suite.

**UniManager:** I test vengono scritti su classi reali del sistema, non su esempi artificiali o semplificati. In particolare, vengono testati servizi applicativi come:

- StudentService, per la gestione degli studenti;
- ExamService, per la gestione di appelli, iscrizioni e voti.

Ogni classe di test verifica regole di business precise, come vincoli di iscrizione o condizioni di validità dei dati. Questo consente di osservare come JUnit venga usato su codice che potrebbe realmente evolvere nel tempo.

**Test completo:**

```java
class ExamServiceTest {

    private ExamService examService;
    private Student student;
    private Exam exam;

    @BeforeEach
    void setUp() {
        examService = new ExamService();
        student = new Student("12345");
        exam = new Exam("Software Engineering");
    }

    @Test
    void shouldNotEnrollStudentTwice() {
        examService.enroll(student, exam);
        assertThrows(IllegalStateException.class,
            () -> examService.enroll(student, exam));
    }
}
```

**Come rendiamo i test semplici:** Per mantenere i test di unità rapidi e focalizzati, le dipendenze infrastrutturali vengono eliminate o sostituite. In particolare:

- i repository sono implementati in memoria;
- non viene utilizzato alcun database reale;
- non vengono coinvolti framework o componenti esterni.

Questo permette di concentrarsi esclusivamente sulla logica di dominio e sui comportamenti da verificare. Il risultato è una suite di test eseguibile rapidamente, adatta al feedback continuo durante lo sviluppo.

**Cosa stiamo davvero testando:** I test non verificano come il codice è scritto, ma cosa il sistema deve fare in determinate condizioni. In pratica, vengono testate:

- regole di dominio (vincoli, validazioni, divieti);
- comportamenti attesi in scenari leciti e non leciti;
- reazioni del sistema a stati non validi.

I dettagli implementativi possono cambiare nel tempo senza invalidare i test, se il comportamento rimane corretto. Questo rende i test uno strumento di protezione durante refactoring ed evoluzione del codice.

**Verso il mocking:** Finora i test hanno riguardato classi con dipendenze semplici e facilmente controllabili. Nei sistemi reali, però, le classi collaborano con componenti esterni come repository, servizi o gateway. Queste dipendenze rendono i test più complessi, se non vengono gestite in modo esplicito.

**Messaggio finale:** Scrivere test significa rendere esplicite le regole e le aspettative sul comportamento del sistema. Test ben strutturati migliorano il design, perché rendono evidenti responsabilità e dipendenze. Una buona suite di test riduce il rischio del cambiamento e rende il codice più facile da mantenere ed evolvere.

**Riepilogo:**

- JUnit 5 fornisce il supporto pratico per trasformare principi di unit testing in test automatici eseguibili con continuità.
- Struttura dei test, annotazioni e fixture servono soprattutto a ottenere test indipendenti, ripetibili e leggibili.
- Assert mirati e naming esplicito migliorano la qualità del feedback e rendono i test più utili anche come documentazione eseguibile.
- I test dovrebbero descrivere il comportamento atteso del sistema, non i dettagli accidentali dell'implementazione.
- Suite rapide e isolate aumentano la frequenza di esecuzione e rendono il testing parte naturale del workflow quotidiano.
- La base costruita qui prepara il passaggio al mocking per gestire collaboratori esterni e dipendenze più complesse

## Mocking con Mockito

Finora i test che abbiamo visto verificano soprattutto:

- input → output,
- regole di business in una singola classe o metodo.

Ma in un sistema reale le classi raramente sono autosufficienti:

- un service delega a un repository,
- un manager usa un gateway verso un servizio esterno,
- un componente notifica un altro componente.

In questi casi il comportamento corretto non è solo “calcolare bene”, ma anche parlare correttamente con i collaboratori. Consideriamo un esempio tipico:

- ExamService gestisce la logica di iscrizione a un appello;
- salva una registrazione tramite ExamRepository.

Se nel test usiamo un repository reale, spesso succede che:

- dobbiamo configurare un database o una persistenza,
- il setup diventa lungo e fragile,
- il test diventa più lento e meno deterministico.

La domanda corretta non è “posso farlo?”, ma “vale la pena portarsi dietro tutto questo per testare una regola di business?”. La soluzione è sostituire le dipendenze. Nel test vogliamo controllare l'ambiente, quindi:

- sostituiamo la dipendenza reale con un oggetto controllabile,
- decidiamo noi cosa restituisce e cosa registra.

Questo approccio produce tre benefici immediati:

- **isolamento:** testiamo davvero solo la classe sotto test;
- **velocità:** niente I/O, niente setup pesante;
- **precisione:** quando fallisce sappiamo dove guardare.

Qui entrano in gioco i sostituti di test.

**Sostituti di test:** Nei test usiamo spesso oggetti “sostituti” dei reali, ma con scopi diversi:

- ridurre il costo del setup (niente DB, rete, filesystem),
- rendere il test deterministico (stesso input ⇒ stesso risultato),
- isolare il comportamento della classe sotto test dai dettagli esterni.

In pratica, tre categorie ricorrono continuamente:

- **Stub:** restituisce valori predefiniti per far proseguire il flusso (ci interessa il risultato, non come quel valore è stato ottenuto).
- **Fake:** implementazione semplificata ma funzionante (es. repository in-memory al posto di un database reale).
- **Mock:** oggetto controllato che non solo restituisce valori, ma permette di verificare le chiamate (chi ha chiamato cosa, con quali parametri).




**Stub: risposte predefinite:** Uno stub serve quando la classe sotto test:

- ha bisogno di un valore per poter proseguire l'esecuzione,
- ma non è importante capire come quel valore venga calcolato o prodotto.

In altre parole, lo stub viene usato per controllare lo scenario del test, fornendo risposte coerenti e prevedibili alle richieste della classe sotto test.

> **Un repository che deve rispondere alla domanda “esiste già questa iscrizione?”**

- nel test decidiamo esplicitamente la risposta, ad esempio true oppure false,
- così possiamo verificare il comportamento del service nei diversi casi.

Lo stub non registra chiamate e non impone vincoli sulle interazioni: il suo unico scopo è restituire valori utili al test. In breve: lo stub è “un distributore di risposte”, semplice e mirato.

**Esempio di stub Java manuale:**

Una classe minimale che fornisce risposte predefinite. L'obiettivo è quello di controllare in modo esplicito lo scenario del test attraverso i valori restituiti dal collaboratore.

```java
class ExamRepositoryStub implements ExamRepository {
    @Override
    public boolean isEnrolled(Student s, Exam e) {
        return true; // risposta predefinita per il test
    }

    @Override
    public void saveRegistration(Student s, Exam e) {
        // no-op: nello stub non ci interessa questa parte
    }
}
```

**Fake: implementazione semplificata ma reale:** Un fake è un oggetto che funziona davvero, cioè esegue una logica coerente con quella reale, ma in una forma più leggera e più facile da controllare. A differenza di uno stub, un fake mantiene uno stato interno e può essere utilizzato per far “girare” un flusso completo. Tipici esempi industriali includono:

- database in memoria (basati su liste o mappe) al posto di un DB vero,
- file system finto usato solo all'interno dei test,
- servizi locali o embedded invece di chiamate di rete reali.

Il fake è utile quando vogliamo un comportamento realistico e coerente con il sistema finale, ma non vogliamo pagare il costo dell'infrastruttura reale in termini di configurazione, tempo di esecuzione e fragilità dei test.

**Esempio Java:**

Un fake non restituisce solo valori fissi: mantiene uno stato interno e si comporta in modo realistico. Qui il repository salva davvero le iscrizioni, ma in memoria.

```java
class InMemoryExamRepository implements ExamRepository {
    private final Set<String> enrolled = new HashSet<>();

    @Override
    public boolean isEnrolled(Student s, Exam e) {
        return enrolled.contains(key(s, e));
    }

    @Override
    public void saveRegistration(Student s, Exam e) {
        enrolled.add(key(s, e));
    }

    private String key(Student s, Exam e) {
        return s.id() + ":" + e.code();
    }
}
```

```java
InMemoryExamRepository repo = new InMemoryExamRepository();
assertFalse(repo.isEnrolled(student, exam)); // prima del salvataggio
repo.saveRegistration(student, exam);
assertTrue(repo.isEnrolled(student, exam));  // dopo il salvataggio
```

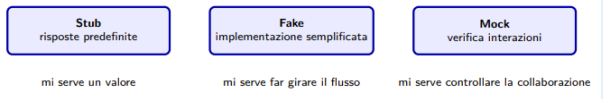
**Mock: verificare le interazioni:** Un mock è pensato per testare comportamenti collaborativi, cioè situazioni in cui una classe non lavora da sola ma delega parte del lavoro:

- la classe sotto test deve chiamare un collaboratore,
- con certi parametri specifici,
- un certo numero di volte (o mai),

- eventualmente rispettando un certo ordine.

In questi casi l'output finale non è sempre sufficiente: anche se il metodo non restituisce nulla, vogliamo controllare che la collaborazione sia avvenuta correttamente. In breve: il mock è un “collaboratore controllato” che permette anche di osservare le interazioni tra oggetti.

**Riassunto:**




- Stub, fake e mock sono concetti di testing, non strumenti specifici.
- Nella pratica, per creare e configurare mock in Java possiamo usare una libreria di supporto: qui entra in gioco **Mockito**.

**Mockito:** Nei test collaborativi vogliamo sostituire alcuni oggetti reali con collaboratori controllabili. Mockito è una libreria Java per il testing. Mockito permette di creare questi collaboratori sotto forma di mock:

- possiamo configurarli come stub, quando servono risposte predefinite;
- possiamo usarli come mock, quando vogliamo verificare le interazioni.

Questo ci consente di testare la logica applicativa senza trascinarci dietro database, rete o infrastruttura reale. Mockito però non sostituisce il design:

- funziona bene solo se le dipendenze sono **sostituibili**;
- cioè se il codice rispetta il **Dependency Inversion Principle** (DIP).

Per usare Mockito in modo ordinato, conviene ragionare con una struttura fissa:

- **Given:** preparo scenario e collaboratori;
- **When:** eseguo l'azione da testare;
- **Then:** verifico risultato e/o collaborazione.

**Iscrizione a un appello:**

Nel test vogliamo simulare uno scenario semplice: lo studente non è ancora iscritto e quindi il service può procedere.

```
// Given
// il repository indica che lo studente non e' gia' iscritto

// When
service.enroll(student, exam);

// Then
// il service richiede il salvataggio della registrazione
```

Il punto chiave è separare bene i tre momenti:

- prima preparo il comportamento dei collaboratori;
- poi invoco solo il metodo che voglio testare;
- infine controllo che il service abbia collaborato correttamente.

**Caso d'uso in UniManager:**

Uno studente prova a iscriversi a un appello d'esame disponibile nel sistema. ExamService applica le regole di business principali e poi delega alcune operazioni a componenti collaboratori:

- controlli di pre-condizione, ad esempio la doppia iscrizione,
- salvataggio della registrazione tramite ExamRepository.

Il nostro obiettivo nel test è verificare il comportamento del service in isolamento:

- in condizioni valide, il service deve salvare correttamente la registrazione;
- in condizioni non valide, il service deve non invocare alcun salvataggio.

**Costruzione testabile: DIP in pratica:** Prima di entrare nei dettagli operativi di Mockito, è fondamentale guardare come è costruita la classe. Per poter scrivere test isolati e usare sostituti di test, la classe deve essere progettata in modo testabile:

- deve dipendere da un'astrazione (ExamRepository),
- deve ricevere la dipendenza dall'esterno (costruttore o setter), invece di crearla internamente.

Questo è un esempio di applicazione pratica del Dependency Inversion Principle (DIP).

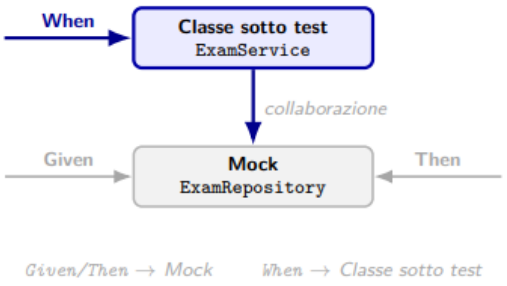
```java
class ExamService {
    private final ExamRepository repo;

    ExamService(ExamRepository repo) {
        this.repo = repo;
    }

    void enroll(Student s, Exam e) {
        if (repo.isEnrolled(s, e)) {
            throw new IllegalStateException("Student already enrolled");
        }

        repo.saveRegistration(s, e);
    }
}
```

Schema mentale Mockito:




Nei test collaborativi, il flusso tipico segue Given / When / Then:

1. **Given:** si crea il mock e si definisce lo scenario (when);
2. **When:** si invoca il metodo del service (azione sotto test);
3. **Then:** si verifica la collaborazione (verify). Regola pratica: when prepara il comportamento atteso, verify controlla quello avvenuto.

**Come creare un mock:** A questo punto, nel test, vogliamo sostituire la dipendenza reale con un oggetto controllabile. Mockito fornisce il metodo `mock(...)` per creare un mock:

```java
ExamRepository repo = mock(ExamRepository.class);
```

`mock(...)` non è una parola chiave di Java, ma è un metodo statico della libreria Mockito. Tipicamente importato con `import static org.mockito.Mockito.mock;` Questo oggetto:

- prende il posto del collaboratore reale nel test,
- non ha un'implementazione reale,
- registra le chiamate che riceve.

**Mockito: definire comportamento (stub):** Un mock, appena creato, non fa nulla di utile: restituisce valori di default (null, false, 0). Per questo, nel test, dobbiamo dirgli come deve rispondere quando viene interrogato. Mockito usa il costrutto `when(...).thenReturn(...)` :

```java
when(repo.isEnrolled(student, exam)).thenReturn(false);
```

Questo codice non esegue il metodo:

- sta programmando il comportamento del mock,
- dice in modo esplicito: “se viene chiamato `repo.isEnrolled(student, exam)` , allora il mock restituisce false”.

In questo modo controlliamo lo scenario del test:

- iscrizione inesistente ⇒ il service può procedere;
- se restituiamo true ⇒ il service deve bloccare l'operazione.

**Mockito: verify per controllare la collaborazione:** Dopo aver eseguito l'azione sotto test, vogliamo controllare come la classe ha collaborato con i suoi dipendenti. Mockito fornisce il metodo `verify(...)` per osservare le chiamate già avvenute:

```
verify(repo).saveRegistration(student, exam);
```

Questo codice non esegue il metodo:

- non salva nulla,
- non modifica lo stato del sistema.

Sta invece affermando che:

- durante l'esecuzione del test,
- il service ha invocato quel metodo su quel collaboratore, con quei parametri.

Se quella chiamata non è avvenuta, il test fallisce. Questo test non dice “il DB contiene una riga”, ma una cosa più adatta a un unit test: il service ha chiesto al repository di salvare.

```
class ExamServiceMockitoTest {

    private ExamRepository repo;
    private ExamService service;

    private Student student;
    private Exam exam;

    @BeforeEach
    void setUp() {
        // Given
        repo = mock(ExamRepository.class);
        service = new ExamService(repo);

        student = new Student("12345");
        exam = new Exam("Software Engineering");
    }

    @Test
    void shouldSaveRegistrationWhenValid() {
        // Given
        when(repo.isEnrolled(student, exam)).thenReturn(false);

        // When
        service.enroll(student, exam);

        // Then
        verify(repo).saveRegistration(student, exam);
    }
}
```

Esempio completo di un caso valido: Il test verifica il comportamento di ExamService in una situazione valida di iscrizione. La struttura segue una sequenza standard:

1. Preparazione dell'ambiente e dei sostituti di test (**Given**);
2. Esecuzione dell'azione sotto test (**When**);
3. Verifica del comportamento collaborativo (**Then**).

Flusso concreto del caso valido:

- **Given:** isEnrolled(...) restituisce false (studente non ancora iscritto);
- **When:** il service esegue enroll(student, exam);
- **Then:** il repository deve ricevere saveRegistration(student, exam).

L'obiettivo del test è verificare che:

- l'iscrizione risulti consentita,
- il repository venga correttamente invocato.

```
@Test
void shouldNotSaveIfAlreadyEnrolled() {
    // Given
    when(repo.isEnrolled(student, exam)).thenReturn(true);

    // When
    assertThrows(IllegalStateException.class,
        () -> service.enroll(student, exam));

    // Then
    verify(repo, never()).saveRegistration(student, exam);
}
```

Caso non valido: Il test verifica il comportamento di ExamService in una situazione NON valida di iscrizione. Scenario considerato: lo studente risulta già iscritto all'appello. L'aspettativa è duplice:

- l'operazione deve essere rifiutata,
- il repository non deve essere invocato per il salvataggio.

Flusso concreto del caso non valido:

- **Given:** isEnrolled(...) restituisce true (studente già iscritto);
- **When:** il service esegue enroll(student, exam)
- **Then:** viene lanciata IllegalStateException e saveRegistration(...) non deve essere chiamato.

Il test verifica quindi la segnalazione dell'errore e l'assenza di collaborazioni indesiderate.

**Varianti di verify: times / never:** Con `verify(...)` non si controlla solo che un metodo sia stato chiamato, ma anche quante volte è stato invocato. Mockito consente di esprimere vincoli quantitativi sulle interazioni:

```
verify(repo, times(1)).saveRegistration(student, exam);
verify(repo, never()).deleteRegistration(student, exam);
```

Nota didattica:

- `times(1)` è il comportamento di default di verify, quindi spesso è ridondante;
- diventa utile quando il numero di chiamate è parte esplicita del comportamento atteso;
- `never()` è invece utile per affermare che una collaborazione non deve avvenire.

**Mocking e buon design: DIP come prerequisito:** Il mocking funziona bene solo quando il design lo permette:

- dipendenze esplicite e visibili,
- costruttore (o setter) che riceve i collaboratori,
- dipendenza da interfacce e astrazioni, non da implementazioni concrete.

Se una classe crea internamente le proprie dipendenze (ad esempio usando new su un repository):

- la dipendenza non è sostituibile,
- il test deve portarsi dietro l'infrastruttura reale,
- il codice risulta più accoppiato e meno evolvibile.

I mock quindi non sono solo uno strumento di testing: rappresentano anche un feedback diretto sul design e sul rispetto del Dependency Inversion Principle (DIP).

**Mocking come feedback:** I test con mock possono essere letti anche come uno strumento di diagnosi del design. Se per testare una singola classe è necessario creare:

- molti mock,
- molte configurazioni when/thenReturn,
- numerosi verify molto dettagliati,

allora spesso il problema non è Mockito:

- la classe sotto test probabilmente ha troppe responsabilità (violazione di SRP);
- oppure dipende da dettagli concreti invece che da astrazioni (violazione di DIP);
- oppure espone un comportamento poco coeso e difficile da isolare.

In questi casi il test sta facendo emergere la necessità di un refactoring del design.

**Cattive pratiche:** Alcuni errori ricorrenti nell'uso dei mock includono:

- mockare tutto, inclusi oggetti di dominio semplici e stabili;
- verificare interazioni superflue (test troppo “pettegoli”);
- scrivere test che replicano l'implementazione interna (“prima chiama A, poi B, poi C”);
- abuso di verifiche troppo restrittive su interazioni secondarie.

L'effetto tipico di queste pratiche è:

- test fragili, che falliscono a ogni refactoring,
- perdita del beneficio principale dei test automatici: la possibilità di cambiare codice con fiducia.

**Regole pratiche per usare bene i mock:** Alcune regole empiriche utili per un uso corretto dei mock:

- mockare i collaboratori, non l'oggetto sotto test;
- verificare solo ciò che conta per il comportamento osservabile;
- preferire test che descrivono cosa deve accadere, non come accade internamente;
- se un test richiede troppi mock, considerare una riduzione di responsabilità o dipendenze.

L'obiettivo finale è avere test che rimangano stabili nel tempo, anche quando cambia l'implementazione, purché il comportamento esterno resti coerente.

> **Riepilogo:**
>
> - Stub, fake e mock (sostituti di test) servono a scopi diversi e vanno scelti in base al tipo di verifica.
> - Mockito è utile soprattutto per test collaborativi: configurare risposte dei mock e verificare interazioni.
> - Il mocking funziona bene quando il design è testabile (DIP, dipendenze sostituibili).
> - Troppi mock o verifiche troppo dettagliate sono un segnale di design da rifattorizzare. 28

## Git: controllo di versione

Senza un sistema di controllo di versione, la gestione delle modifiche tende a degenerare in:

- copie multiple dello stesso progetto (progetto_finale_v 3_definitivo 2.zip),
- file sovrascritti e versioni perse,
- difficoltà a ricostruire uno stato funzionante dopo un errore.

In questo scenario, due domande diventano rapidamente costose (tempo, stress, rischio):

- Chi ha cambiato cosa?
- Quando e perché è stato cambiato?

Git introduce una risposta strutturata:

- ogni modifica significativa viene registrata come commit,
- la storia risulta tracciabile, consultabile e verificabile.

Git è un sistema di controllo di versione distribuito:

- ogni copia locale del repository contiene l'intera storia del progetto,
- il lavoro quotidiano non dipende dalla disponibilità continua di un server centrale.

Il concetto fondamentale è il commit: ogni commit rappresenta una fotografia coerente e consistente dello stato del progetto in un dato momento.

> **Contesto storico e nome:**
>
> - Git è stato creato da Linus Torvalds nel 3005;
> - il nome è stato usato in modo ironico; compare anche il backronym "Global Information Tracker".

Il valore pratico di questo modello è operativo:

- è possibile sperimentare in modo controllato,
- correggere errori senza perdere informazioni,
- recuperare stati precedenti grazie alla storia registrata.

**Git non è GitHub:**

- Git:
  - strumento locale di controllo di versione,
  - responsabile della gestione della storia del progetto,
  - supporta operazioni come commit, merge e recupero degli errori.
- GitHub (così come GitLab, Bitbucket,…):
  - servizio di hosting per repository remoti,
  - consente la condivisione del codice tra più sviluppatori,
  - aggiunge funzionalità di collaborazione e automazione sopra Git.

**Setup minimo: init, clone, status:** L'inizializzazione di un progetto Git può avvenire in due modi principali:

- creare un nuovo repository all'interno di una cartella esistente tramite git init,
- ottenere una copia locale di un repository già esistente tramite git clone.

In entrambi i casi, è fondamentale verificare lo stato del repository:

- il comando git status fornisce una fotografia immediata della situazione corrente del progetto,
- permette di capire quali file sono modificati, quali sono già tracciati e quali no.

```
git init
git status

git clone https://github.com/<org>/<repo>.git
git status
```

**Lettura rapida di git status:** Il comando git status fornisce una sintesi immediata dello stato corrente del repository. In particolare, indica:

- il branch attualmente attivo (tipicamente main),
- i file che risultano modificati ma non ancora selezionati,
- i file che si trovano in staging e sono pronti per il commit.

Dal punto di vista operativo, git status è il comando di riferimento:

- chiarisce sempre “a che punto si è”,
- riduce il rischio di operazioni non intenzionali.

Regola pratica: in caso di dubbio sullo stato del progetto, eseguire git status prima di qualsiasi altra operazione.

```
git status
# modified:   src/...
# untracked:  notes.txt
# changes to be committed: ...
```

**Scegliere cosa entra nel commit:** Git non include automaticamente tutte le modifiche in un commit: la selezione è un'operazione esplicita e controllata. Il comando `git add` sposta una modifica dall'area di lavoro alla staging area. Questo passaggio permette di:

- rendere esplicito cosa si intende salvare nella storia,
- costruire commit coerenti anche in presenza di più modifiche,
- evitare l'inclusione involontaria di file o cambiamenti non rilevanti.

```
git status
# modified:   src/main/java/unimanager/ExamService.java
# modified:   src/test/java/unimanager/ExamServiceTest.java
# untracked:  notes.txt

git add src/main/java/unimanager/ExamService.java
git add src/test/java/unimanager/ExamServiceTest.java

git status
# changes to be committed:
#   modified: src/main/java/unimanager/ExamService.java
#   modified: src/test/java/unimanager/ExamServiceTest.java
#
# untracked files:
#   notes.txt
```

**Diff:** Prima di creare un commit è importante comprendere quali modifiche verranno effettivamente salvate. Git mette a disposizione due comandi distinti per questo scopo:

1. `git diff`: mostra le differenze presenti nel working tree ma non ancora aggiunte alla staging area;
2. `git diff --staged` (o --cached): mostra invece le modifiche già selezionate e pronte per il commit.

Questa distinzione consente di:

- verificare che ogni commit contenga solo ciò che è intenzionale,
- individuare rapidamente modifiche accidentali o incomplete,
- mantenere una storia del progetto più chiara e leggibile.

```
git diff          # modifiche non ancora in staging
git diff --staged  # modifiche pronte per il commit
```

Regola operativa: prima di eseguire git commit, controllare sempre l'output di almeno uno dei due comandi.

**Commit:** Un commit rappresenta una fotografia coerente dello stato del progetto in un preciso momento. Ogni commit include metadati essenziali:

- l'autore della modifica,
- il timestamp,
- un messaggio descrittivo.

Il messaggio di commit non è un dettaglio formale:

- costituisce una forma di documentazione tecnica,

- serve a spiegare cosa è cambiato e, soprattutto, perché.

```
git commit -m "Prevent double enrollment in ExamService"
```

**Messaggi di commit:** I messaggi di commit svolgono un ruolo centrale nella comprensione e nella manutenzione della storia del progetto.

- Un buon messaggio di commit:
  - descrive chiaramente cosa è stato modificato,
  - esplicita, quando rilevante, perché la modifica è stata introdotta,
  - rimane sintetico, ma sufficientemente informativo.
- Al contrario, messaggi poco informativi:
  - non aggiungono contesto tecnico,
  - non aiutano a ricostruire le motivazioni delle scelte,
  - rendono la storia del progetto difficile da leggere nel tempo.

**Esempi tipici di messaggi da evitare:**

- "fix"
- "changes"
- "final version"

```
Esempio: Add validation to prevent double enrollment
    (cosa) prevenzione della doppia iscrizione   (perché) applicazione di una regola di business
```

**Log:** Il comando `git log` consente di esplorare la storia delle modifiche registrate nel repository. In particolare, è utile per:

- comprendere l'evoluzione del codice nel tempo,
- individuare il commit che ha introdotto un errore o una regressione,
- ricostruire le motivazioni tecniche dietro determinate scelte progettuali.

Per una visualizzazione più compatta e leggibile, Git mette a disposizione alcune opzioni di formattazione:

```
git log --oneline --decorate --graph --all
```

Questa vista aiuta a cogliere rapidamente:

- la sequenza dei commit,
- la presenza di rami e merge,
- lo stato dei puntatori principali (branch e tag).

**Granularità:** La dimensione di un commit influenza direttamente leggibilità, manutenzione e capacità di recupero nel tempo.

- Commit piccoli (coerenti e focalizzati):
  - risultano più semplici da comprendere e discutere,
  - possono essere annullati con minore impatto (revert più mirato),
  - producono una storia del progetto più chiara e navigabile.
- Commit molto grandi (eterogenei):
  - nascondono cambiamenti non intenzionali o non correlati,
  - aumentano la probabilità di conflitti durante l'integrazione,
  - rendono più difficile isolare la causa di un problema e tornare indietro.

Regola operativa: un commit dovrebbe rappresentare una singola idea o modifica logica.

**.gitignore:** In un progetto Git non tutti i file presenti nella cartella di lavoro devono essere tracciati e salvati nella storia. In particolare, è buona pratica escludere:

- file temporanei o generati automaticamente (ad esempio tmp, log),
- output del processo di build (come target/ o build/),
- file specifici dell'ambiente di sviluppo (configurazioni di IDE come `.idea/`, `*.iml`).

Il file .gitignore permette di dichiarare in modo esplicito quali file o directory Git deve ignorare:

- tali elementi restano locali,
- non vengono inclusi nei commit,
- non “inquinano” la storia del progetto.

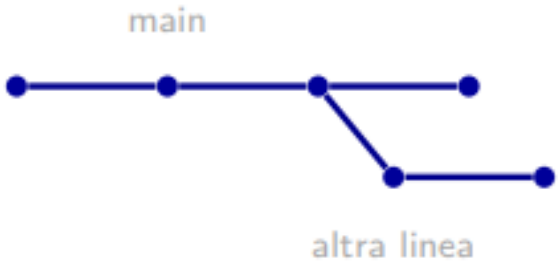
```
# Maven / Gradle
target/
build/

# IntelliJ
.idea/
*.iml

# OS / temp
.DS_Store
*.log
```

**Branch:** Un branch rappresenta una linea alternativa della storia, che nasce da un commit esistente. `main` è la linea principale, ma da un punto preciso la storia può biforcarsi. È sufficiente sapere che:

- le linee di sviluppo nascono da un commit comune,
- quando si ricongiungono è necessaria un'operazione di merge.

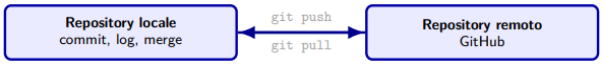



**Locale vs remoto:** In un contesto di sviluppo, è comune distinguere tra:

- *repository locale:* copia presente sul computer, usata per lavorare e creare commit;
- *repository remoto:* copia condivisa, usata come punto di riferimento per il team (ad esempio su GitHub).

Il lavoro quotidiano avviene principalmente in locale, mentre la condivisione avviene tramite lo scambio di commit. In questo modello:

- `git push` pubblica i commit locali sul remoto,
- `git pull` scarica e integra i commit presenti sul remoto.




**Remote:** In un repository Git, il remoto principale è convenzionalmente chiamato `origin`. Il remoto rappresenta il punto di sincronizzazione tra il lavoro locale e la copia condivisa del progetto. Alcuni comandi di base permettono di gestire questa relazione:

- `git remote -v`: consente di visualizzare i remoti configurati e i relativi URL;
- `git push`: invia i commit locali al repository remoto, rendendoli disponibili agli altri;
- `git pull`: recupera i commit dal remoto e li integra nel repository locale.

L'uso consapevole di questi comandi è essenziale per mantenere allineate le diverse copie del progetto.

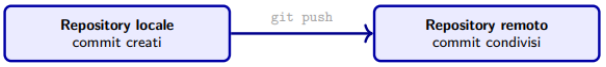
```
git remote -v
git push
git pull
```

Un modello mentale utile distingue chiaramente due momenti:

- `commit`: la storia viene salvata localmente,
- `push`: la storia viene resa pubblica.

Buona pratica operativa:

- eseguire push frequenti,
- ma solo su commit coerenti, compilabili e significativi.




**Pull:** Il comando `git pull` serve a recuperare i commit presenti sul remoto e ad aggiornarne la copia locale. In pratica, il pull combina due operazioni:

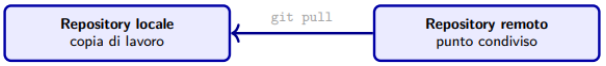
- scarica i nuovi commit dal repository remoto,
- integra tali commit nella storia locale.

Possono emergere problemi quando:

- nel frattempo altri hanno modificato gli stessi file o le stesse righe,
- la storia locale e quella remota devono essere integrate tramite merge.

Regola operativa:

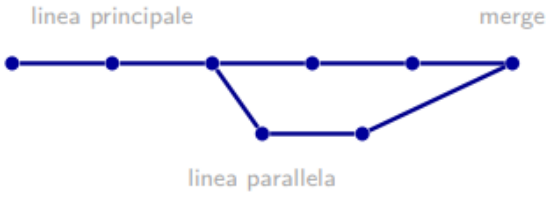
- prima di iniziare a lavorare su un progetto condiviso: eseguire `git pull`,
- prima di pubblicare nuove modifiche: eseguire `git pull` per ridurre conflitti e sorprese.




**Merge:** Un merge è l'operazione che consente di integrare due linee di sviluppo in una storia comune. Il merge si verifica tipicamente quando:

- il repository locale viene aggiornato con commit remoti,
- più sviluppatori hanno lavorato in parallelo.

Se le modifiche non si sovrappongono, Git integra automaticamente. In caso contrario, è richiesto un intervento esplicito: si genera un conflitto di merge.




**Conflitti:** Un conflitto indica semplicemente che Git non è in grado di decidere automaticamente quale versione di una modifica mantenere. La presenza di un conflitto:

- non segnala un errore concettuale,
- non è “colpa” di qualcuno,
- è una conseguenza normale del lavoro in parallelo sullo stesso codice.

La gestione corretta di un conflitto segue alcuni passi chiave:

1. comprendere l'obiettivo delle due modifiche in conflitto;
2. scegliere la versione corretta o combinarle in modo coerente;

3. ricompilare il progetto ed eseguire i test per verificare il risultato.

L'obiettivo non è evitare ogni conflitto, ma saperli risolvere in modo consapevole e controllato.

**Come appare un conflitto:** Quando Git non riesce a risolvere automaticamente un merge, inserisce dei marcatori di conflitto direttamente nel file coinvolto. Questi marcatori delimitano le due versioni concorrenti della modifica:

```
<<<<<<< HEAD
return enrollIfValid(student, exam);
=======
return enroll(student, exam);
>>>>>>> origin/main
```

La risoluzione del conflitto richiede un intervento manuale:

- analizzare le due versioni e comprenderne l'intento,
- rimuovere i marcatori ( `<<<`, `===`, `>>>` ),
- lasciare nel file una sola versione coerente del codice.

Una volta risolto il conflitto:

- il file va aggiunto nuovamente allo staging con `git add` ,
- il merge si conclude con un `git commit` dedicato.

**Example**

- Contesto: due sviluppatori lavorano sullo stesso file, uno ha già fatto push sul remoto, l'altro prova a fare push con una storia non aggiornata.
- Tentativo di push:

```
git push
# rejected: non-fast-forward
# error: failed to push some refs
```

- Soluzione: aggiornare e integrare le modifiche remote

```
git pull
# Git prova il merge

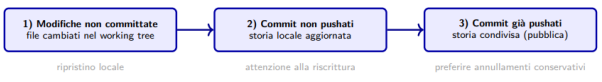
# se c'e' conflitto:
# modificare i file, risolvere manualmente

git add file.java
git commit   # completa il merge

git push     # ora funziona
```

- Idea chiave: il push fallisce perché la storia remota è più avanti, il pull integra le modifiche (merge), poi il push può andare a buon fine.

**Undo:** Git consente di recuperare situazioni non desiderate in modo controllato, senza interventi improvvisati. Il comando corretto dipende da dove si trovano le modifiche.




La distinzione tra "privato" e "pubblico" è determinante: alcuni comandi riscrivono la storia, altri la compensano.

**Ripristinare file non committati:** Quando sono presenti modifiche nel working tree che non devono essere mantenute, è possibile tornare allo stato dell'ultimo commit. In questo caso, il comando `git restore`

ripristina il contenuto del file come registrato nella storia:

```
git restore src/main/java/unimanager/ExamService.java
```

È anche possibile intervenire sullo staging:

- se un file è stato aggiunto per errore con `git add`,
- lo si può rimuovere dallo staging senza perdere le modifiche locali.

```
git restore --staged src/main/java/unimanager/ExamService.java
```

In entrambi i casi, il ripristino agisce solo sullo stato locale e non modifica la storia dei commit.

**Modificare l'ultimo commit locale:** Un caso frequente durante lo sviluppo locale è:

- un file dimenticato nel commit precedente,
- un messaggio di commit impreciso o errato.

Altro caso tipico: aver committato per errore un file molto grande (ad esempio 1 GB) che non doveva entrare nella storia. L'opzione `--amend` consente di modificare l'ultimo commit, aggiornandone il contenuto e/o il messaggio. Questa operazione riscrive la storia e deve essere usata solo su commit che non sono ancora stati pubblicati.

```
git add src/test/java/unimanager/ExamServiceTest.java
git commit --amend
```

Regola operativa:

- usare `--amend` solo su commit locali,
- evitare sempre `--amend` su commit già pushati.

`Reset:` Il comando `git reset` sposta il puntatore della storia locale ad un commit precedente. È uno strumento potente ma delicato:

- utile per correggere errori in locale,
- potenzialmente pericoloso se applicato a commit già pubblici.

Il comportamento di `git reset` dipende dall'opzione utilizzata:

```
git reset --soft <commit>   # mantiene le modifiche nello staging
git reset --mixed <commit>  # mantiene le modifiche nel working tree
git reset --hard <commit>   # elimina le modifiche locali (attenzione!)
```

Regola operativa fondamentale:

- usare `git reset` solo su commit non pushati,
- evitare `--hard` se non si è certi di voler perdere il lavoro locale.

### git reset --soft: fondere commit

- Situazione:
  - C1: struttura progetto
  - C2: aggiungo funzione
  - C3: fix piccolo bug
- Problema: C2 e C3 dovevano essere un unico commit.
- Soluzione:

```
git reset --soft HEAD-2
git commit -m "aggiunta funzione + fix"
```

- Effetto:
  - torno a C1
  - modifiche di C2 e C3 nello staging
  - creo un nuovo commit unico

### git reset --mixed: riorganizzare commit

- Situazione:
  - C1: base
  - C2: modifica A + modifica B (troppo insieme)
- Problema: voglio separare A e B in due commit distinti.
- Soluzione:

```
git reset --mixed HEAD~1

# ora scelgo cosa aggiungere
git add fileA
git commit -m "modifica A"

git add fileB
git commit -m "modifica B"
```

- Effetto:
  - modifiche restano nei file
  - staging vuoto → scelgo io cosa includere

### git reset --hard: tornare indietro e scartare tutto

- Situazione:
  - ho creato un commit locale sbagliato
  - voglio eliminarlo insieme alle sue modifiche
- Obiettivo: tornare allo stato del commit precedente.
- Soluzione:

```
git reset --hard HEAD~1
```

- Effetto:
  - l'ultimo commit locale viene eliminato dalla storia
  - staging e working tree tornano allo stato del commit precedente
  - eventuali modifiche locali non salvate vengono perse

**Revert:** Quando un commit è già stato condiviso con altri, la strategia più sicura per annullarne gli effetti è `git revert`. Il comando `git revert`:

- crea un nuovo commit che annulla logicamente le modifiche introdotte da un commit precedente,
- preserva la storia esistente senza eliminarla o riscriverla.

Questo approccio è particolarmente adatto a repository condivisi, perché mantiene una cronologia completa e tracciabile.

```
git revert <commit>
```

Messaggio operativo da ricordare:

- reset riscrive la storia,
- revert la compensa in modo sicuro.

**Pubblico vs privato.** Prima di utilizzare comandi potenzialmente invasivi, è fondamentale chiarire il contesto in cui si sta operando. Due domande guida aiutano a scegliere l'azione corretta:

- il commit è già stato pubblicato su un repository remoto?
- altri sviluppatori potrebbero basarsi su quel commit?

In termini generali, vale la seguente distinzione:

- in locale è possibile adottare strategie più aggressive (ad esempio `reset`, `--amend`);
- su repository condivisi è preferibile usare operazioni conservative che preservano la storia (ad esempio revert).

> **In caso di dubbio:** consolidare il lavoro con un commit, rimandare il push e confrontarsi prima di procedere.

**Tag:** I tag permettono di associare un nome stabile a un commit specifico della storia. Sono tipicamente usati per identificare:

- versioni rilasciate (v 1.0, v 1.1),
- snapshot significativi del progetto,
- punti di riferimento per build, deploy o documentazione.
- indica esattamente quel commit e nient'altro.

I tag non vengono pushati automaticamente:

```
1  git tag -a v1.0 -m "Prima versione stabile"
2  git push origin v1.0
```




Il tag v1.0 rimane fissato su un commit anche se HEAD avanza

**Errori comuni (e come evitarli):**

- Aggiungere tutto indiscriminatamente al commit (git add .) senza verificare cosa viene incluso.
- Usare messaggi di commit vaghi o generici: "fix" non fornisce alcuna informazione utile (nemmeno a distanza di poche settimane).
- Creare commit senza aver controllato le differenze con git diff o git diff `--staged`.
- Aggiornarsi raramente dal remoto, accumulando conflitti che emergono solo all'ultimo momento.
- Utilizzare comandi potenzialmente distruttivi senza aver chiarito se il lavoro è già pubblico o condiviso.
- Dimenticare che il file .gitignore va condiviso:
  - funziona localmente, ma
  - per avere effetto su tutto il team deve essere committato.

**Un flusso minimo e sano:**

1. git status (verifica dello stato del repository)
2. sviluppo di una modifica piccola e circoscritta
3. git diff (analisi delle modifiche introdotte)
4. git add (selezione esplicita dei contenuti del commit)
5. git commit -m "..." (una singola modifica logica)
6. git pull (sincronizzazione con il remoto, se presente)
7. git push (pubblicazione dei commit)

Questo flusso non è pensato per essere sofisticato, ma per essere robusto, ripetibile e facilmente verificabile. L'adozione sistematica di questa sequenza riduce conflitti, errori evitabili e interventi correttivi non pianificati.

> **Riepilogo:**
>
> - Git è utile soprattutto perché rende tracciabili le modifiche e riduce il caos della collaborazione sul codice.
> - Il modello working tree / staging area / repository è la chiave per capire cosa sta succedendo prima di usare i comandi.
> - Un flusso quotidiano semplice e ripetibile (status, diff, add, commit, pull, push) previene molti errori evitabili.

- Merge e conflitti fanno parte del lavoro normale: gestirli presto e spesso è meglio che rimandarli.
- Le operazioni di undo richiedono attenzione al contesto (locale vs condiviso), non solo memoria dei comandi.
- Buone pratiche di commit e versionamento migliorano review, tracciabilità e manutenzione del progetto.

## Continuous integration

Nelle lezioni precedenti sono stati introdotti diversi strumenti fondamentali per migliorare la qualità del software sviluppato:

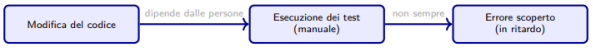
- i test automatici, utilizzati per verificare in modo sistematico il comportamento atteso del codice;
- il mocking, impiegato per isolare le dipendenze e rendere i test più mirati e controllabili;
- Git, come sistema di controllo di versione per tenere traccia dell'evoluzione del progetto nel tempo.

L'uso combinato di questi strumenti contribuisce a rendere il codice più affidabile e più semplice da mantenere. Rimane tuttavia una questione aperta: come garantire che tali verifiche vengano eseguite in modo regolare, sistematico e affidabile nel tempo?

I test automatici rappresentano uno strumento efficace di verifica solo nella misura in cui vengono effettivamente eseguiti. In un processo basato esclusivamente su attività manuali:

- l'esecuzione dei test dipende dall'attenzione, dall'esperienza e dalla disciplina delle persone coinvolte;
- è possibile dimenticare di lanciare i test prima di condividere una modifica o di proseguire con altro lavoro;
- gli errori possono emergere quando il codice è già stato modificato più volte, rendendo più difficile individuarne la causa.

Questo tipo di approccio tende a diventare fragile e poco affidabile man mano che il progetto cresce in dimensione e complessità.




**Il problema delle regressioni:** Una regressione si verifica quando una modifica introduce un errore in una funzionalità che in precedenza funzionava correttamente. Le regressioni sono spesso non intenzionali:

- una modifica locale può avere effetti indiretti su altre parti del sistema;
- il problema può non essere immediatamente evidente.

Senza verifiche sistematiche, le regressioni tendono ad accumularsi nel tempo.

**Controllo manuale e controllo automatico:** Il controllo manuale del codice presenta alcuni limiti strutturali, legati alla sua dipendenza dalle persone che lo eseguono:

- non è uniforme, poiché sviluppatori diversi possono adottare criteri o livelli di attenzione differenti;
- non è sempre ripetibile nello stesso modo, specialmente in momenti diversi o sotto pressione;
- può essere influenzato da fattori esterni, come fretta, distrazione o carichi di lavoro elevati.

Il controllo automatico, al contrario, introduce un meccanismo più stabile:

- segue regole esplicite e definite in modo formale;
- viene eseguito sempre nelle stesse condizioni operative;
- produce risultati oggettivi, ripetibili e verificabili nel tempo.

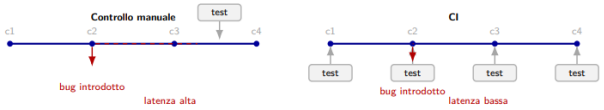
**L'idea alla base della CI:** La CI nasce con l'obiettivo di rendere automatiche le verifiche sul codice durante il suo sviluppo. L'idea di fondo è che ogni modifica introdotta nel progetto debba essere controllata in modo sistematico, e non soltanto in maniera occasionale o informale. In questo modo, la verifica diventa una parte integrante del processo di sviluppo del software, e non un'attività opzionale affidata alla sola buona volontà.




**CI:** CI è una pratica di sviluppo software che rende l'integrazione delle modifiche un'attività frequente e controllata. In un processo basato sulla CI:

- le modifiche vengono integrate frequentemente nel progetto;
- il sistema viene verificato automaticamente ad ogni integrazione.

L'obiettivo principale è ridurre il tempo che intercorre tra l'introduzione di un problema e la sua individuazione.




**Cosa non è la CI:** La CI non sostituisce le attività di progettazione del software né la progettazione dei test automatici:

- richiede comunque decisioni progettuali corrette;
- si basa sulla presenza di test adeguati.

La CI non garantisce, da sola, l'assenza di errori nel software:

- verifica solo ciò che viene effettivamente controllato;
- non intercetta problemi non coperti dai test.

La CI non è legata a una tecnologia o a una piattaforma specifica, ma rappresenta una pratica di processo indipendente dagli strumenti utilizzati.

**CI come estensione del testing:** I test automatici definiscono cosa deve essere verificato nel sistema:

- descrivono il comportamento atteso del codice;
- formalizzano le condizioni di correttezza.

La CI stabilisce quando tali verifiche devono avvenire:

- ad ogni integrazione di una modifica;
- senza dipendere da decisioni manuali.

Il contenuto dei test rimane invariato:

- ciò che cambia è la frequenza di esecuzione;
- le verifiche diventano sistematiche e ripetibili nel tempo.

**Quando avviene la verifica:** La verifica viene eseguita ogni volta che il codice del progetto viene modificato:

- in seguito all'introduzione di nuove funzionalità;
- dopo correzioni o refactoring del codice esistente.

In pratica, ciò avviene tipicamente quando una modifica viene condivisa nel repository remoto (ad esempio con push):

- il sistema intercetta l'aggiornamento del codice;
- la verifica viene avviata senza azioni esplicite da parte degli sviluppatori.

L'intero processo avviene in modo automatico:

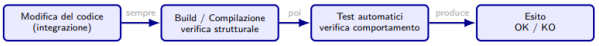
- l'esecuzione segue sempre le stesse regole;
- i risultati sono ripetibili e indipendenti dalle persone.
  **Collegamento con Git:** Nel flusso di lavoro visto con Git, una modifica diventa “parte della storia” quando viene registrata con un commit. In un contesto collaborativo, i commit vengono poi condivisi su un repository remoto, rendendo la modifica visibile anche ad altri. La CI si inserisce esattamente in questo passaggio:
- ogni modifica condivisa può attivare automaticamente una verifica;
- build e test vengono eseguiti con regole sempre uguali per tutti;
- il risultato rende immediatamente chiaro se l'integrazione è “sana”.

In questo modo, la verifica non dipende più da scelte individuali, ma diventa una proprietà del processo.

**Verifiche minime:** Una configurazione di CI minimale esegue un insieme di verifiche fondamentali per controllare la correttezza del progetto. In particolare, il sistema verifica che:

- il progetto possa essere compilato correttamente, senza errori strutturali o di dipendenze;
- i test automatici definiti vengano eseguiti e passino senza segnalare anomalie.

Queste verifiche di base permettono di intercettare molti problemi comuni già nelle prime fasi di integrazione.




**Errori di compilazione:** Un errore di compilazione segnala la presenza di un problema strutturale nel codice del progetto. Questo tipo di errore può dipendere, ad esempio, da:

- errori sintattici introdotti durante la scrittura del codice;
- incoerenze tra interfacce o firme dei metodi;
- dipendenze mancanti, errate o incompatibili.

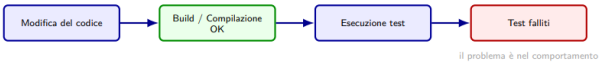
La CI segnala questi problemi in modo immediato, prima che possano propagarsi o influenzare altre parti del sistema.




**Fallimento dei test:** Un test che fallisce indica che il comportamento del sistema non soddisfa più le condizioni attese o dichiarate dai test. Questo può segnalare, ad esempio:

- una regressione, cioè un comportamento corretto in passato che non lo è più dopo una modifica;
- un errore logico introdotto durante lo sviluppo;
- un cambiamento voluto che richiede un aggiornamento dei test per riflettere il nuovo comportamento atteso.

La CI rende questi fallimenti immediatamente visibili, permettendo di intervenire prima che il problema si diffonda.




**Esempio (UniManager): individuare una regressione subito**

Si consideri una modifica al progetto UniManager, ad esempio sul calcolo della media dei voti di uno studente. La modifica viene integrata nel repository e attiva automaticamente la pipeline di CI:

- la compilazione verifica che la struttura del progetto sia consistente;
- i test controllano che il comportamento atteso sia ancora valido.

In assenza di CI:

- l'errore può emergere molto più tardi, ad esempio durante l'uso di quella funzionalità in un altro modulo;
- individuare la causa risulta più difficile, perché il codice è già cambiato più volte.

Con la CI:

- il test fallisce subito dopo l'integrazione della modifica;
- la regressione viene associata a una modifica recente;
- il problema diventa visibile nell'arco di pochi secondi.

L'obiettivo non è evitare ogni errore, ma ridurre drasticamente il tempo che intercorre tra l'introduzione di un problema e la sua individuazione.

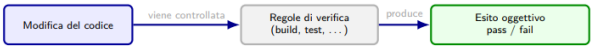
**Esito della verifica:** Ogni esecuzione della CI produce un esito chiaro e non ambiguo. In particolare, la verifica può risultare:

- **superata**, quando tutte le condizioni definite (build e test) sono soddisfatte;
- **fallita**, quando almeno una di tali condizioni non viene rispettata.

L'esito della verifica è oggettivo e riproducibile nel tempo, poiché deriva dall'applicazione automatica delle stesse regole.




**La CI come guardiano automatico:** La CI non prende decisioni progettuali e non valuta la qualità architetturale di una scelta. Non interpreta le intenzioni degli sviluppatori e non stabilisce se una modifica sia “giusta” dal punto di vista del dominio. Si limita a confrontare lo stato del progetto con un insieme di regole di verifica definite in anticipo (ad esempio compilazione e test), producendo un esito oggettivo.




**Impatto della CI sul processo:** La CI rende la verifica una proprietà del processo, non una scelta individuale. Effetto principale: feedback rapido sulle modifiche condivise.

- i problemi emergono subito dopo l'introduzione di una modifica;
- diventa più semplice risalire alla causa del problema.

Riduce il rischio di regressioni che si accumulano nel tempo:

- i comportamenti già verificati vengono controllati continuamente;
- le modifiche non corrette vengono intercettate prima.

Migliora l'affidabilità complessiva del progetto:

- le verifiche vengono eseguite sempre nello stesso modo;
- la qualità diventa più osservabile e meno soggettiva.

**Limiti e responsabilità:** La CI verifica solo ciò che è stato esplicitamente definito (build, test, altri check). Una pipeline “verde” non dimostra l'assenza totale di difetti:

- test incompleti possono nascondere problemi reali;
- alcune anomalie emergono solo in esercizio.

La CI non sostituisce la progettazione software:

- non corregge scelte architetturali sbagliate;
- non elimina la necessità di ragionare sul dominio applicativo.

Responsabilità condivisa del team:

- mantenere test significativi;
- migliorare progressivamente le regole di verifica.

**Dai principi agli strumenti:** La CI nasce come insieme di principi, indipendenti da qualunque tecnologia specifica. Nella pratica, tali principi vengono realizzati tramite strumenti che automatizzano le verifiche viste finora. Il ruolo di questi strumenti è operativo:

- eseguire automaticamente build e test;
- fornire un esito chiaro e immediato;
- rendere il processo ripetibile e sistematico.

> **Esempi di piattaforme di CI**
>
> Nel tempo sono nate diverse piattaforme per supportare la CI. Alcuni esempi diffusi sono:
>
> - Jenkins;
> - GitLab CI;
> - GitHub Actions.
>
> Queste piattaforme svolgono lo stesso ruolo concettuale:

- automatizzano le verifiche;
- segnalano l'esito delle integrazioni.

**Un esempio concreto di utilizzo**

Si consideri un progetto software con:

- una fase di compilazione;
- una suite di test automatici.

A ogni modifica del codice, la piattaforma di CI:

- compila automaticamente il progetto;
- esegue i test definiti;
- produce un esito complessivo.

Lo sviluppatore non deve eseguire manualmente queste operazioni ogni volta.

**GitHub Actions**

GitHub Actions è una delle piattaforme che permettono di realizzare la CI. In questo contesto, viene considerata semplicemente come un esempio concreto di strumento. I dettagli tecnici di configurazione non rientrano negli obiettivi di questa lezione:

- ciò che conta è il ruolo svolto;
- non il modo specifico in cui viene configurata.

**Stato della verifica:** Ogni esecuzione della CI produce uno stato di verifica associato alla modifica effettuata. Questo stato riassume l'esito delle verifiche automatiche:

- build e test superati;
- oppure presenza di errori.

Nella pratica, questo stato viene mostrato attraverso indicatori visivi immediati:

- ✓ Verde — verifica superata;
- × Rosso — verifica fallita;
- ⟳ Giallo — verifica in corso.

Questi segnali forniscono un colpo d'occhio sullo stato di salute del progetto in quel momento, senza garantire l'assenza totale di problemi.

**Riepilogo:**

- La CI rende sistematiche verifiche che altrimenti dipenderebbero dalla disciplina manuale del team.
- Il valore principale della CI è il feedback rapido su modifiche condivise.
- Build e test automatici sono i primi controlli da integrare; altri check possono essere aggiunti in modo progressivo.
- La CI collega in modo operativo Git, testing automatico e collaborazione.
- Gli strumenti (come GitHub Actions) sono mezzi per realizzare la pratica, non il centro concettuale della lezione.

## Manutenzione ed evoluzione del software

Una convinzione tipica (soprattutto all'inizio) è che il progetto “finisca” quando il software funziona. In realtà, il rilascio è spesso il momento in cui il sistema inizia davvero a vivere:

- arrivano utenti reali con bisogni reali,

- emergono casi limite non previsti,
- cambiano vincoli tecnici e contesti d'uso.

Per questo motivo, una parte rilevante dei costi e del lavoro si sposta su:

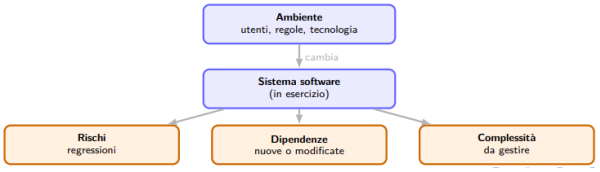
- correzioni,
- adattamenti,
- miglioramenti qualitativi,
- nuove funzionalità.

L'ingegneria del software serve a rendere questi cambiamenti gestibili, non “eroici”.

Il software non è un oggetto statico: è un sistema che opera in un ambiente che cambia. Cambiano le persone (utenti e team), cambiano le regole, cambiano gli strumenti. Ogni cambiamento introduce inevitabilmente:

- nuovi rischi di regressione,
- nuove dipendenze,
- nuove complessità da controllare.

Una buona progettazione non elimina il cambiamento, ma riduce il costo di “pagare” quel cambiamento nel tempo.




**Che cos'è la manutenzione del software:** Per manutenzione intendiamo le attività svolte dopo il rilascio per mantenere il sistema:

- corretto rispetto ai requisiti,
- compatibile con il contesto,
- utilizzabile e affidabile per gli utenti,
- sostenibile per chi lo mantiene.

Non è solo “fixare bug”: spesso significa anche migliorare il design, ridurre debito tecnico, rafforzare i test e rendere la pipeline più robusta. Il punto chiave è che la manutenzione è parte naturale del ciclo di vita: se la ignoriamo, il progetto tende a deteriorarsi.

**Le quattro forme di manutenzione:** Tradizionalmente distinguiamo quattro forme, utili per capire perché stiamo intervenendo:

- **Correttiva:** eliminare malfunzionamenti (bug).
- **Adattativa:** adeguarsi a cambiamenti di piattaforme, librerie, dipendenze.
- **Perfettiva:** migliorare qualità interna (prestazioni, leggibilità, struttura) senza nuove funzionalità.
- **Evolutiva:** introdurre nuove funzionalità o estensioni del sistema.

Questa classificazione è pratica perché aiuta a prevedere:

- impatto su test e architettura,
- rischio di regressioni,
- priorità e pianificazione.




Le quattro forme descrivono **motivazioni diverse** di intervento: da correzioni locali a cambiamenti che richiedono pianificazione, test e aggiornamento della documentazione.

**Manutenzione correttiva:** La manutenzione correttiva interviene quando il sistema non si comporta come atteso:

- bug funzionali,
- errori di validazione,
- crash, eccezioni non gestite,
- comportamenti incoerenti in casi limite.

Spesso nasce da:

- segnalazioni degli utenti,
- ticket e issue interne,
- test automatici che iniziano a fallire,
- problemi in produzione.

Anche un fix “piccolo” può essere rischioso se il sistema è accoppiato: per questo i test sono cruciali.

**Manutenzione adattativa:** La manutenzione adattativa si verifica quando il contesto cambia, non (necessariamente) i requisiti. Esempi tipici:

- aggiornamento di una libreria con API diversa,
- migrazione a una nuova versione del database,
- cambiamento di sistema operativo o ambiente di deploy,
- nuovi vincoli di sicurezza o autenticazione.

Qui diventa evidente il valore di una buona architettura: separare dominio e infrastruttura riduce quanto “si propaga” l'impatto delle dipendenze.

**Manutenzione perfettiva:** La manutenzione perfettiva migliora la qualità del sistema senza cambiare le funzionalità “visibili”. È spesso associata a:

- refactoring e semplificazione del codice,
- riduzione del debito tecnico accumulato,
- miglioramenti prestazionali mirati,
- riorganizzazione di moduli per aumentare coesione e ridurre accoppiamento.

È facile rimandarla perché “non aggiunge feature”, ma nel lungo periodo è ciò che rende il progetto modificabile e quindi più economico.

**Manutenzione evolutiva:** La manutenzione evolutiva introduce nuove funzionalità o estende il comportamento del sistema. È la forma che più coinvolge tutto il ciclo:

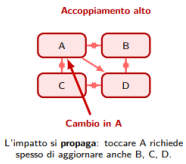
- aggiornamento requisiti,
- aggiornamento modelli e design,
- nuovi test e revisione di quelli esistenti,
- aggiornamento documentazione e release notes.

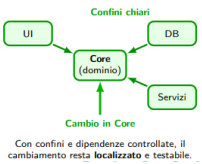
È anche quella dove il rischio di “rompere qualcosa” è più alto, perché cambiano casi d'uso e flussi. Per questo serve un processo disciplinato e una base di test affidabile.

**Architettura e costo del cambiamento:** Il costo di un cambiamento non dipende solo dalla quantità di codice da scrivere, ma da quanto lontano si propaga l'impatto. Se le responsabilità sono mescolate e le dipendenze sono “ovunque”:

- un fix locale può richiedere modifiche in molti punti,
- diventa difficile capire cosa si sta rompendo,
- aumenta la paura di cambiare.

I principi visti nel corso (SRP, DIP, separazione dei livelli, confini architetturali) servono proprio a rendere i cambiamenti localizzati e quindi più gestibili.




**Dal problema al codice:** Quando emerge un problema (da un utente, da un log, da un test che fallisce), la tentazione naturale è “aprire l'IDE e correggere subito”. In un progetto serio, invece, conviene fermarsi un attimo e seguire un ordine:

- tracciare il problema in modo esplicito (issue), così da renderlo visibile e discutibile dal team;
- riprodurre il difetto in modo controllato, per evitare di lavorare su ipotesi vaghe o non verificabili;
- capire la causa (root cause), distinguendo chiaramente il sintomo osservato da ciò che lo genera.

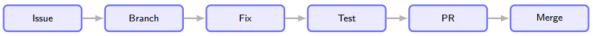
Questo passaggio iniziale sembra “tempo perso”, ma in realtà riduce due rischi classici:

- correggere la cosa sbagliata (fix che maschera il sintomo ma non risolve il problema reale);
- introdurre regressioni non previste, perché si interviene senza sapere quali parti del sistema sono coinvolte.

In altre parole: prima comprendiamo e delimitiamo il problema, poi lo risolviamo in modo verificabile.

**Il ciclo disciplinato di modifica:** Un flusso robusto e ripetibile per introdurre un cambiamento è: issue → branch → fix → test → PR → merge. Dopo aver sviluppato e verificato la modifica su un branch dedicato, il cambiamento non viene integrato direttamente, ma passa attraverso una richiesta di integrazione (Pull Request). La Pull Request rende il cambiamento:

- visibile agli altri membri del team,
- revisionabile prima dell'integrazione,
- verificabile automaticamente tramite test e CI.




**Il ruolo centrale dei test:** I test non servono solo a “controllare” il codice alla fine: sono parte del lavoro di manutenzione. Un modello pratico è:

- un test che fallisce riproduce il bug in modo oggettivo,
- il fix rende il test verde,
- il test rimane come garanzia contro future regressioni.

Se correggiamo un bug senza aggiungere o aggiornare test, stiamo lasciando aperta la porta al ritorno dello stesso problema. Nel lungo periodo, una buona suite di test è ciò che rende possibile cambiare con fiducia.

**CI come rete di sicurezza:** La Continuous Integration automatizza controlli fondamentali:

- build,
- test,
- eventualmente linting e analisi statica.

Il valore della CI non è “fare i test più velocemente”, ma garantire che ogni modifica passi da una verifica ripetibile e indipendente. Collegamento con le lezioni precedenti:

- i test riducono regressioni,
- la CI impedisce che regressioni entrino nel ramo principale,
- la review riduce errori di design e di implementazione.

**Versioning semantico:** Per comunicare chiaramente il tipo di cambiamento introdotto si usa spesso il Semantic Versioning. Formato: MAJOR.MINOR.PATCH

- **MAJOR:** cambiamenti incompatibili (breaking changes).
- **MINOR:** nuove funzionalità retrocompatibili.
- **PATCH:** correzioni e piccoli fix senza modifiche di API/comportamento atteso.

È una convenzione semplice, ma estremamente utile quando:

- più team dipendono dallo stesso componente,
- dobbiamo pianificare aggiornamenti,
- vogliamo evitare “sorprese” agli utenti.




**Perché il versioning conta davvero:** Il versioning non è solo un dettaglio formale, ma un vero e proprio linguaggio comune tra chi sviluppa software e chi lo utilizza o lo integra. Attraverso il numero di versione, il progetto comunica in modo sintetico il tipo di cambiamento introdotto e il livello di rischio associato. In pratica, il versioning aiuta a rispondere rapidamente a domande molto concrete:

- Questo aggiornamento è sicuro da applicare senza effetti collaterali?
- Potrebbe rompere la compatibilità con codice o servizi esistenti?
- Introduce nuove funzionalità oppure solo correzioni e miglioramenti interni?

Questo diventa ancora più importante quando un progetto dipende da altri pacchetti o servizi: una gestione chiara e coerente delle versioni riduce conflitti, ambiguità e tempi di diagnosi in caso di problemi. In

altre parole, un buon versioning rende i cambiamenti prevedibili e aiuta team e utenti a prendere decisioni informate sugli aggiornamenti.

**Gestione delle release:** Una release non è semplicemente “fare merge” sul ramo principale, ma un momento preciso in cui un insieme di cambiamenti viene reso adottabile da utenti e altri sistemi. Preparare una release significa decidere consapevolmente che cosa entra e che cosa resta fuori in quel rilascio. In pratica, una gestione corretta delle release include:

- la selezione delle issue incluse, spesso organizzate in milestone o sprint target;
- una fase di verifica della qualità, basata su test automatici, CI e controlli aggiuntivi;
- la documentazione dei cambiamenti, così che gli utenti possano capire cosa è cambiato.

A supporto di questo processo si usano strumenti e pratiche consolidate:

- un changelog per tenere traccia delle modifiche nel tempo;
- delle release notes orientate all'utente e all'impatto pratico del rilascio;
- automazioni per creare tag di versione, generare artefatti e pubblicare il rilascio.

Una buona gestione delle release riduce sorprese e rende il ciclo di evoluzione più prevedibile.

> **Caso di studio: UniManager — una feature nuova**
>
> Per rendere concreto il concetto di evoluzione del software, consideriamo l'introduzione di una nuova funzionalità in UniManager, ad esempio:
>
> - il rifiuto del voto da parte dello studente,
> - oppure la visualizzazione della media ponderata degli esami sostenuti.
>
> Si tratta di funzionalità realistiche e plausibili, che riflettono esigenze comuni nei sistemi informativi universitari. Il punto centrale, però, non è la feature in sé, ma osservare come anche un cambiamento apparentemente “semplice” abbia un impatto su più livelli del sistema. In particolare, l'evoluzione coinvolge:
>
> - i requisiti, che devono essere chiariti e aggiornati;
> - il modello e il design, che possono richiedere nuove responsabilità;
> - l'architettura, per verificare che i confini restino coerenti;
> - i test, che devono proteggere il nuovo comportamento;
> - la versione e la release, per comunicare correttamente il cambiamento.

**Impatto sui requisiti:** Aggiungere una funzionalità significa prima di tutto chiarire:

- cosa deve essere possibile fare,
- in quali condizioni,
- con quali vincoli e casi limite.

Esempio (rifiuto del voto):

- quando lo studente può rifiutare?
- quante volte è consentito?
- cosa succede allo stato del verbale?

Requisiti chiari diventano la base per:

- progettare correttamente,
- scrivere test verificabili,
- evitare ambiguità tra team e utenti.

**Impatto su UML e architettura:** La nuova feature spesso richiede di aggiornare:

- diagrammi (stati, sequenze, classi) se li stiamo usando per documentare comportamento e responsabilità,
- confini architetturali: chi decide cosa? dove sta la regola?

Obiettivo: evitare di “spargere” logica di dominio ovunque. Una buona domanda guida è: questa regola è dominio (stabile) o infrastruttura (variabile)? Se i confini sono chiari, l'evoluzione diventa più locale e

meno rischiosa.

**Impatto su test e CI:** Una feature nuova implica:

- nuovi test (unità e/o integrazione),
- aggiornamento di test esistenti se cambiano regole o flussi.

I test vanno pensati anche come protezione futura: la feature di oggi diventa la regressione di domani se non viene coperta. La CI deve continuare a garantire che ogni PR:

- compili,
- passi i test,
- non introduca rotture evidenti.

**Versione e rilascio della nuova funzionalità:** Se la nuova funzionalità è compatibile con quanto esisteva prima, tipicamente incrementiamo la MINOR: ad esempio 1.0.0 → 1.1.0. A quel punto una release “fatta bene” include:

- changelog con le issue chiuse,
- note chiare su eventuali modifiche di comportamento,
- indicazioni di migrazione se serve.

Il valore aggiunto è la prevedibilità: gli utenti sanno cosa aspettarsi e quando adottare l'aggiornamento.

**Evoluzione continua:** La manutenzione non è un tema separato: è il punto in cui tutto ciò che abbiamo visto converge. Requisiti chiari riducono ambiguità; architettura pulita riduce propagazione dei cambiamenti. Test e CI permettono di cambiare senza paura, documentazione riduce attrito e onboarding. Un progetto che cura questi aspetti:

- dura più a lungo,
- cresce in modo controllato,
- rimane comprensibile anche dopo anni.

**Messaggio finale:** Costruire buon software non significa solo farlo funzionare oggi. Significa progettarlo e mantenerlo in modo che sia ancora:

- comprensibile,
- modificabile,
- affidabile, domani, quando i requisiti, il contesto e le persone saranno cambiati.

L'evoluzione non è un incidente del percorso, ma la naturale prosecuzione di ogni progetto reale. Messaggio finale: il buon software si progetta pensando al cambiamento.

**Riepilogo:**

- Il software va pensato come un sistema vivo: il rilascio iniziale apre una fase di manutenzione ed evoluzione continua.
- I diversi tipi di manutenzione hanno obiettivi e rischi diversi, quindi richiedono priorità e strategie operative differenti.
- Test automatici, CI e workflow disciplinato sono ciò che rende i cambiamenti più frequenti e meno rischiosi.
- Architettura, refactoring e documentazione incidono direttamente sul costo futuro della manutenzione, non solo sulla qualità immediata.
- Il versioning semantico aiuta a comunicare l'impatto dei rilasci e a rendere l'evoluzione più prevedibile per chi usa il software.
- La visione finale del corso è che il buon software si progetta, testa e documenta pensando esplicitamente al cambiamento.